Non-Structural Subtype Entailment Is Decidable

ANONYMOUS AUTHOR(S)

Subtype entailment asks whether every solution of a finite system of subtype constraints satisfies a further inequality. It underlies constraint simplification in type inference. We prove decidability for non-structural subtyping with bottom, top, and one constructor of any finite arity that is covariant or contravariant in each argument, over arbitrary, regular, and finite trees. This answers TLCA Problem 16, which Jakob Rehof posed in 1996 for a binary constructor of unspecified variance, for every binary variance and each tree domain, including the covariant reading used in the literature.

When the constraints have a solution, a failure of entailment is witnessed by a finite path, written as a word over argument positions. Such a word is a counterexample exactly when a finite-state reader built from the constraints rejects it and the word has none of the periodic comparisons between its suffixes that the reader forbids. Shortening a word can create such comparisons, so ordinary pumping does not bound the shortest counterexample. For the readers that arise from subtype constraints, every forbidden comparison ends no later than the point at which the reader first rejects the word. This yields a counterexample whose part after the first rejection is short, and hence whose hierarchy of repeated blocks is shallow. Bounding the number of distinct blocks at each level and shortening all blocks of a level at once then bounds the length of a counterexample triply exponentially in the number of variables. Countermodel transfers extend the procedure to regular and finite trees. A search that keeps only a finite summary of one level of the hierarchy at a time places entailment in 2-EXPSPACE and 3-EXPTIME in each domain, and in PSPACE for a unary constructor. For a covariant constructor of each fixed arity at least two, entailment is PSPACE-hard in each domain, and shortest counterexamples can be exponentially long in the number of variables. For flat constraints, the three decision procedures, their correctness, and the combinatorial content of the complexity bounds are mechanized in Lean 4.

CCS Concepts: • **Theory of computation** → **Formal languages and automata theory**; **Type theory**; *Automated reasoning*.

Additional Key Words and Phrases: subtyping, subtype constraints, entailment, non-structural subtyping, variance, word equations, periodicity, decidability, mechanized proof

1 Introduction

Type inference with subtyping expresses the typing requirements of a program as *subtype constraints* $t \le t'$ between type expressions with unknowns [Aiken and Wimmers 1993; Eifrig et al. 1995; Fuh and Mishra 1990; Mitchell 1991; Pottier 1998a]. An inferred *constrained type* consists of a type and constraints on its variables. Such constraint sets grow quickly, so simplification removes redundant constraints [Pottier 1996, 2001; Rehof 1997; Trifonov and Smith 1996]. For example, suppose a constrained type includes $u \le a \to b$, $a \to b \le v$, and $u \le v$. The first two inequalities imply the third by transitivity, so a simplifier can remove $u \le v$ without changing the solutions. They do not imply $v \le u$: the assignment $u = \bot$, $v = \top$, and $a = b = \bot$ satisfies the first two inequalities and refutes the reversed query.

The general question is *entailment*: given a conjunction ϕ and an inequality $x \le y$, does every solution of ϕ satisfy $x \le y$? We write $\phi \models x \le y$. Entailment is a universally quantified implication. It differs from satisfiability, which asks for only one solution. For structural subtyping, which relates trees of the same shape, entailment is coNP-complete over finite trees and PSPACE-complete over possibly infinite trees [Henglein and Rehof 1997, 1998]. Bottom and top make subtyping non-structural: $\bot \le t \le \top$ can relate trees of different shapes.

TLCA Problem 16, submitted by Jakob Rehof in 1996, asks whether entailment of subtype constraints over these non-structurally ordered trees is decidable. Its simple type expressions are built from bottom, top, and a binary constructor [Rehof 1996]. The statement does not specify

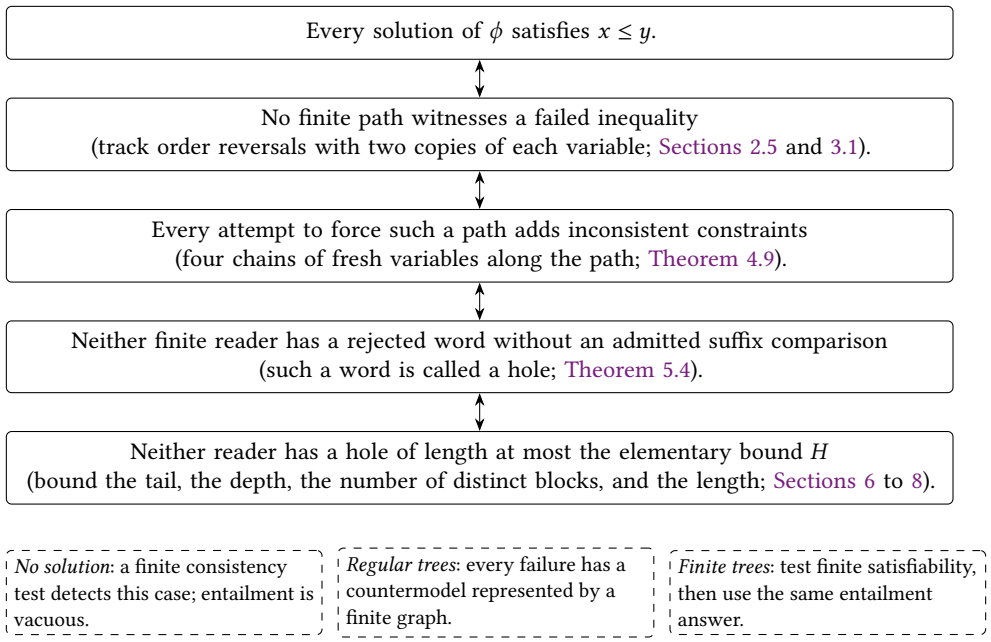


Fig. 1. The reduction for a satisfiable system ϕ. Each double arrow is an equivalence. The dashed boxes explain the satisfiability test and the transfers to regular and finite trees.

variance; the literature treats the constructor as covariant [Niehren and Priesnitz 2003], hereafter NP03. That case is PSPACE-hard [Henglein and Rehof 1998] (Theorem 10.7 gives another proof) and has been described as open in subsequent work [Dudenhefner et al. 2017; Jiang et al. 2025; Niehren and Priesnitz 2003]. We decide non-structural subtype entailment (NSSE) for every variance of one constructor.

Theorem 1.1 (Main theorem). *For every finite arity n and variance map $c : \mathbb{D} \to \{0, 1\}$, where $\mathbb{D} = \{0, \dots, n-1\}$, non-structural subtype entailment over $\{\bot, f, \top\}$, where f is the single constructor of arity n, is decidable over arbitrary trees, regular trees, and finite trees. The decision procedures are uniform in n, c.*

The procedures take flat literals $z \le f(a_i)_i$, $f(a_i)_i \le z$, $z = \bot$, and $z = \top$, with variables z, a_i, and a variable query $x \le y$. Proposition 2.2 gives the reduction from general finite systems at positive arity and the direct enumeration at arity zero. The mechanized procedures take flat systems; flattening is proved here but not mechanized.

The theorem thus answers TLCA Problem 16 for every choice of binary variance and for each of the three tree domains. Binary covariance is $c(0) = c(1) = 0$; arrows have $c(0) = 1, c(1) = 0$, with a contravariant domain and a covariant codomain. Regular entailment coincides with arbitrary-tree entailment. Finite entailment also coincides with it whenever a finite solution exists; otherwise finite entailment is vacuous. A *countermodel* is a solution that refutes the query. Figure 1 summarizes the proof.

Why Entailment Is Hard and What Makes It Decidable. A failed tree inequality can be witnessed by a finite path, written as a word over argument positions. After variance is accounted for, a word w is *left unsafe* if, in some solution, x has a top leaf on a prefix of w but y has none, and *right unsafe*

if y has a bottom leaf on a prefix of w but x has none. Entailment fails exactly when some word is left or right unsafe.

Suffix comparisons arise when constraints connect subtrees at different cuts of this path. In the covariant case, let $s < e$ be cuts of w, and suppose that some input variable lies above the subtree of x at cut e and below the subtree of y at cut s. Every top leaf of x on w then lies at or beyond cut e, so the subtree of x at cut e has a top leaf on a prefix of the remaining suffix $w[e:]$. The variable passes such a leaf up to the subtree of y at cut s, again on a prefix of $w[e:]$. If $w[e:]$ is a prefix of $w[s:]$, this leaf of y lies on w, so w is not left unsafe after all. A pair of cuts $s < e$ with $w[e:]$ a prefix of $w[s:]$ is a *comparison* of w; equivalently, $w[s:]$ has period $e - s$. Under variance, comparing a subtree of x with a subtree of its own signed copy gives an analogous obstruction.

For example, take a binary covariant constructor, the single literal $f(y, x) \le y$, and the query $x \le y$. In every solution $y \neq \bot$, and whenever y is a constructor, it lies below its own 0-child and x lies below its 1-child. Hence, if y has a bottom leaf on a word w, then $w = 0^i 1z$, the nodes of y along 0^i are constructors, and the subtree of y at cut $i + 1$ lies above x. This is the order dual of the situation above, with cuts $0 < i + 1$ and x itself as the connecting variable: the bottom leaf of y lies in that subtree on a prefix of z, and it forces a bottom leaf of x on a prefix of z. If $(0, i + 1)$ is a comparison, that is, if z is a prefix of w, then x also has a bottom leaf on w, and w is not right unsafe. Conversely, a word $0^i 1z$ for which $(0, i+1)$ is not a comparison is right unsafe (Section 14.2). The right-unsafe words must therefore avoid a periodic comparison between their suffixes, and they do not form a regular set: those in 0^*10^* are exactly the words $0^i 10^j$ with $j > i$.

A finite *reader* records the constraints along a word. It *rejects* a word when none of the tests at single cuts rules the word out, so rejection marks a candidate counterexample word. A comparison is *admitted* when its two reader states certify an obstruction of the kind just described. A *hole* is a rejected word with no admitted comparison. For a satisfiable system, the holes of two such readers are exactly the left-unsafe and the right-unsafe words. Ordinary loop deletion can preserve the final reader state while creating a new admitted comparison (Example 5.7). Preserving the run is therefore insufficient.

Readers constructed from subtype constraints satisfy a *rejected-path property*: the rejected cuts of a word form an interval, and every admitted comparison ends no later than the first rejected cut. The reason is semantic. In the left label reader, at a rejected cut, no variable that the constraints place above the subtree of x is bounded above by a constructor. From the next cut on, the constraints therefore bound the subtree of x by no variable at all, whereas every admission needs such a bound at its later cut (Section 5.3). The right label reader is dual. Table 2 records which bounds need this property.

From Constraints to a Finite Search. To handle variance, normalization exchanges $\bot$ and $\top$ at paths of odd polarity, that is, paths with an odd number of contravariant edges. Positive and negative copies of each variable then give a covariant system; the solutions that matter assign dual trees to the two copies. We obtain such a solution by selecting labels on a constructor shape common to both copies. Four chains of fresh constraints along the candidate word, called *spines*, impose the two unsafety conditions and their signed counterparts. Classifying the inconsistencies of this extension gives the reader. For k input variables, a state stores two sets of the $2k$ signed variables and one bit recording an earlier obstruction, so there are $N = 2 \cdot 4^{2k} = 2^{4k+1}$ states.

The finite search uses three bounds.

(1) *Tail* (Section 6). If a reader with the rejected-path property has a hole, then it has one whose *rejected tail*, the part after the first rejected cut, has length linear in N. If a cycle of rejected states allows a second letter, repeating the cycle and then taking the other letter breaks every

relevant period; otherwise continuation within rejected states is forced, and a long periodic tail can be truncated.

(2) *Depth* (Section 7). For every reader, a suffix comparison of a rejected word that joins equal states has gap at most that word's rejected-tail length. Cutting a word at the occurrences of its last letter, and repeating this on the resulting word of blocks, gives a hierarchy of block decompositions. The gap bound limits its depth, quadratically in N when the tail is linear in N.

(3) *Length* (Section 8). For every reader, a hole of bounded depth can be shortened. Merging interchangeable letters bounds the number of distinct blocks at each positive level, and the blocks of a level are then shortened simultaneously. Finite summaries preserve transitions, admissions, and block equalities.

Together, the three bounds give a hole whose length is triply exponential in k whenever some hole exists. For arbitrary readers, the remaining question is whether a computable rejected-tail bound exists (Section 14.2).

Contributions and Scope.

- A decision procedure for one constructor of any finite arity and variance over arbitrary, regular, and finite trees (Theorem 1.1).
- A reduction of non-entailment to holes of finite readers (Sections 3 to 5). Closure-based satisfiability characterizations are classical [Kozen et al. 1994; Palsberg and O'Keefe 1995; Palsberg et al. 1997]. The new semantic ingredients are a least constructor shape common to the two signed copies of each variable, the selection of labels on that shape to handle variance, and the four-spine analysis that defines the reader.
- Alphabet-generic bounds on tails, depth, and length (Sections 6 to 8); the latter two apply to every reader. For readers with the rejected-path property, they bound the length of a shortest hole doubly exponentially in a polynomial of N, hence triply exponentially in k for the label readers (Theorem 9.1).
- Countermodel transfers (Section 9 and Appendix D), including coincidence of finite and arbitrary-tree entailment whenever a finite solution exists. That solution excludes cycles in the base constraints; the spines preserve this exclusion and permit a finite countermodel.
- Complexity bounds in each domain (Section 10). A search that stores one level of the block hierarchy at a time places entailment in 2-EXPSPACE and 3-EXPTIME (Theorem 10.3); constructors of arity at most one admit stronger bounds (Appendix F). For a covariant constructor of every fixed arity $n \geq 2$, entailment is PSPACE-hard (Theorem 10.7), which for $n = 2$ reproves the non-structural hardness theorems of Henglein and Rehof [1998], and shortest holes can have length $2^{\Omega(k)}$ (Proposition 10.6).
- An end-to-end Lean 4 mechanization, for flat constraints, of the three procedures and their correctness, and of the combinatorial content of the complexity bounds: the summary search with its configuration bound, the one-letter bound, the hardness reduction, and the exponentially long holes (Section 11 and Appendix H).
- A counterexample to Proposition 16 of Niehren and Priesnitz [2003] (Appendix J); Section 12.1 explains which published proofs depend on that proposition.

The proof does not use the cap-automaton construction or its back-translation from Niehren and Priesnitz [2003]. An earlier version of this work proved the binary covariant case through a mechanized form of the forward cap-automaton characterization; Section 13 describes that proof and explains why we replaced it.

How to Read This Paper. Section 2 fixes the semantics and the input language. Sections 3 to 5 turn subtype constraints into a word problem. They build on one another and end with the label

readers and their rejected-path property. Sections 6 to 8 solve that word problem; their principal results are the tail, depth, and bounded-length theorems (Theorems 6.1, 7.7 and 8.1). From the earlier sections they use only the definitions of comparisons, readers, and holes and the rejected-path property, all in Section 5, so a reader interested mainly in the word combinatorics can start there. Section 9 assembles the decision procedures and the domain transfers. Section 10 replaces the explicit enumeration of words by a search over summaries; it uses the lettered monitors and summaries of Section 8, the reader construction of Section 5, and the statements of Section 9. Section 11 describes the Lean artifact, Section 12 discusses related work, Section 13 compares the present proof with the earlier binary proof, and Section 14 discusses open problems. The appendices supply the spine proofs, the preservation arguments for cores and compression, the domain transfers, the resource accounting and lower-bound details for the complexity results, and the correspondence with the Lean development; they end with a navigation table of proof distinctions and the counterexample to NP03's Proposition 16.

2 Background

2.1 Words and Trees

Fix an arity $n \in \mathbb{N}$, the child alphabet $\mathbb{D} = \{0, \dots, n-1\}$, and a variance map $c : \mathbb{D} \to \{0, 1\}$. Here 0 means covariant and 1 contravariant. The signature contains exactly $\bot$, $\top$, and one constructor f of arity n. A *cut* of a word w of length m is an integer $0 \le j \le m$; cut j separates its first j letters from the remaining suffix. Write $w[i]$ for its letter at position i, with indices starting at 0 and $0 \le i < m$. Words are finite; ε is the empty word, concatenation is juxtaposition, $u \preceq w$ means that u is a prefix of w, and $w[: j]$, $w[j : h]$, $w[j :]$ are its prefix, factor, and suffix at the indicated cuts. For example, over $\{0, 1\}$, $010[: 2] = 01$ and $010[1 :] = 10$. A positive integer ϖ is a *period* of w when $w[i] = w[i - \varpi]$ for $\varpi \le i < |w|$. For a set L of words, $\mathrm{pr}(L)$ is the set of prefixes of its members, and $\mathrm{pr}(v)$ abbreviates $\mathrm{pr}(\{v\})$. For a nonempty root word ρ, $\mathrm{pr}(\rho^*)$ thus consists of the prefixes of its finite powers; these are exactly the finite prefixes of ρ^ω.

Lemma 2.1 (Periodic prefixes). *For $\rho \neq \varepsilon$, $u \in \mathrm{pr}(\rho^*)$ iff $u \preceq \rho^\omega$ iff $u \preceq \rho u$.*

Proof. The first equivalence follows by choosing a sufficiently long power. If $u \preceq \rho^\omega$, then both u and ρu prefix the same infinite word. Conversely, $u \preceq \rho u$ either gives $u \preceq \rho$, or lets us cancel an initial ρ and repeat on a shorter word. □

The *polarity* of a path counts contravariant positions modulo two:

$$p(w) = \bigoplus_{j < |w|} c(w[j]), \qquad p(\varepsilon) = 0, \qquad p(uv) = p(u) \oplus p(v). \tag{1}$$

Occurrences are counted with multiplicity: if $c(0) = 1$ and $c(1) = 0$, then $p(0) = 1$, $p(00) = 0$, and $p(010) = 0$.

A *tree* is a partial labeling $t : \mathbb{D}^* \rightharpoonup \{\bot, f, \top\}$, with a defined root and prefix-closed domain, such that

$$wi \in \mathrm{dom}(t) \iff t(w) = f \qquad (i \in \mathbb{D}). \tag{2}$$

The *extreme labels* are $\bot$ and $\top$. They label leaves; an f-node has n children, so $f()$ is also a leaf at arity zero. Write $f(t_i)_{i \in \mathbb{D}}$ for a constructor tree. A subtree at η has labeling $w \mapsto t(\eta w)$, when η belongs to the domain. For example, the finite binary tree $f(\bot, \top)$ has domain $\{\varepsilon, 0, 1\}$. The full constructor tree $F = f(F, \dots, F)$ is infinite when $n > 0$ and is $f()$ when $n = 0$.

Let $\mathcal{T}$ contain all trees, $\mathcal{T}_{\mathrm{fin}}$ those with finite domain, and $\mathcal{T}_{\mathrm{reg}}$ those with finitely many distinct rooted subtrees. A tree is regular iff it is the unfolding of a finite labeled graph with n ordered outgoing edges at each constructor node; for the forward direction, take the graph of its finitely

many distinct subtrees. The tree F has a one-state graph. Every finite tree is regular, but F shows that the converse fails at positive arity. At arity zero, the only trees are $\bot, f(), \top$.

2.2 Non-Structural Subtyping

Order the labels by $\bot < f < \top$. Define

$$t \le_c u \quad \Longleftrightarrow \quad \forall w \in \mathrm{dom}(t) \cap \mathrm{dom}(u), \quad \begin{cases} t(w) \le u(w), & p(w) = 0, \\ u(w) \le t(w), & p(w) = 1. \end{cases} \tag{3}$$

For $c = 0$, write $\le$ and call the order *covariant*. The order is non-structural because $\bot \le_c t \le_c \top$ holds even for trees of different shapes. For binary products $c(0) = c(1) = 0$; for arrows $c(0) = 1, c(1) = 0$, and the usual rule is $f(a_0, a_1) \le_c f(b_0, b_1)$ iff $b_0 \le_c a_0$ and $a_1 \le_c b_1$. For instance, $f(\top, \bot) \le_c f(\bot, \top)$ for arrows.

For a relation R on trees, define the textbook one-step rule

$$\begin{aligned} \mathrm{Step}_c(R)(t,u) \quad \Longleftrightarrow \quad & t = \bot \ \vee \ u = \top \\ & \vee \ \exists \vec{a}, \vec{b} \ [t = f(\vec{a}) \wedge u = f(\vec{b}) \wedge \forall i \ \begin{cases} R(a_i, b_i), & c(i) = 0, \\ R(b_i, a_i), & c(i) = 1. \end{cases}] \end{aligned} \tag{4}$$

Then $\le_c = \nu R.\, \mathrm{Step}_c(R)$, the *greatest fixed point*: the largest relation preserved by this rule. To prove it, compare roots in (3) and apply (1) to the children; this shows that (3) is a fixed point. Conversely, if $R \subseteq \mathrm{Step}_c(R)$, induction on a common path proves (3) for every pair in R. A contravariant first letter exchanges the trees and toggles polarity; a leaf case leaves only the root comparison. The greatest fixed point is necessary: at positive arity $F \le_c F$, whereas the least fixed point cannot justify this comparison by finitely many unfoldings. On finite trees, the same rule is the usual inductive order. The relation is a partial order, as also follows from normalization and the pointwise characterization in (9).

2.3 Constraints, Solutions, and Entailment

Let V be a finite set of k variables containing x, y. A *flat constraint system* ϕ is a finite conjunction of literals

$$z \le f(a_i)_i, \qquad f(a_i)_i \le z, \qquad z = \bot, \qquad z = \top, \tag{5}$$

where each a_i is a variable. An assignment $\sigma : V \to \mathcal{D}$, for $\mathcal{D} \in \{\mathcal{T}, \mathcal{T}_{\mathrm{reg}}, \mathcal{T}_{\mathrm{fin}}\}$, is a *solution* if it satisfies all literals, interpreting inequalities by $\le_c$. Write $\mathrm{Sat}_{\mathcal{D},c}(\phi)$ for existence of a solution. Define

$$\phi \models_{\mathcal{D},c} x \le y \quad \Longleftrightarrow \quad \forall \sigma : V \to \mathcal{D}, \quad \sigma \models_c \phi \Longrightarrow \sigma(x) \le_c \sigma(y). \tag{6}$$

Omit $\mathcal{D}$ for arbitrary trees, and use reg and fin for the other domains; omit c as well in the covariant case. Satisfiability asks for one solution; entailment asks about every solution. A *countermodel* is a solution that violates the query. An unsatisfiable system entails every query. For example, $x \le f(a_i)_i$ and $f(a_i)_i \le y$ entail $x \le y$. With distinct variables they do not entail $y \le x$: set $x = \bot, y = \top$. For a binary covariant constructor, $f(x, x) \le x \le f(x, x)$ has a regular infinite solution but no finite solution. The query $x \le y$ is a variable inequality. It is accepted directly and is not a literal of (5); general systems and queries reduce to this form as follows.

Proposition 2.2 (Flattening). *At positive arity, any finite system of inequalities between finite type expressions, with a type-expression query, reduces to the flat syntax (5) and a variable query. Solutions correspond under projection in each of the three tree domains. At arity zero, such entailments are decidable by enumerating assignments in* $\{\bot, f(), \top\}$.

Proof. Give each nonvariable subexpression a fresh name. For a constructor subexpression, impose both flat bounds between its name and the constructor applied to the names of its children; for a constant, impose its constant literal. Antisymmetry forces each name to denote its subexpression. Replace each original inequality, and the query, by the variable inequality between the names of its two sides. The query is now a variable query.

A variable inequality $u \le v$ in the system is not a literal of (5), but at positive arity it can be expressed with fresh auxiliaries. Choose a child i, put u, v in the ith coordinates of vectors $\vec{a}, \vec{b}$ (exchanged if $c(i) = 1$), use a common filler variable in the other coordinates, and add the two literals $f(\vec{a}) \le z \le f(\vec{b})$ with a fresh *sandwich variable* z. A solution extends by evaluating subexpressions and taking z to be $f(\vec{a})$. Conversely, the two literals give $f(\vec{a}) \le_c f(\vec{b})$, and decomposing it at child i gives $u \le_c v$. These extensions use only constants and finitely many constructor applications, which preserve finite and regular trees. At arity zero, all expressions evaluate in the displayed three-element chain, and the sandwich encoding is unavailable.[1] □

This reduction explains the mathematical scope; the exported formal procedures take the flat syntax directly (Appendix H).

2.4 Normalization

Normalization moves all order reversals from the comparison rule into the leaf labels: after it, every path is compared covariantly. Define *tree duality* $\mathfrak{d}(t)$ by exchanging $\bot$ and $\top$ and leaving every f-label unchanged, including the nullary constructor $f()$. For $q \in \{0, 1\}$, define $H_q t$ on the same domain as t, exchanging $\bot$ and $\top$ in its label at w precisely when $q \oplus p(w) = 1$. The following lemma shows that H_0 turns $\le_c$ into the covariant order.

Lemma 2.3 (Normalization).

$$H_q f(t_i)_i = f(H_{q \oplus c(i)} t_i)_i, \qquad H_1 = \mathfrak{d} H_0, \qquad H_0 H_0 = \mathrm{id}, \tag{7}$$

$$t \le_c u \iff H_0 t \le H_0 u. \tag{8}$$

Proof. Exchanging the extreme labels preserves (2). At a child path iw, (1) gives the sign in (7); flipping twice proves the involution identities. At every common path, exchanging the extreme labels reverses their order. Thus the comparison in (3) is exactly the covariant comparison after applying H_0, proving (8). In particular, H_0 is an order isomorphism to the covariant tree order. Both H_q and $\mathfrak{d}$ preserve finite domains. They preserve regularity: in a finite graph, attach one polarity bit to each node and update it on an edge labeled i by xor with $c(i)$. □

2.5 Unsafe Words and Sides

For a covariant tree t, define its *constant leaf extension* $\widehat{t} : \mathbb{D}^* \to \{\bot, f, \top\}$ by following a word until its end or the first leaf, and then retaining that leaf's label. Thus $\widehat{t}(w) = \top$ exactly when some prefix of w is a $\top$-leaf of t; the analogous assertion holds for $\bot$.

Lemma 2.4 (Leaf-extension characterization). *The covariant order has the characterization*

$$t \le u \iff \forall w \quad \widehat{t}(w) \le \widehat{u}(w). \tag{9}$$

[1] Every flat literal then concerns one variable. Solution sets, even after existential projection, are products of unary sets; the order on $\bot < f() < \top$ is not such a product.

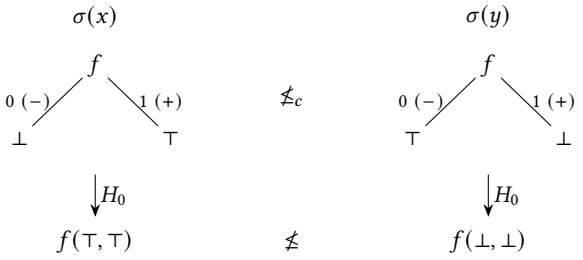


Fig. 2. The countermodel of Example 2.5. The argument edge 0 (the arrow's domain) reverses the order: $x \leq_c y$ would require $\top \leq_c \bot$ there. Normalization flips that edge's leaf labels, exposing the top-prefix and bottom-prefix failures at the same word 0.

Proof. The reverse implication follows by considering common domain paths. For the forward implication, follow both trees along w. Until a leaf is reached, labels compare by hypothesis. If the left tree stops at $\bot$, or the right at $\top$, the extended comparison holds forever. If the left stops at $\top$, the right must also be $\top$ at that common node; if the right stops at $\bot$, the left must also be $\bot$. Otherwise, both reach the end and the original comparison applies. □

For a tree t ordered by $\leq_c$, put

$$\mathrm{Pref}_c^+(t,w) :\Longleftrightarrow \widehat{H_0 t}(w) = \top, \qquad \mathrm{Pref}_c^-(t,w) :\Longleftrightarrow \widehat{H_0 t}(w) = \bot. \tag{10}$$

We say that t has a *top prefix* along w if $\mathrm{Pref}_c^+(t,w)$, and a *bottom prefix* along w if $\mathrm{Pref}_c^-(t,w)$. Thus a top prefix is an actual top leaf at an even-polarity prefix of w, or an actual bottom leaf at an odd-polarity prefix. Its polarity is measured at that prefix, not at a later point past the leaf. A word w is *left unsafe* for an assignment σ if $\mathrm{Pref}_c^+(\sigma x, w)$ and not $\mathrm{Pref}_c^+(\sigma y, w)$, and *right unsafe* for σ if $\mathrm{Pref}_c^-(\sigma y, w)$ and not $\mathrm{Pref}_c^-(\sigma x, w)$. It is left (right) unsafe for (ϕ, x, y) if it is left (right) unsafe for some solution of ϕ.

By (8)–(9),

$$\phi \not\models_c x \leq y \quad \Longleftrightarrow \quad \text{some word is left unsafe or right unsafe.} \tag{11}$$

Indeed, the only failures of order in the three-element label chain are a left top against a non-top, or a right bottom against a non-bottom. The same argument shows, for each assignment σ, that $\sigma(x) \not\leq_c \sigma(y)$ iff some word is left or right unsafe for σ.

Example 2.5 (Running arrow system). Take $c = (1, 0)$, variables x, y, a, b, and the three flat literals

$$\phi = \{x \leq f(a,b),\ f(a,b) \leq x,\ y \leq f(a,b)\}.$$

The first two force $x = f(a,b)$, so every solution has $y \leq_c x$. The converse query fails: the finite assignment

$$\sigma(a) = \bot, \quad \sigma(b) = \top, \qquad \sigma(x) = f(\bot, \top), \quad \sigma(y) = f(\top, \bot)$$

solves ϕ, but $\sigma(x) \not\leq_c \sigma(y)$. As Figure 2 shows, normalization gives $H_0\sigma(x) = f(\top, \top)$ and $H_0\sigma(y) = f(\bot, \bot)$. Thus $w = 0$ witnesses both sides of unsafety: x has a top prefix and y does not, while y has a bottom prefix and x does not. The same assignment witnesses both sides for 00: the leaves are reached at 0, whose polarity is 1, even though $p(00) = 0$. We reuse this system with the failing query $x \leq y$ below.

2.6 Notation at a Glance

The following table collects the main notation, including symbols introduced later, grouped by the part of the paper that introduces them. In particular, the four dual operations have different input and output types.

Symbol	Type	Reading or effect
Semantics and constraints (Sections 2 to 4)		
$\mathcal{T}$, $\mathcal{T}_{\mathrm{reg}}$, $\mathcal{T}_{\mathrm{fin}}$	sets of trees	Arbitrary, regular, and finite trees; $\mathcal{D}$ ranges over these domains.
V, $V^{\pm}$, σ	variable sets; assignment	Original and signed variables; a tree for each variable.
ϕ, ψ, χ	finite constraint systems	Original system, signed translation, and a generic covariant system.
k	integer	Number of input variables (never a block count).
$\mathbb{D}$, c, p	alphabet; $\mathbb{D} \to \{0,1\}$; $\mathbb{D}^* \to \{0,1\}$	Child positions, their variances, and path polarity.
$w[i]$, $w[:j]$, $w[j:]$	letter; word; word	Letter at position i; prefix and suffix at cut j.
$u \preceq w$, $\mathrm{pr}(L)$	relation on words; set of words	Prefix; all prefixes of words in L, with $\mathrm{pr}(v) = \mathrm{pr}(\{v\})$.
$\mathfrak{d}$	$\mathcal{T} \to \mathcal{T}$	Tree duality: exchange $\bot, \top$, retain f.
H_q	trees under $\leq_c \to$ covariant trees	Normalize labels using $q \oplus p(w)$.
$\mathrm{Pref}_c^{\pm}(t, w)$	truth value	Top (+) or bottom (−) prefix of t along w.
ι	$V^{\pm} \to V^{\pm}$	Variable flip: complement the sign.
S	signed assignments → signed assignments	Assignment involution: $(S\sigma)(v) = \mathfrak{d}(\sigma(\iota v))$; fixed witnesses are S-fixed.
$\chi^{\mathfrak{d}}$	covariant systems → covariant systems	Order dual: reverse inequalities and exchange constants; keep variable names.
θ; ψ_θ, X_θ, Y_θ	side; system; variables	Left or right side, its covariant system, and its two query roots.
D_χ	relation on variables	Reflexive, transitive decomposition closure.
Up, Lo	word × variable × variable → truth value	Supported upper and lower path bounds.
$\mathrm{emb}_b(\eta, t)$	tree	Embed t at path η, with off-path filler b.
$\ell_f, u_f, u_\bot$	predicates on pairs of a variable and a path	Lower constructor, upper constructor, and upper bottom support.
Ψ_w^4; P_j, Q_j, Y_j, W_j	system; signed variables	Four-spine extension along w and its spine variables at cut j.
U_j, L_j; U_j^R, L_j^Z	sets of signed variables	Variables reached by upper and lower path bounds at cut j, from x^+ and y^+, or from the spine roots R_0 and Z_0.
Readers and holes (Sections 5 to 7 and 9)		
$(\mathcal{M}, \mathsf{Adm}, \mathsf{Rej})$, N	reader; integer	Deterministic automaton with admission relation and rejected states; its number of states.
$\mathcal{M}_\theta$, Adm_θ, Rej_θ	reader	Left or right label reader: the label reader of Section 5.2 on side θ.
E_i, G_i	maps on sets of signed variables	One-letter updates of the upper and lower sets.
(s, e)	pair of cuts	Comparison when $s < e$ and $w[e:] \preceq w[s:]$.

Symbol	Type	Reading or effect
q_j, Rej, Adm	state; set of states; relation on states	Reader run, rejection, and admission.
$J(w)$, $r(w)$	integers	First rejected cut and rejected-tail length $\lvert w\rvert - J(w)$.
hole; fixed witness	word; assignment	Rejected word without admitted comparison, for a satisfiable system an unsafe word; S-fixed solution showing unsafety along a word. *Witness* always means the latter.
B, d, H	integers	Tail bound $4N-2$, depth bound $2N(B+1)-1$, and length bound $\mathsf{Len}(N,d)$ (Theorem 9.1).
Hierarchies and compression (Sections 7, 8 and 10)		
u_j, ℓ_j, C_j, $\mathcal{S}_j$	word; letter; set of cuts; set of states	Hierarchy level, marker, ordinary cores, and their reader states (state support).
$\mathcal{M}_j$, $\triangleright$, $\star$	lettered monitor; label; state	Derived monitor, start label, and new derivative start state.
T^{fin}, Λ, Λ^{fin}	predicate; two relations on labels	Final target, nonterminal admission, and terminal admission.
$\mathcal{A}_i$, g_i	set of letters; integer	Level-i dictionary (letters of u_i) and its size.
Π_i, Z_i	set of letters; set of states	Protected letters and monitor states in use at level i.
$\mathcal{E}_j$	set of letters	Letter support kept at level j during shortening.
Δ	integer	Depth of the hole being shortened in Section 8.
$\mathsf{Ty}(N,d)$, $\mathsf{Al}(N,d)$, $\mathsf{Dict}(N,d)$	integers	Number of exchange types, allowance, and dictionary bound (43).
h, $col(N,h)$	integers	Number of blocks (tracks) shortened together; column count of their summaries (47).
$\mathsf{Len}(N,d)$	integer	Length bound for holes of depth at most d (48).
dictionary-normal	property of holes	Normal form of Lemma 8.7; unrelated to the normalization H_q.
Complexity (Section 10)		
$\lVert\phi\rVert$	integer	Size of the explicit input.
cycle clash	property of a system	$\mathsf{Lo}(\eta,a,a)$, $D_\chi(a,b)$ and $\mathsf{Up}(\eta,b,b)$ for a nonempty η; excludes finite solutions (Section 9).
$\mathcal{W}$, $\Gamma_{\mathcal{W}}$, Θ	lettered monitor; alphabet; summary	Current monitor of the summary search, its abstract letters $1,\dots,h$, and a column summary determining the next monitor.
$\mathbb{T}$	alphabet	Counter tokens $\{0,1,\#,\$\}$ of the long-hole construction.
T_P, T_P^m	trees	Top-free tree with constructor positions P, and its truncation at depth m with $\top$ at the cut.
token encoding; code	maps into $\{0,1\}^*$	Binary encoding of counter tokens (long holes); prefix-free code of words (hardness).

3 Signed Constraints and Fixed Witnesses

We remove variance by giving each variable two signed copies. The copies must denote dual trees; the assignment involution expresses this condition. We then express each side of a failed query by a *fixed witness*: a solution of the signed system that is fixed by the assignment involution and

exhibits the failure along a given word. The four-spine construction of Section 4 will enforce such witnesses.

3.1 The Signed System

Write $V^{\pm} = V \times \{0,1\}$, $v^{+} = v^{0}$, $v^{-} = v^{1}$, and define the *variable flip* $\iota(v^{q}) = v^{1-q}$. Translate ϕ to a covariant constraint system $\psi = \phi^{\pm}$ as follows:

Original literal	Signed literals	
$z \le f(a_i)_i$	$z^{+} \le f(a_i^{c(i)})_i$	$f(a_i^{1\oplus c(i)})_i \le z^{-}$
$f(a_i)_i \le z$	$f(a_i^{c(i)})_i \le z^{+}$	$z^{-} \le f(a_i^{1\oplus c(i)})_i$
$z = \bot$	$z^{+} = \bot$	$z^{-} = \top$
$z = \top$	$z^{+} = \top$	$z^{-} = \bot$

Lemma 3.1 (Signed correspondence). *For an assignment σ on signed variables define $(S\sigma)(v) = \mathfrak{d}(\sigma(\iota v))$. The map S, called the* assignment involution, *is an involution and preserves solutions of ψ. Moreover,*

$$\sigma \longmapsto \sigma^{\pm}, \qquad \sigma^{\pm}(v^{q}) = H_q(\sigma v) \tag{12}$$

is a bijection between solutions of ϕ and S-fixed solutions of ψ.

Proof. Tree duality $\mathfrak{d}$ reverses inequalities and commutes with f. Applying it and ι exchanges the two literals in each row of the table. This proves preservation by S; applying both involutions twice gives $S^2 = \mathrm{id}$. By (7)–(8), (12) satisfies each translated pair and is fixed by S. Conversely, for an S-fixed solution define $\sigma_0(v) = H_0(\sigma(v^{+}))$. The fixed-point identity gives $\sigma(v^{-}) = \mathfrak{d}(\sigma(v^{+}))$, so (7) reconstructs (12). Applying (8) to the positive literal of each pair proves $\sigma_0 \models_c \phi$. The reconstruction also proves both inverse identities. □

3.2 Fixed Witnesses

For a covariant system χ, its *order dual* $\chi^{\mathfrak{d}}$ reverses every inequality and exchanges bottom and top, leaving variable names unchanged. Pointwise tree duality maps its solutions bijectively to those of χ. Write $\theta \in \{\mathsf{l}, \mathsf{r}\}$ for the left and right sides; l-unsafe means left unsafe, and similarly on the right. Put

$$(\psi_{\mathsf{l}}, X_{\mathsf{l}}, Y_{\mathsf{l}}) = (\psi, x^{+}, y^{+}), \qquad (\psi_{\mathsf{r}}, X_{\mathsf{r}}, Y_{\mathsf{r}}) = (\psi^{\mathfrak{d}}, y^{+}, x^{+}).$$

We call a covariant solution $\sigma \models \psi_\theta$ a *fixed witness* for w on side θ if $S\sigma = \sigma$ and

$$\widehat{\sigma(X_\theta)}(w) = \top, \qquad \widehat{\sigma(Y_\theta)}(w) \neq \top. \tag{13}$$

Here *fixed* means fixed by the assignment involution S, not that the word w is held fixed.

Lemma 3.2 (Fixed-witness criterion). *For either side θ, a word w is θ-unsafe iff it has a fixed witness on side θ.*

Proof. For the left side this is exactly signed correspondence and the definition of unsafe words after normalization. For the right side, normalize a variance solution and dualize every resulting tree. A bottom prefix at y^{+} becomes a top prefix, and absence of a bottom prefix at x^{+} becomes absence of a top prefix. Pointwise tree duality commutes with S, so fixedness is preserved. Conversely, dualize back and decode by Lemma 3.1. These operations are inverse and prove both directions, including paths that terminate before the end of w. □

Example 3.3 (The signed running witness). For Example 2.5, the six signed literals can be grouped as

$$x^+ \le f(a^-, b^+), \qquad f(a^-, b^+) \le x^+, \qquad y^+ \le f(a^-, b^+),$$
$$f(a^+, b^-) \le x^-, \qquad x^- \le f(a^+, b^-), \qquad f(a^+, b^-) \le y^-.$$

The contravariant child uses the opposite sign; passing to the negative root also reverses the inequality. The assignment of Example 2.5 translates to

v	x	y	a	b
$\sigma^{\pm}(v^+)$	$f(\top, \top)$	$f(\bot, \bot)$	$\bot$	$\top$
$\sigma^{\pm}(v^-)$	$f(\bot, \bot)$	$f(\top, \top)$	$\top$	$\bot$

and each column consists of dual trees, so $S\sigma^{\pm} = \sigma^{\pm}$. At $w = 0$, the positive x, y labels are $\top, \bot$, giving the left fixed witness. Dualizing this assignment and exchanging the query roots gives the right fixed witness for the same word.

A covariant solution need not itself be fixed. Section 4 constructs a fixed solution from the least shape (Section 4.2) of any consistent system that is flip-closed and sign coherent (Section 4.1). Flip closure means closure under reversing inequalities, flipping variables, and exchanging constants. Sign coherence means that each constructor child has the sign of its root, toggled by the variance of its position. Applying that construction to an extension encoding (13) will preserve the witness conditions as well as satisfaction.

4 Closure and Spine Clashes

This section turns unsafety of a fixed word w into a syntactic test on the constraints. By Lemma 3.2, w is unsafe on a side exactly when the signed system of that side has a solution that is fixed by the assignment involution S and satisfies two prefix conditions along w. We encode these conditions, together with their sign duals, by an extension Ψ_w^4 with four chains of fresh variables, the spines (Section 4.4). Two facts are needed before the spines, and they fix the order of this section. First, a covariant system is satisfiable exactly when it has no label clash, a condition on its finite closure (Sections 4.1 and 4.2). Second, a clash-free system that is flip-closed and sign coherent has a solution fixed by S, obtained by polarity selection on its least shape (Section 4.3). Together they reduce unsafety of w to the absence of a label clash in Ψ_w^4 (Theorem 4.9). We then express the closure of Ψ_w^4 through the closure of the original system and two sets U_j, L_j of old variables at each cut (Section 4.5). Classifying the clashes in these terms (Section 4.6) yields tests at single cuts and tests on suffix comparisons. Section 5 packages these tests as a finite reader with the rejected-path property.

4.1 Closure and Supported Path Bounds

For this subsection, let χ be any finite covariant constraint system over a finite variable set $\mathcal{V}$. Its *closure* D_χ is the least reflexive and transitive relation satisfying, for every $i \in \mathbb{D}$,

$$\frac{f(\vec{a}) \le u \in \chi \qquad D_\chi(u, v) \qquad v \le f(\vec{b}) \in \chi}{D_\chi(a_i, b_i)}. \tag{14}$$

Repeatedly adding reflexive, transitive, and decomposition consequences computes D_χ: there are only $|\mathcal{V}|^2$ pairs. Induction on a derivation proves soundness, $D_\chi(u, v) \Rightarrow \sigma(u) \le \sigma(v)$ for every solution. For the decomposition step use the constructor case of (4).

Define *supported upper and lower path bounds* by

$$\begin{aligned}
\mathsf{Up}(\varepsilon,u,v) &\iff D_\chi(u,v),\\
\mathsf{Up}(i\eta,u,v) &\iff \exists z,\vec{b}\,[D_\chi(u,z)\wedge z\le f(\vec{b})\in\chi\wedge\mathsf{Up}(\eta,b_i,v)],\\
\mathsf{Lo}(\varepsilon,u,v) &\iff D_\chi(u,v),\\
\mathsf{Lo}(i\eta,u,v) &\iff \exists\vec{a},z\,[f(\vec{a})\le z\in\chi\wedge D_\chi(z,v)\wedge\mathsf{Lo}(\eta,u,a_i)].
\end{aligned}\tag{15}$$

In every solution, $\mathsf{Up}(\eta,u,v)$ says that the subtree of u at η, when present, is below v; $\mathsf{Lo}(\eta,u,v)$ says that u is below the subtree of v there. The variable arguments follow the order of the inequality: the path is on the first variable for Up and on the last for Lo. To state soundness without assuming the path exists, let $\mathrm{emb}_b(\eta,t)$ embed t at η, putting the constant tree b in every off-path child. Then

Lemma 4.1 (Path rules). *In every solution of χ,*

$$\begin{aligned}
\mathsf{Up}(\eta,u,v) &\Longrightarrow \sigma(u)\le \mathrm{emb}_\top(\eta,\sigma(v)),\\
\mathsf{Lo}(\eta,u,v) &\Longrightarrow \mathrm{emb}_\bot(\eta,\sigma(u))\le\sigma(v).
\end{aligned}\tag{16}$$

Composition and factorization, including empty factors, satisfy

$$\begin{aligned}
\mathsf{Up}(\eta\tau,u,v) &\iff \exists a\,[\mathsf{Up}(\eta,u,a)\wedge\mathsf{Up}(\tau,a,v)],\\
\mathsf{Lo}(\eta\tau,u,v) &\iff \exists a\,[\mathsf{Lo}(\tau,u,a)\wedge\mathsf{Lo}(\eta,a,v)].
\end{aligned}\tag{17}$$

At an empty factor the path judgment retains the closure relation D_χ, rather than requiring equality of its endpoints. For later use, cancellation of a common path gives

$$\begin{aligned}
\mathsf{Lo}(\eta,a,z)\wedge\mathsf{Up}(\eta\tau,z,b) &\Longrightarrow \mathsf{Up}(\tau,a,b),\\
\mathsf{Lo}(\eta\tau,a,z)\wedge\mathsf{Up}(\eta,z,b) &\Longrightarrow \mathsf{Lo}(\tau,a,b).
\end{aligned}\tag{18}$$

Proof. Induction on (15), using (14) and the extreme off-path fillers, proves (16). Induction on the length of η proves both directions of (17); closure transitivity absorbs the intermediate pairs at empty factors. For cancellation, when $\eta=\varepsilon$, this is transitivity. For $\eta=i\eta'$, the first lower and upper constructor bounds $f(\vec{a}')\le z'$ and $z''\le f(\vec{b}')$, with their intervening closure pairs, give $D_\chi(a_i',b_i')$ by (14); apply the induction hypothesis to η'. This proves both formulas and supplies the claimed factor and cancellation rules in all arities. □

A signed variable set has a sign map sg and a fixed-point-free variable flip ι complementing signs. A system is *flip-closed* if reversing inequalities, flipping variables, and exchanging constants preserves its literals. It is *sign coherent* if every constructor child a_i of root z has sign $\mathrm{sg}(z)\oplus c(i)$, in either inequality orientation.

Return now to $\psi=\phi^\pm$; everything below also holds for its order dual $\psi^\mathfrak{d}$ with the query (y^+,x^+), the right side system of Section 3.2. The literals of ψ are closed under reversing inequalities, flipping all signs, and exchanging constants. Induction on (14) and (15) therefore proves

$$\begin{aligned}
D_\psi(u,v) &\iff D_\psi(\iota v,\iota u),\\
\mathsf{Lo}(\eta,v,\iota u) &\iff \mathsf{Up}(\eta,u,\iota v).
\end{aligned}\tag{19}$$

The signed translation is sign coherent by its defining table.

Lemma 4.2 (Sign purity). *In a sign-coherent system, closure pairs have equal signs. In either path judgment along η, the endpoint signs differ by $p(\eta)$.*

Proof. Reflexivity and transitivity preserve equality of signs. Decomposition compares children at the same position, so both signs toggle by $c(i)$. Induction on (15) proves the path assertion. □

In particular, a path from a positive query variable along η has its other endpoint of sign $p(\eta)$.

4.2 Label Clashes and the Least Shape

A supported lower root label at z is f if some $f(\vec{a}) \le u \in \chi$ has $D_\chi(u, z)$, and is $\top$ if some $u = \top \in \chi$ has $D_\chi(u, z)$. Dually, an upper root label is f or $\bot$ if it is supplied by $v \le f(\vec{b})$ or $v = \bot$, with $D_\chi(z, v)$. A *label clash* is a supported pair $(\top, \bot)$, $(\top, f)$, or $(f, \bot)$. Equivalently, it is a lower literal and an upper literal with a closure pair between their roots, where the root label of the lower literal exceeds that of the upper one.

Define the three support predicates by

$$\begin{aligned} \ell_f(z, \eta) &\iff \exists v, \vec{a}\ [\mathsf{Lo}(\eta, v, z) \wedge f(\vec{a}) \le v \in \chi], \\ u_f(z, \eta) &\iff \exists v, \vec{b}\ [\mathsf{Up}(\eta, z, v) \wedge v \le f(\vec{b}) \in \chi], \\ u_\bot(z, \eta) &\iff \exists v\ [\mathsf{Up}(\eta, z, v) \wedge v = \bot \in \chi]. \end{aligned} \tag{20}$$

We retain a constructor when lower and upper constructor support force one. Otherwise, we choose an extreme label compatible with the upper bounds. A prescribed top needs no separate branch: in a clash-free system it excludes the upper supports that could select another label. Define

$$h(z, \eta) = \begin{cases} f, & \ell_f(z, \eta) \wedge u_f(z, \eta), \\ \bot, & \neg \ell_f(z, \eta) \wedge (u_\bot(z, \eta) \vee u_f(z, \eta)), \\ \top, & \text{otherwise.} \end{cases} \tag{21}$$

The *least-shape assignment* σ_{sh} retains this label exactly at paths whose proper prefixes all have label f. The root is retained, and a child is retained exactly when its parent is labeled f, so this defines trees even when their domains are infinite. Here "least" refers to the set of constructor positions, as proved below, rather than to order-leastness: for binary covariant $f(a, a) \le a$ the rule gives $\top$, while the least solution is the infinite constructor tree.

The construction makes the least constructor shape explicit; its common shape under the assignment involution enables the polarity selection in Lemma 4.5.

Theorem 4.3 (Least shape and satisfiability). *If χ has no label clash, $\sigma_{\mathrm{sh}} \models \chi$. Hence*

$$\mathrm{Sat}(\chi) \iff \chi \text{ has no label clash.} \tag{22}$$

Proof. First, the label rule is monotone in lower constructor support and antitone in both upper supports. More precisely, if lower support at (v, η) implies that at (v', η'), and each upper support at (v', η') implies its counterpart at (v, η), then $h(v, \eta) \le h(v', \eta')$. For $h(v', \eta') = \top$, this is automatic. For $h(v', \eta') = f$, upper constructor support at (v, η) gives $h(v, \eta) \le f$. For $h(v', \eta') = \bot$, absence of lower constructor support transfers to (v, η), as does the upper support causing bottom, so $h(v, \eta) = \bot$.

For $f(\vec{a}) \le z$, the root supports a lower constructor, so $f \le h(z, \varepsilon)$. At a common child path $i\eta$, lower support at (a_i, η) propagates to $(z, i\eta)$ by (15), and upper support propagates in the reverse direction by (18). The monotonicity just proved gives the required comparison. For $z \le f(\vec{b})$, upper constructor support gives the root comparison; cancellation transports lower support from $(z, i\eta)$ to (b_i, η), and the path rule transports upper support back. Thus this inequality also holds at every common path. Pruning changes no label on common paths. A prescribed bottom has upper bottom support and cannot have lower constructor support without a clash, so the rule selects bottom. A prescribed top excludes both upper supports and hence selects top. This verifies every literal. Conversely, closure soundness excludes a solution in the presence of an incompatible pair of supported labels. □

Lemma 4.4 (Minimal constructor positions). *If χ has no label clash, every constructor position of $\sigma_{\mathrm{sh}}(z)$ is a constructor position of every solution's value at z.*

Proof. Induct along a retained path. At each constructor position η, (21) supplies both constructor supports. By (16), together with their supporting literals, they bound the solution between $\mathrm{emb}_\bot(\eta, f(\sigma(a_i))_i)$ and $\mathrm{emb}_\top(\eta, f(\sigma(b_i))_i)$ for suitable children. Both embeddings have label f at η. At the root the solution's label is therefore f; at subsequent positions the induction hypothesis ensures the path exists, and the same sandwich forces its label to be f. Since Theorem 4.3 makes σ_{sh} a solution, its constructor set is least by inclusion. □

4.3 Polarity Selection

Use the assignment involution S of Lemma 3.1 on any signed variable set. The signed translation and its order dual are flip-closed and sign coherent. The proofs of (19) and sign purity apply to every such system; flip closure also implies $\sigma \models \chi \Rightarrow S\sigma \models \chi$ by evaluating each flipped literal and reversing its tree inequality.

Lemma 4.5 (Common shape and selection). *For a flip-closed system, σ_{sh} and $S\sigma_{\mathrm{sh}}$ have the same domains and constructor positions. If it is also sign coherent and has no label clash, the assignment*

$$\sigma_{\mathrm{sel}}(z)(\eta) = \begin{cases} \sigma_{\mathrm{sh}}(z)(\eta), & \mathrm{sg}(z) \oplus p(\eta) = 0, \\ (S\sigma_{\mathrm{sh}})(z)(\eta), & \mathrm{sg}(z) \oplus p(\eta) = 1 \end{cases} \tag{23}$$

on this common domain satisfies χ, is fixed by S, and preserves that domain and its constructor positions exactly.

Proof. Variable flip exchanges lower and upper constructor support at (z, η) and $(\imath z, \eta)$. Thus (21) labels one by f iff it labels the other by f. Their proper-prefix pruning predicates coincide. Tree duality preserves domains and constructor labels, giving the common shape. Both assignments solve χ by Theorem 4.3 and flip closure. Selecting between them retains exactly their constructor positions, so it gives a well-formed tree on their common domain.

At a constructor-literal root either input has the required comparison with f. At a path $i\eta$, selection on the root side uses the bit $\mathrm{sg}(z) \oplus c(i) \oplus p(\eta)$, the same bit used on the child side at η, by coherence. Both labels therefore come from one satisfying assignment. This proves either orientation of the literal. Both inputs have the prescribed leaf at a constant literal. Finally, at $\imath z$ the choice bit is complemented. If the bit at z is zero, the selected label at $\imath z$ is the dual of the first input at z; if it is one, the selected label there is the first input at $\imath z$. Applying tree duality in either case gives the selected label at z. Common domains prove the same equality at absent paths, so $S\sigma_{\mathrm{sel}} = \sigma_{\mathrm{sel}}$. □

In particular, normalization in one direction and selection followed by decoding in the other prove

$$\mathrm{Sat}_c(\phi) \iff \mathrm{Sat}(\phi^{\pm}). \tag{24}$$

Together with the finite closure test, this decides satisfiability.

Example 4.6 (Selecting on the least shape). The running signed system of Example 3.3 has identity closure and no label clash. Both constructor bounds occur at x^+ and x^-; their children have no constructor or constant bounds. Hence (21) gives

v	x	y	a	b
$\sigma_{\mathrm{sh}}(v^+)$	$f(\top,\top)$	$\bot$	$\top$	$\top$
$\sigma_{\mathrm{sh}}(v^-)$	$f(\top,\top)$	$\top$	$\top$	$\top$

This solution is not fixed: at x^+ its image under S is $f(\bot,\bot)$. The domains nevertheless agree. Selection at x^+ takes the domain leaf from $S\sigma_{\mathrm{sh}}$ and the codomain leaf from σ_{sh}, as in Figure 3. In

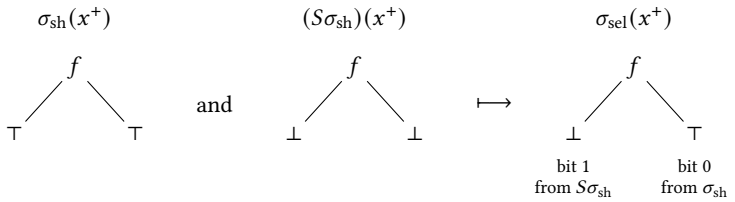


Fig. 3. Selection in Example 4.6. All three trees have the same shape. At the positive root the choice bit is 0; at children 0, 1 it is respectively 1, 0, yielding $f(\bot, \top)$.

full, it gives

v	x	y	a	b
$\sigma_{\mathrm{sel}}(v^+)$	$f(\bot,\top)$	$\bot$	$\top$	$\top$
$\sigma_{\mathrm{sel}}(v^-)$	$f(\top,\bot)$	$\top$	$\bot$	$\bot$

Decoding gives another solution of ϕ: $x = f(\top, \top)$, $y = \bot$, and $a = b = \top$.

4.4 The Four-Spine Extension

Throughout Sections 4.4 to 4.6, ψ may be any flip-closed, sign-coherent system on a signed variable set, and x^+, y^+ any two positive variables, with $x^- = \imath x^+$ and $y^- = \imath y^+$. In particular, this includes both side systems $(\psi_\theta, X_\theta, Y_\theta)$ of Section 3.2.

Fix $w = w[0] \cdots w[m-1]$ and $p_j = p(w[:j])$. To enforce the two fixed-witness conditions and their sign duals, we seek

$$\begin{aligned} \mathrm{emb}_\bot(w, \top) \le \sigma(x^+), \qquad & \sigma(x^-) \le \mathrm{emb}_\top(w, \bot), \\ \sigma(y^+) \le \mathrm{emb}_\top(w, f(\top)_i), \quad & \mathrm{emb}_\bot(w, f(\bot)_i) \le \sigma(y^-). \end{aligned} \tag{25}$$

The four families flatten these bounds along w:

Spine	Bound direction	Intended condition along w	Sign-dual partner
P	lower	x^+ has a top prefix	Q
Q	upper	x^- has a bottom prefix	P
Y	upper	y^+ has no top prefix	W
W	lower	y^- has no bottom prefix	Y

The families P and Q start at the two signed copies x^+ and x^- of the query root x, and Y and W at y^+ and y^-. Thus Q and W are the sign duals of P and Y: flipping signs exchanges P with Q and Y with W. The spines P and Y express the fixed-witness conditions. Their sign-dual partners make the extension flip-closed, so selection yields a fixed solution.

Set $P_0 = x^+$, $Q_0 = x^-$, $Y_0 = y^+$, $W_0 = y^-$. For $1 \le j \le m$ introduce fresh base variables x_j, y_j, which carry the spines of x and y at cut j, and name

$$P_j = x_j^{p_j}, \quad Q_j = x_j^{1 \oplus p_j}, \qquad Y_j = y_j^{p_j}, \quad W_j = y_j^{1 \oplus p_j}.$$

Two further base variables o_0, o_1, for the off-path fillers, supply four signed fillers $B^0 = o_0^0$, $T^1 = o_0^1$, $T^0 = o_1^0$, $B^1 = o_1^1$. Thus $\imath B^q = T^{1-q}$, $\imath P_j = Q_j$, and $\imath Y_j = W_j$. The variables with positive index and the fillers are fresh. The cut-zero names $P_0 = x^+$, $Q_0 = x^-$, $Y_0 = y^+$, $W_0 = y^-$ are old variables; $P_0 = Y_0$ and $Q_0 = W_0$ when $x = y$. There are $2k + 4m + 4$ signed variables. For $R \in \{P, W\}$ and

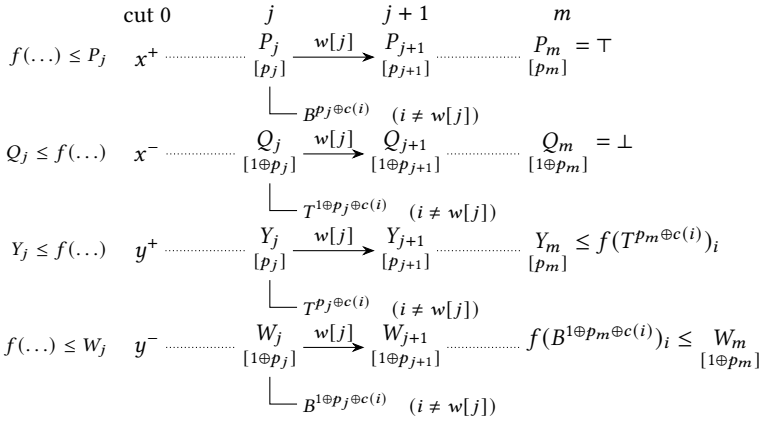


Fig. 4. The four-spine extension along w, with $Q_j = \iota P_j$ and $W_j = \iota Y_j$. Brackets give node signs; the root signs are $0, 1, 0, 1$. Each arrow reads the child $w[j]$. The P, W links are lower constructor bounds, and the Q, Y links are upper bounds, with the displayed off-path fillers. Dots abbreviate the other cuts. Example 4.7 instantiates the construction at $w = 0$.

$Z \in \{Q, Y\}$ put

$$a_i^{R,j} = \begin{cases} R_{j+1} & i = w[j], \\ B^{\mathrm{sg}(R_j) \oplus c(i)} & i \neq w[j], \end{cases} \qquad b_i^{Z,j} = \begin{cases} Z_{j+1} & i = w[j], \\ T^{\mathrm{sg}(Z_j) \oplus c(i)} & i \neq w[j]. \end{cases}$$

The extension Ψ_w^4 consists of ψ and

$$\begin{array}{ll} f(a_i^{P,j})_i \le P_j, & f(a_i^{W,j})_i \le W_j \quad (0 \le j < m), \\ Q_j \le f(b_i^{Q,j})_i, & Y_j \le f(b_i^{Y,j})_i \quad (0 \le j < m), \\ P_m = \top, & Q_m = \bot, \\ Y_m \le f(T^{p_m \oplus c(i)})_i, & f(B^{1 \oplus p_m \oplus c(i)})_i \le W_m, \\ B^q = \bot, & T^q = \top \quad (q = 0, 1). \end{array} \tag{26}$$

It is sign coherent: the distinguished child uses $p_{j+1} = p_j \oplus c(w[j])$, and all other children, including terminal children, have the prescribed sign by definition. It is flip-closed: flip pairs the P and Q links, the W and Y links, the terminal constants, the terminal constructor bounds, and the opposite-sign filler constants. These identities also hold at cut zero and when $m = 0$.

Figure 4 displays the four parallel families of bounds; the arrows indicate the distinguished child, not closure pairs.

Example 4.7 (One contravariant step). For the running query and $w = 0$, the root signs of (P, Q, Y, W) are $(0, 1, 0, 1)$, and their signs at cut 1 are $(1, 0, 1, 0)$. The new links and terminal bounds are exactly

$$\begin{array}{llll} f(P_1, B^0) \le x^+, & & x^- \le f(Q_1, T^1), & \\ y^+ \le f(Y_1, T^0), & & f(W_1, B^1) \le y^-, & \\ P_1 = \top, & Q_1 = \bot, & Y_1 \le f(T^0, T^1), & f(B^1, B^0) \le W_1, \end{array}$$

together with $B^q = \bot$ and $T^q = \top$ for $q = 0, 1$. Extend the fixed witness of Example 3.3 by

$$\sigma(P_1) = \top, \quad \sigma(Q_1) = \bot, \qquad \sigma(Y_1) = f(\top, \top), \quad \sigma(W_1) = f(\bot, \bot).$$

With the prescribed fillers this is a fixed solution of the extension, which therefore has no label clash.

Lemma 4.8 (Spine extension). *An assignment $\sigma \models \psi$ extends to a solution of Ψ^4_w iff*

$$\begin{aligned} \widehat{\sigma(x^+)}(w) &= \top, \quad \widehat{\sigma(x^-)}(w) = \bot, \\ \widehat{\sigma(y^+)}(w) &\neq \top, \quad \widehat{\sigma(y^-)}(w) \neq \bot. \end{aligned} \tag{27}$$

Proof. Composing the links gives (25). The first bound forces a top at or before the final cut: an earlier bottom violates the on-spine constructor bound, while reaching the end forces top. Conversely, a top prefix satisfies the bound; all off-path bottom comparisons are automatic. Duality proves the second equivalence. The third excludes a top at every cut through the end, since the upper bound has constructors there. A bottom ends comparison harmlessly; otherwise the terminal label is at most f. Duality gives the fourth equivalence. For sufficiency, assign, for each $1 \le j \le m$,

$$\begin{aligned} \sigma(P_j) &= \mathrm{emb}_\bot(w[j:], \top), & \sigma(Q_j) &= \mathrm{emb}_\top(w[j:], \bot), \\ \sigma(Y_j) &= \mathrm{emb}_\top(w[j:], f(\top)_i), & \sigma(W_j) &= \mathrm{emb}_\bot(w[j:], f(\bot)_i). \end{aligned}$$

These are assignments to distinct fresh signed variables, so they impose no extra condition on the old assignment, even when it is not fixed by S. Assign each filler its prescribed constant. For $m \ge 1$, internal links and terminal literals hold with equality, and the root links are the four bounds (25); all newly assigned trees are finite, so the extension stays in each of the three domains. For $m = 0$ the terminal literals themselves are those bounds at the old roots. □

Theorem 4.9 (Four-spine unsafe-word criterion). *On either side, w is unsafe iff the four-spine extension of that side's system and query has no label clash. No satisfiability assumption is needed.*

Proof. For the left side, an unsafe word gives a fixed witness by Lemma 3.2. Fixedness supplies the opposite-sign conditions in (27), so Lemma 4.8 extends it to a solution and excludes a clash. Conversely, apply Lemma 4.5 to the whole clash-free extension (26). The selected least shape is a fixed solution; restriction to the old variables preserves satisfaction and fixedness, and Lemma 4.8 supplies all four prefix conditions. Its first and third conditions give the fixed witness and hence unsafety. For the right side use $\psi^\flat$ and (y^+, x^+). Order duality preserves flip closure and coherence, since child signs do not depend on inequality orientation. Conclude by Lemma 3.2 for $\theta = \mathsf{r}$. □

Lemma 4.10 (Separation). *Partition the variables into old variables and the two fresh classes*

$$\textit{sources } \{P_j, W_j : 1 \le j \le m\} \cup \{B^0, B^1\}, \qquad \textit{sinks } \{Q_j, Y_j : 1 \le j \le m\} \cup \{T^0, T^1\}.$$

Then

$$\begin{aligned} &D_{\Psi^4_w}(a, s),\ s \textit{ a source} && \Longrightarrow a = s, \\ &D_{\Psi^4_w}(t, b),\ t \textit{ a sink} && \Longrightarrow b = t, \\ &a, b \textit{ old}: \quad D_{\Psi^4_w}(a, b) && \Longleftrightarrow D_\psi(a, b). \end{aligned} \tag{28}$$

Proof. Induct simultaneously on closure derivations. An upper constructor literal has only old or sink children, never source children; hence decomposition cannot have a source as its right endpoint. Dually, a lower constructor literal has only old or source children, so decomposition cannot have a sink as its left endpoint. This proves the first two assertions, with transitivity collapsing the intervening identity. For an old-to-old transitivity step, a fresh middle variable would be a source or a sink and contradict these assertions. For an old-to-old decomposition step, both bounding literals must be old: every new lower literal has only source children, and every new upper literal has only sink children. Apply induction to their old root pair. The other implication follows by including the original derivation. Constants add no closure rule. □

The classification here is by provenance of a literal, not the class of its root: a new literal can be rooted at an old variable at cut zero.

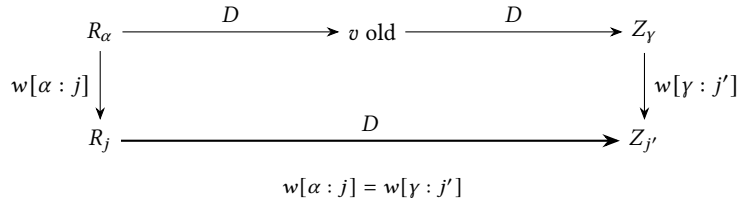


Fig. 5. A bridge in Lemma 4.11. The old variable belongs to $U^R_\alpha \cap L^Z_\gamma$. Horizontal arrows are closure pairs; vertical arrows descend through distinguished children along the same factor. Decomposition propagates the top-row pair $D(R_\alpha, Z_\gamma)$ to $D(R_j, Z_{j'})$. The common factor may be empty.

4.5 The Bridge Formula

Write $D = D_{\Psi^4_w}$ for the closure of the four-spine extension. All path judgments in this subsection are in the base system ψ. Define

$$U^R_j = \{v : \mathsf{Up}(w[:j], R_0, v)\}, \qquad L^Z_j = \{v : \mathsf{Lo}(w[:j], v, Z_0)\}, \tag{29}$$

where $R \in \{P, W\}$ and $Z \in \{Q, Y\}$. Put $U_j = U^P_j$ and $L_j = L^Y_j$. By (19), we have $L^Q_j = \iota U_j$ and $U^W_j = \iota L_j$, where $\iota\mathcal{U} = \{\iota v : v \in \mathcal{U}\}$. The sets U^R_j are upward closed under D_ψ, and L^Z_j are downward closed. At cut zero, $U^R_0 = D_\psi{\uparrow}R_0$ and $L^Z_0 = D_\psi{\downarrow}Z_0$, where $D_\psi{\uparrow}z = \{v : D_\psi(z, v)\}$ and $D_\psi{\downarrow}z = \{v : D_\psi(v, z)\}$.

These sets already show why a clash can relate two different cuts. Take cuts $s < e$ and $v \in U_e \cap L_s$. Where the subtrees exist, the path bounds read

$$\sigma(x^+)|_{w[:e]} \le \sigma(v) \le \sigma(y^+)|_{w[:s]}.$$

The upper embedding bound excludes a top leaf of x^+ before cut e. If x^+ has a top leaf along w, its remaining path is therefore a prefix of $w[e:]$, and the same bound forces a top prefix of v there. The lower embedding bound then forces a top leaf of y^+ along $w[:s]w[e:]$. When $w[e:] \preceq w[s:]$, this word is a prefix of w, contradicting the target condition. The embedding formulation handles an earlier top leaf of y^+ as well. In the extension, the old variable v links the P-spine at cut e to the Y-spine at cut s, and the equal suffixes carry this link down to the final cut. The bridge formula describes all such links.

Lemma 4.11 (Exact bridge formula). *For every old v, both lower names R, both upper names Z, and all $0 \le j, j' \le m$,*

$$D_{\Psi^4_w}(R_j, v) \iff v \in U^R_j, \qquad D_{\Psi^4_w}(v, Z_{j'}) \iff v \in L^Z_{j'}, \tag{30}$$

$$\begin{aligned} D_{\Psi^4_w}(R_j, Z_{j'}) \iff \exists \alpha \le j,\ \gamma \le j' \quad & U^R_\alpha \cap L^Z_\gamma \neq \varnothing, \\ & j - \alpha = j' - \gamma, \qquad w[\alpha : j] = w[\gamma : j']. \end{aligned} \tag{31}$$

The bridge consists of an old-variable intersection followed by parallel descent through equal word factors (Figure 5).

Proof. *Right to left.* At zero, (30) follows from Lemma 4.10. Factor an upper path along $w[:j]$ at its last letter. By induction the upper path along $w[:j-1]$ gives a closure pair from R_{j-1} to the root of an old upper literal; decompose against the R-link at position $w[j-1]$, then use the terminal closure of the path judgment. This gives the right-to-left direction of the first equivalence

in (30). The lower-path argument with a Z-link proves the second. For (31), choose $v \in U^R_\alpha \cap L^Z_\gamma$. By (30), we have $D(R_\alpha, v)$ and $D(v, Z_\gamma)$. Their composite is $D(R_\alpha, Z_\gamma)$. Decompose the two spine links successively along the common factor $w[\alpha : j] = w[\gamma : j']$. This gives $D(R_j, Z_{j'})$.

Left to right. Strengthen the induction on a closure derivation to four simultaneous obligations: old-to-old reflection in (28), the two implications (30) for each lower or upper name at positive indices, and (31) for each of the four name pairs when both indices are positive. Pairs with a bottom filler on the left or a top filler on the right carry no additional obligation; such pairs can occur.

For transitivity, an intermediate source forces its incoming pair to be an identity, and an intermediate sink forces its outgoing pair to be an identity, by (28). An old intermediate variable joins the two path memberships, producing the intersection in (31). Decomposition advances both spines along the same letter; empty factors retain closure, so the invariant also covers cut-zero aliases. The remaining rule cases and zero endpoints are detailed in Appendix A. □

4.6 Exact Clash Analysis

Let $\mathrm{Bot} = \{v : v = \bot \in \psi\}$ and $\mathrm{Top} = \{v : v = \top \in \psi\}$. Define

$$\begin{aligned} \mathrm{bot}(U) &\iff U \cap \mathrm{Bot} \neq \varnothing, \\ \mathrm{top}(L) &\iff L \cap \mathrm{Top} \neq \varnothing, \\ \mathrm{refl}(U, L) &\iff U \cap L \neq \varnothing, \\ \mathrm{ready}(U, L) &\iff \mathrm{bot}(U) \vee \mathrm{top}(L) \vee \mathrm{refl}(U, L), \\ \mathrm{child}(U) &\iff \exists v \in U, \vec{b} \quad v \leq f(\vec{b}) \in \psi. \end{aligned} \tag{32}$$

At cut j, bot-readiness bounds x^+ by bottom along $w[: j]$, excluding a top prefix along w. Top-readiness forces a top prefix of y^+ by cut j. Reflexivity readiness bounds x^+ by y^+ at the same cut, so any remaining top prefix transfers to y^+. These readings use the embedding bounds (16), and thus also cover trees that terminate before the cut. The child test at the terminal cut bounds x^+ by a constructor along all of w, excluding a top prefix there. It does not depend on a next letter: it asks only whether U contains the root of an upper constructor bound.

The two-cut events concern the following pairs of cuts.

Definition 4.12 (Comparison). A *comparison* of a word w of length m is a pair of cuts (s, e) satisfying

$$0 \leq s < e \leq m, \qquad w[e :] \preceq w[s :]. \tag{33}$$

Its *period* is $\varpi = e - s$, and its *root* is the factor $w[s : e]$. A comparison is *terminal* if $e = m$.

Section 5 interprets comparisons as periodic suffixes and gives examples. A comparison of w is *cross-admitted* if $L_s \cap U_e \neq \varnothing$, and *self-admitted* if $U_s \cap \imath U_e \neq \varnothing$.

Theorem 4.13 (Exact clash analysis). *For either side system, its four-spine extension Ψ^4_w has a label clash iff the side system ψ has a label clash or at least one of the following events occurs:*

$$\begin{aligned} &(a) \quad \exists j \leq m \quad \mathrm{ready}(U_j, L_j), \\ &(b) \quad \mathrm{child}(U_m), \\ &(c) \quad \exists s < e \leq m \quad w[e :] \preceq w[s :] \ \wedge \ [L_s \cap U_e \neq \varnothing \ \vee \ U_s \cap \imath U_e \neq \varnothing]. \end{aligned} \tag{34}$$

Readiness (a) rules out a fixed witness at a single cut, and the child event (b) rules it out at the final cut. A cross event in (c) transfers a top prefix from x^+ to y^+ across two cuts. A self event makes the top-prefix requirement on x^+ incompatible with the bottom-prefix requirement on x^-.

Lower bound \ upper bound	old	$Q_{j'}$	$Y_{j'}$	B^q
old	inherited clash	bot / child	top	impossible
P_j	bot / child	self	refl / cross	impossible
W_j	top	refl / cross	none (f/f)	impossible
T^q	impossible	impossible	impossible	impossible

The entries classify incompatible labels only; a closure pair between compatible constructor labels is not a clash. Here bot, top, and refl are the three readiness kinds, and child is the terminal event.

Proof. Suppose first that a clash occurs. Express the clash by its two bounding literals and a closure pair between their roots. The only relevant new lower bounds are the P_j-bounds, with label f for $j < m$ and $\top$ for $j = m$, the W_j-bounds, all labeled f (including $j = m$), and the bounds $T^q = \top$. The relevant new upper bounds are the $Q_{j'}$-bounds, labeled f for $j' < m$ and $\bot$ for $j' = m$, the $Y_{j'}$-bounds, all labeled f, and $B^q = \bot$. The fillers cannot participate: a pair out of T^q, or into B^q, is an identity by (28), and neither has an opposite incompatible bound. Classify the remaining bounds by whether their literals belong to ψ or were added in (26).

If both are old, separation gives an inherited clash in ψ. If the lower bound is a P_j-bound and the upper bound is old, an upper bottom yields $\mathrm{bot}(U_j)$ by (30). An upper constructor clashes only against $P_m = \top$, and yields $\mathrm{child}(U_m)$.

The remaining old/new cases are their sign duals or give top-readiness; see Appendix A.

For a new P-bound against a new Y-bound, the Y-bound has label f, so the lower endpoint must be P_m. Apply (31) to $D(P_m, Y_{j'})$. Its cuts satisfy $m - \alpha = j' - \gamma$, so $\alpha \geq \gamma$. If $\alpha = \gamma$, the intersection $U_\alpha \cap L_\alpha$ gives reflexivity readiness. If $\alpha > \gamma$, set $s = \gamma, e = \alpha$. The factor equality reads $w[e : m] = w[s : s + m - e]$; this is exactly (33), and the intersection is the cross admission $L_s \cap U_e \neq \emptyset$.

The W/Q case is the sign dual of this cross case; W/Y labels are compatible.

For a P-bound against a Q-bound, at least one endpoint is terminal: $j = m$ or $j' = m$. The cuts α, γ given by (31) have $U_\alpha \cap \iota U_\gamma \neq \emptyset$. Purity excludes $\alpha = \gamma$, since U_α and ιU_α have opposite signs. If $\alpha < \gamma$, the equation $j - \alpha = j' - \gamma$ forces $j' > j$, hence $j' = m$; set $s = \alpha, e = \gamma$. If $\alpha > \gamma$, it forces $j > j'$, hence $j = m$; set $s = \gamma, e = \alpha$ and replace a common element v by ιv. In both cases the equal factors give (33) and the intersection is $U_s \cap \iota U_e \neq \emptyset$. These two orientations are both needed. All other new/new pairs either have equal constructor labels or cannot supply a lower and an upper incompatible label. This exhausts the clashes, including literals rooted at old variables when $m = 0$.

Conversely, an inherited clash persists. Readiness and child events produce the clashes in the table, as detailed in Appendix A. For a cross admission choose $v \in U_e \cap L_s$. The bridge yields $D(P_e, Y_s)$. By (33), descend $m - e$ matching letters to obtain $D(P_m, Y_{s+m-e})$, a top/constructor clash with $P_m = \top$. For a self admission choose $v \in U_s \cap \iota U_e = U_s \cap L_e^Q$. Descend from $D(P_s, Q_e)$ to $D(P_{s+m-e}, Q_m)$. Since $s < e$, the left index is strictly less than m, so this is a constructor/bottom clash. When $e = m$, no descent is needed, and the same clashes remain valid. □

Remark 4.14 (Special cases of the clash analysis).

- For satisfiable ψ, inherited clashes are absent, so the events (a)–(c) are exactly the extension clashes.
- Purity shows that a self-admitted comparison must have $p(w[s : e]) = 1$: its two upper sets must have opposite signs. Thus self-admissions disappear in the fully covariant case, but cannot be discarded for general variance.
- When $x = y$, reflexivity puts x^+ in $U_0 \cap L_0$, so readiness at zero excludes every unsafe word.
- When $m = 0$, only readiness and the terminal child test remain; the zero-cut bridge formulas cover all aliases.

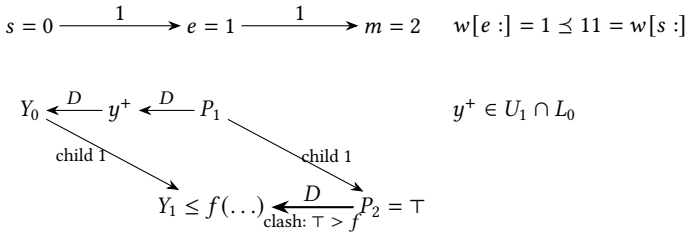


Fig. 6. The nonterminal cross event of Example 4.15, for $\phi = \{x \leq f(a, y),\ y \leq f(a, b)\}$ with $c = (1, 0)$, the query $x \leq y$, and $w = 11$. The comparison $(0, 1)$ is cross-admitted through $y^+ \in U_1 \cap L_0$. The bridge $D(P_1, Y_0)$ descends one matching letter to $D(P_2, Y_1)$, where top meets an upper constructor bound.

- At arity zero, the constructor $f()$ still supplies a genuine terminal bound and is detected by the child test.
- Child is tested only at the terminal cut; readiness is tested at every cut, including zero and the terminal cut.

Example 4.15 (A cross event). Replace the running system by $\phi = \{x \leq f(a, y),\ y \leq f(a, b)\}$, keeping $c = (1, 0)$ and the query $x \leq y$. Its signed literals with positive roots are $x^+ \leq f(a^-, y^+)$ and $y^+ \leq f(a^-, b^+)$; the other two are their sign duals, and the closure is the identity. Along $w = 11$, the upper sets are $\{x^+\}, \{y^+\}, \{b^+\}$ and the lower sets are $\{y^+\}, \varnothing, \varnothing$. No cut is ready, the terminal child test fails, and there is no self event. The only event is the comparison $(0, 1)$, with $w[1:] = 1 \preceq 11 = w[0:]$, which is cross-admitted because $y^+ \in U_1 \cap L_0$. Figure 6 shows the resulting clash. The shared variable y^+ supplies the bridge $D(P_1, Y_0)$ between two different cuts, and the matching suffix transports it one letter down to $D(P_2, Y_1)$, where $P_2 = \top$ meets the upper constructor bound of Y_1.

Example 4.16 (A self event). Keep the arrow variance $c = (1, 0)$ and take $\phi = \{x \leq f(x, b)\}$, that is, $x \leq x \to b$, with the query $x \leq y$ and y unconstrained. The signed literals are $x^+ \leq f(x^-, b^+)$ and $f(x^+, b^-) \leq x^-$, and the closure is the identity. Along $w = 0$, the upper sets are $U_0 = \{x^+\}$ and $U_1 = \{x^-\}$, and the lower sets are $L_0 = \{y^+\}$ and $L_1 = \varnothing$. There are no constant literals and $U_j \cap L_j = \varnothing$, so neither cut is ready, and $\mathsf{child}(U_1)$ fails because x^- roots no upper constructor bound. The terminal comparison $(0, 1)$ is not cross-admitted, since $L_0 \cap U_1 = \varnothing$, but it is self-admitted: $U_0 \cap \iota U_1 = \{x^+\}$. The self event alone shows that $w = 0$ is not left unsafe. Semantically, $x \leq_c x \to b$ excludes $x = \top$, so a top prefix of x^+ along 0 needs a bottom domain leaf, $x = f(\bot, t)$; but then $x \leq_c x \to b$ forces $x \leq_c \bot$, which is impossible. In the extension, $P_0 = x^+$, $Q_0 = x^-$, $P_1 = \top$, and $Q_1 = \bot$, with the new links $f(P_1, B^0) \leq x^+$ and $x^- \leq f(Q_1, T^1)$. Decomposing the old lower literal $f(x^+, b^-) \leq x^-$ against the new upper link gives $D(P_0, Q_1)$, and the new lower constructor bound on P_0 clashes with $Q_1 = \bot$. This is the terminal self case of Theorem 4.13.

5 Label Readers and Holes

The clash tests now become a finite reader: rejection rules out the single-cut events, while admission records the two-cut events. We show that the resulting readers have the rejected-path property, which is the only subtype-specific input to the word bounds.

5.1 Words, Cuts, and Comparisons

We use the word and cut notation of Section 2.1; put $m = |w|$. Recall from Definition 4.12 that a comparison of w is a pair of cuts $s < e$ with $w[e{:}] \preceq w[s{:}]$; its period is $\varpi = e - s$, its root is $w[s{:}e]$, and it is terminal when $e = m$. Comparisons are what the readers below test.

Equivalently, (s, e) is a comparison iff the suffix $w[s{:}]$ has period ϖ, i.e., $w[i] = w[i - \varpi]$ for all $e \le i < m$, iff $w[s{:}]$ is a prefix of a power of its root (Lemma 2.1). Every pair (s, m) is a terminal comparison. Over $\{0, 1\}$, the nonterminal comparisons of $w = 0100$ are $(0, 3)$, because $w[3{:}] = 0$ is a prefix of w, which has period 3, and $(2, 3)$, because $0 \preceq 00 = w[2{:}]$; the pair $(0, 2)$ is not a comparison, because 00 is not a prefix of 0100. Comparisons are inherited by prefixes: if (s, e) is a comparison of w and $e \le h \le m$, then (s, e) is a comparison of $w[{:}h]$. Extending a word may destroy a comparison when a new letter breaks its period, and may create comparisons that end at the new cuts.

Definition 5.1 (Reader hole). A *reader* $(\mathcal{M}, \mathsf{Adm}, \mathsf{Rej})$ consists of a deterministic finite automaton $\mathcal{M} = (Q, q_0, \delta)$ over a finite alphabet Γ, with transition function $\delta : Q \times \Gamma \to Q$ extended to words by $\delta(q, \varepsilon) = q$ and $\delta(q, wa) = \delta(\delta(q, w), a)$, together with a relation $\mathsf{Adm} \subseteq Q \times Q$, the *admission relation*, and a set $\mathsf{Rej} \subseteq Q$ of *rejected* states. A state is *reachable* if it is $\delta(q_0, w)$ for some word w. For a word w, write $q_i = \delta(q_0, w[{:}i])$ for the state at cut i. A word w is a *hole* of $(\mathcal{M}, \mathsf{Adm}, \mathsf{Rej})$ if $q_{|w|} \in \mathsf{Rej}$ and no comparison (s, e) of w satisfies $\mathsf{Adm}(q_s, q_e)$. States outside Rej are accepted.

Rejection identifies a candidate counterexample word. An admitted comparison disqualifies that candidate. Thus a hole requires both a rejected final state and the absence of every admitted comparison.

5.2 The Label Reader

For a set of signed variables define its one-letter updates

$$\begin{aligned} E_i(U) &= \{v : \exists u \in U \quad \mathsf{Up}(i, u, v)\}, \\ G_i(L) &= \{v : \exists u \in L \quad \mathsf{Lo}(i, v, u)\}. \end{aligned} \tag{35}$$

For a flip-closed, sign-coherent system ψ and positive query roots x^+, y^+, the *label reader* has states (U, L, b), where $U, L \subseteq V^{\pm}$ and $b \in \{0, 1\}$. Its initial state and transition are

$$q_0 = (D_\psi{\uparrow}x^+, D_\psi{\downarrow}y^+, 0), \qquad \delta_i(U, L, b) = (E_i U, G_i L, b \vee \mathsf{ready}(U, L)). \tag{36}$$

The bit b is a *latch*: once set, it stays set and records readiness at an earlier cut. It is needed because readiness at any earlier cut excludes every extension from being unsafe, even if readiness later disappears. Since $|V^{\pm}| = 2k$, each of U, L has 2^{2k} possible values; together with the two latch values this gives $2 \cdot 2^{2k} \cdot 2^{2k} = 2 \cdot 4^{2k}$ states. All transitions and tests are computable from the finite closure. Write q_j for the state after $w[{:}\,j]$. The path rules (Lemma 4.1) and induction on j prove the run invariant

$$q_j = (U_j, L_j, b_j), \qquad b_j = 1 \iff \exists h < j \quad \mathsf{ready}(U_h, L_h). \tag{37}$$

Designate as rejected the states

$$\mathsf{Rej} = \{(U, L, b) : b = 0 \wedge \neg\mathsf{ready}(U, L) \wedge \neg\mathsf{child}(U)\}, \tag{38}$$

and define the binary state relation

$$\mathsf{Adm}((U, L, b), (U', L', b')) \iff L \cap U' \neq \varnothing \ \vee \ U \cap \iota U' \neq \varnothing. \tag{39}$$

A word w is a hole precisely when

$$q_m \in \mathsf{Rej} \quad \wedge \quad \forall s < e \le m, \quad w[e:] \preceq w[s:] \Longrightarrow \neg \mathsf{Adm}(q_s, q_e). \tag{40}$$

The set of holes of a reader need not be a regular language, even for a label reader, since the comparison test relates suffixes that start at different cuts; Section 14.2 gives an example. The argument does not require regularity.

There is one reader construction, with two instances: the *left and right label readers*. The left label reader uses ψ with (x^+, y^+); the right one uses $\psi^\flat$ with (y^+, x^+). In the latter, closure, path updates, readiness, child, and admission are all computed in the dual system by the same formulas. Order duality preserves flip closure, sign coherence, and satisfiability. By Lemma 3.2, a top prefix in the dual system is a bottom prefix in the original one, so the right label reader tests right unsafety. Write $(\mathcal{M}_\theta, \mathsf{Adm}_\theta, \mathsf{Rej}_\theta)$ for the instance on $(\psi_\theta, X_\theta, Y_\theta)$ defined in Section 3.2.

Example 5.2 (Reading the running query). We run the left-side construction on the signed system of Example 3.3 along 00, for the failing query $x \le y$ and, for contrast, for the entailed query $y \le x$, which exchanges the roots. Neither half is the right label reader of $x \le y$, which would use the order-dual system. The last three columns give the latch, readiness, and acceptance status at each cut, before the next letter is read.

Query	cut j	U_j	L_j	b_j	ready	status
$x \le y$	0	$\{x^+\}$	$\{y^+\}$	0	false	accepted (child)
	1	$\{a^-\}$	$\varnothing$	0	false	rejected
	2	$\varnothing$	$\varnothing$	0	false	rejected
$y \le x$	0	$\{y^+\}$	$\{x^+\}$	0	false	accepted (child)
	1	$\{a^-\}$	$\{a^-\}$	0	true	accepted (ready)
	2	$\varnothing$	$\varnothing$	1	false	accepted (latch)

For $x \le y$, the word 0 is a hole: its final state is rejected, and its only comparison $(0, 1)$ is not admitted. The word 00 is also a hole. The empty word is accepted by the child test; it is not a left-unsafe word. For the entailed reverse query, readiness at cut 1 sets the latch at cut 2, so the empty upper and lower sets there do not cause rejection.

Rejection alone does not certify a hole. In the cross event of Example 4.15, the run on 11 has $b_j = 0$ throughout and ends in $(\{b^+\}, \varnothing, 0)$, a rejected state. Nevertheless $(0, 1)$ is cross-admitted, so 11 is not a hole.

5.3 The Rejected-Path Property

For a general reader $(\mathcal{M}, \mathsf{Adm}, \mathsf{Rej})$, write $q_j = \delta(q_0, w[:j])$. If $q_m \in \mathsf{Rej}$, let $J = J(w)$ be the least cut with $q_J \in \mathsf{Rej}$, and put $r(w) = |w| - J(w)$. The reader has the *rejected-path property* if it satisfies *betweenness* and the *admission cone*:

Betweenness
if two prefixes of a word end in Rej, every intermediate prefix also ends in Rej.

Admission cone
in a word ending in Rej, every comparison (s, e) that satisfies $\mathsf{Adm}(q_s, q_e)$ has $e \le J$.

Lemma 5.3 (Rejected paths of label readers). *The label reader, and hence both the left and the right label reader, satisfies the rejected-path property, without any satisfiability assumption.*

Proof. Rejection at a cut implies $\neg\mathsf{child}(U)$. Every reachable U is upward closed under D_ψ: this holds at q_0 and is preserved by each E_i, using (15) and transitivity. If $E_i U \neq \varnothing$, (15) supplies an upper constructor with root z reached from some $u \in U$. Upward closure puts $z \in U$, giving

Table 2. Hypotheses used by the word bounds.

Result	Hypotheses on the reader and word
Rejected extension (Lemma 5.6)	Admission cone; a hole and a rejected extension
Bounded tail (Theorem 6.1)	Admission cone and betweenness; some hole
Return gap and depth (Lemma 7.5 and Theorem 7.7)	Any reader; a rejected word, nonempty for depth
Bounded-depth compression (Theorem 8.1)	Any reader; a nonempty hole of bounded depth

$\mathsf{child}(U)$, a contradiction. Thus U becomes empty on the next letter and remains empty thereafter. Both disjuncts of (39) require a nonempty upper set at their second cut. Any admission therefore has $e \le J$, proving the admission cone.

For betweenness, after a rejected cut every later upper set is empty. A later state can cease to be rejected only through readiness or an already set latch; a child can no longer occur. Once readiness occurs, it is present at that cut and sets the latch at the next one, making every subsequent state accepted. Consequently it cannot occur between two rejected cuts. This proves betweenness for either instance of the construction. □

The proof gives more than the admission cone: it excludes later admissions even for pairs that are not comparisons. The rejected-path property retains only what the tail proof needs; this stronger restriction may help sharpen subtype-specific bounds. Rejection need not be absorbing: a later ready state can become accepted permanently.

5.4 Unsafe Words and Holes

Theorem 5.4 (Unsafe words are holes). *If* $\mathrm{Sat}_c(\phi)$*, then for either side* θ*,*

$$w \text{ is } \theta\text{-unsafe} \iff w \text{ is a hole of } (\mathcal{M}_\theta, \mathsf{Adm}_\theta, \mathsf{Rej}_\theta). \tag{41}$$

Proof. By Theorem 4.9, unsafety is equivalent to absence of a label clash in the side's four-spine extension. Satisfiability excludes inherited clashes in either side system, so Theorem 4.13 identifies this with the simultaneous negation of (34). The run and test formulas (37)–(39) identify those negations precisely with (40), using the right label reader for the right side. Every implication is reversible. The argument uses no positive-arity assumption; the letter-free child test also handles the empty word over the empty alphabet. □

5.5 Consequences for General Readers

For any reader satisfying the rejected-path property, the following consequences hold.

Corollary 5.5 (Hole criterion). *A rejected word* w *is a hole iff it has no admitted comparison* (s, e) *with* $e \le J(w)$*.*

Lemma 5.6 (Rejected extension). *If* v *is a hole and* w *is a rejected word with* $v \preceq w$*, then* w *is a hole.*

Proof. Since v is rejected, $J(w) = J(v) \le |v|$. An admitted comparison (s, e) of w has $e \le J(w) \le |v|$ by the admission cone, so it is also a comparison of v, with the same cut states. This contradicts the assumption that v is a hole. □

Table 2 separates the subtype-specific rejected-path property from the general word arguments.

Example 5.7 (Loop deletion can create an admission). Consider the reader on $\{0,1\}$ with initial state q, rejected set $\mathrm{Rej} = \{r\}$, sole admitted pair (q,b), and all transitions as follows:

state	q	a	b	c	r	h
0	a	a	c	h	r	h
1	h	b	h	r	r	h

Here h is an accepted sink. State q occurs only at cut 0, so a comparison is admitted only if it starts at cut 0 and ends at a cut in state b. Deleting the loop $a \xrightarrow{0} a$ between the equal-state cuts 1 and 2 of 00101 gives 0101; both runs end in the rejected state r:

$$00101: \quad q \xrightarrow{0} a \xrightarrow{0} a \xrightarrow{1} b \xrightarrow{0} c \xrightarrow{1} r,$$
$$0101: \quad q \xrightarrow{0} a \xrightarrow{1} b \xrightarrow{0} c \xrightarrow{1} r.$$

In 00101, state b occurs only at cut 3, so the only candidate is $(0,3)$. It is not a comparison: the suffix 01 at cut 3 is not a prefix of 00101. Thus 00101 is a hole. In 0101, state b occurs at cut 2, and the candidate $(0,2)$ is a comparison, since the suffix 01 at cut 2 is a prefix of 0101. Its states are (q,b), so it is admitted, and the shortened word is not a hole.

This reader has the rejected-path property. State r is absorbing, which proves betweenness. Any admission must start at q and end at b; q occurs only initially, and b can occur only before the first visit to r. Hence every admitted comparison of a rejected word ends before J, proving the admission cone. This example motivates the simultaneous comparison summaries in Section 8.

6 Bounded Rejected Tails

The first bound selects a hole whose rejected tail is short. Throughout, $(\mathcal{M}, \mathrm{Adm}, \mathrm{Rej})$ is an N-state reader over a finite alphabet Γ, with initial state q_0 and transition function δ, that has the rejected-path property (Section 5.3). By Lemma 5.6, every rejected extension of a hole is a hole.

Theorem 6.1 (Bounded rejected tail). *Let $(\mathcal{M}, \mathrm{Adm}, \mathrm{Rej})$ be a reader over a finite alphabet with N states that satisfies betweenness and the admission cone. If it has a hole, then it has a hole w whose rejected tail satisfies $r(w) \le 4N - 2$.*

The theorem *selects* a hole. It bounds neither the tail of every hole nor a continuation of each fixed first rejected prefix. The proof may replace the prefix before the rejected tail.

The Rejected Graph. Call a letter i *safe* at a state $q \in \mathrm{Rej}$ if $\delta(q,i) \in \mathrm{Rej}$. The *rejected graph* G has as vertices the reachable states in Rej and an edge $q \xrightarrow{i} \delta(q,i)$ for every safe letter. We count edges by letters, so a state whose distinct letters lead to the same successor has separate outgoing edges. A vertex is *cyclic* if it lies on a cycle of G. If the initial state is rejected, the empty word is a hole with tail 0; we assume from now on that $q_0 \notin \mathrm{Rej}$. The proof distinguishes whether some cyclic vertex has two safe letters.

6.1 Branching Cycles

Repeating a cycle sufficiently often and then choosing a different next letter destroys every comparison whose right endpoint lies in the preceding prefix.

Lemma 6.2 (Rotation). *Let $v, \kappa \in \Gamma^*$ with $|v| = a$ and $|\kappa| = \nu \ge 1$, let the exponent μ satisfy $\mu\nu \ge a + \nu$, and let $b \ne \kappa[0]$ be a letter. Then the word $w = v\,\kappa^\mu\,b$ has no comparison (s,e) with $e \le a$.*

Proof. Let (s,e) be a comparison with $e \le a$ and period $\varpi = e - s \le a$, so $w[k] = w[k-\varpi]$ for $e \le k < |w|$. For the ν positions $k = a + \varpi, \dots, a + \varpi + \nu - 1$, both k and $k - \varpi$ lie in the block κ^μ, because $\varpi + \nu \le \mu\nu$. Writing $i = (k - \varpi - a) \bmod \nu$, which ranges over all residues, we obtain

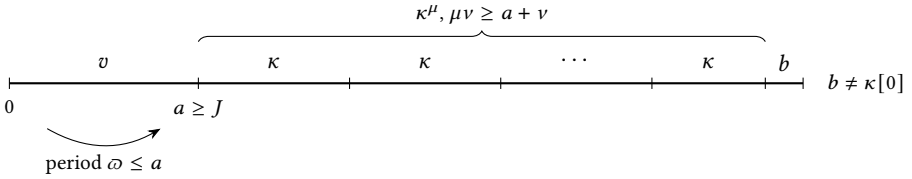


Fig. 7. The branching case (Lemma 6.3). A period $\varpi \leq a$ that survives the block κ^μ makes κ invariant under rotation by ϖ and then predicts the letter $\kappa[0]$ at the end, where the word has $b \neq \kappa[0]$.

$\kappa[(i+\varpi) \bmod \nu] = \kappa[i]$ for all i: the cyclic word κ is invariant under rotation by ϖ. The last position $k = a + \mu\nu$ also satisfies $k \geq e$ and $k - \varpi \geq a$, so $w[k - \varpi] = \kappa[(\mu\nu - \varpi) \bmod \nu] = \kappa[(-\varpi) \bmod \nu]$, which equals $\kappa[0]$ by the rotation invariance with $i = (-\varpi) \bmod \nu$. But $w[k] = b \neq \kappa[0]$. □

Lemma 6.3 (Branching cycle). *If some cyclic vertex of G has two safe letters, the reader has a hole with tail at most $4N - 2$.*

Proof. Let q be such a vertex, v a shortest word with $\delta(q_0, v) = q$, so $a = |v| \leq N - 1$, and κ the label of a simple cycle of G through q, so $1 \leq \nu = |\kappa| \leq N$. Let $b \neq \kappa[0]$ be another safe letter at q and $\mu = \lceil (a + \nu)/\nu \rceil$. The word $w = v\,\kappa^\mu\,b$ (Figure 7) is rejected: after v it stays in G. Its first rejected cut satisfies $J \leq a$, so by the admission cone every admitted comparison has $e \leq a$, and by Lemma 6.2 there is none. Thus w is a hole. Since $\mu\nu < a + 2\nu$, its tail is at most $|w| = a + \mu\nu + 1 \leq 2a + 2\nu \leq 4N - 2$. □

The lemma does not assume that a hole exists. For a satisfiable ϕ, a reachable rejected cycle of its label reader with a second safe letter therefore already refutes entailment. For a fixed finite alphabet, reachability and cycle detection give a sufficient test polynomial in the reader graph size, applicable to either side at any variance.

6.2 Forced Cycles

In the remaining case, every cyclic vertex of G has exactly one safe letter: at least one, the next edge of its cycle, but not two. Then the path of G from a cyclic vertex is *forced*: it follows the unique safe letter and returns to its start. We now use the existence of a hole.

Let w_0 be a hole with first rejected cut J_0 and length m_0. If its tail is shorter than N, we are done. Otherwise, the run of w_0 from cut J_0 to cut m_0 stays in G by betweenness and visits more than N cuts, so it repeats a state; from that point it is on a forced cycle, and so is its last state q_{m_0}. Extending w_0 along the forced cycle forever gives an infinite word $\Omega = w_0\,\kappa^\omega$, where κ spells the cycle from q_{m_0}. Every finite prefix of Ω of length at least m_0 is a rejected extension of w_0, hence a hole. Write $\Omega[s{:}]$ for the infinite suffix from cut s, and call a pair of cuts $s < e$ an *equal-suffix pair* if $\Omega[s{:}] = \Omega[e{:}]$.

Lemma 6.4 (No admitted equal-suffix pair). *Ω has no equal-suffix pair (s, e) with $\mathrm{Adm}(q_s, q_e)$.*

Proof. Such a pair is a comparison of every prefix of Ω of length at least $\max(m_0, e)$, with the same cut states, and these prefixes are holes. □

Let $\nu \leq N$ be the least period of κ^ω, and let A be the *earliest periodic onset*, the least cut from which Ω is ν-periodic: $\Omega[k] = \Omega[k + \nu]$ for all $k \geq A$. Then $A \leq m_0$, and if $A > 0$, minimality gives

$$\Omega[A - 1] \neq \Omega[A - 1 + \nu]. \tag{42}$$

Lemma 6.5 (Earliest onset). *Every equal-suffix pair (s, e) of Ω has $s \geq A$.*

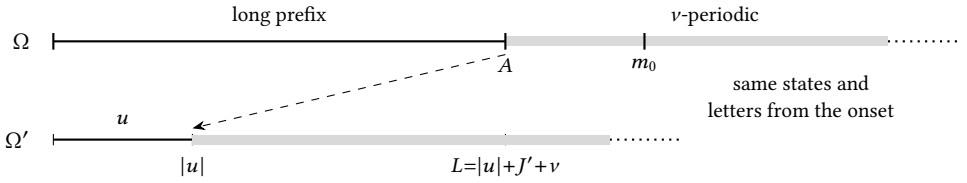


Fig. 8. The forced case. The prefix of Ω before the earliest periodic onset A is replaced, for $A > 0$, by a shortest path to q_{A-1} followed by the original letter $\Omega[A-1]$; for $A = 0$, the replacement prefix is empty. Every cut of the resulting word Ω' from its first rejected cut J' on is rejected, and its prefix of length $L = |u| + J' + \nu$ is a hole.

Proof. Let $\varpi = e - s$. The equality $\Omega[s:] = \Omega[e:]$ gives $\Omega[k] = \Omega[k + j\varpi]$ for all $k \geq s$ and $j \geq 0$. For $k \geq s$ and j large enough that $k + j\varpi \geq A$, we get $\Omega[k+\nu] = \Omega[k+\nu+j\varpi] = \Omega[k+j\varpi] = \Omega[k]$. So Ω is ν-periodic from s, and $s \geq A$ by minimality of A. □

Compressing the Prefix. The word Ω can be long before its periodic part; we replace that prefix by a shorter one. If $A = 0$, let $u = \varepsilon$. Otherwise, let $u = v'\,\Omega[A-1]$, where v' is a shortest word with $\delta(q_0, v') = q_{A-1}$; then $|u| \leq N$ and $\delta(q_0, u) = q_A$. Let $\Omega' = u\,\Omega[A:]$ (Figure 8). The suffix of Ω' after cut $|u|$ is the suffix of Ω after cut A, read from the same state, so cuts $|u| + k$ of Ω' and $A + k$ of Ω carry the same states and the same suffixes. The letter before the onset is preserved, so by (42) the earliest periodic onset of Ω' is exactly $|u|$.

Lemma 6.6 (No admitted pair after compression). *Ω' has no equal-suffix pair (s, e) with $\mathrm{Adm}(q'_s, q'_e)$, where q'_i are the cut states of Ω'.*

Proof. By Lemma 6.5 applied to Ω', such a pair has $s \geq |u|$. It corresponds to the pair $(s - |u| + A,\ e - |u| + A)$ of Ω, with the same states and suffixes, contradicting Lemma 6.4. □

Rejection After the First Rejected Cut. Cut $|u| + k$ of Ω' carries the state of cut $A + k$ of Ω. Every cut of Ω from $m_0 \geq A$ on is rejected, so every cut of Ω' from $|u| + m_0 - A$ on is rejected. Let J' be the first rejected cut of Ω'. Every later cut lies between J' and a rejected cut further on, so by betweenness every cut of Ω' from J' on is rejected. The truncation therefore need not wait until Ω' returns to the original state and phase of the cycle.

Lemma 6.7 (Periodic truncation). *Let J' be the first rejected cut of Ω' and $L = |u| + J' + \nu$. Then the prefix $w' = \Omega'[:L]$ of length L is a hole.*

Proof. Since $L \geq J'$, the word w' is rejected, and its first rejected cut is J'. Suppose (s, e) is an admitted comparison of w'. By the admission cone, $e \leq J'$, so its period satisfies $\varpi = e - s \leq J'$, and $\Omega'[k] = \Omega'[k - \varpi]$ for $e \leq k < L$. Put $h = \max(e, |u| + \varpi)$. Since $e \leq J'$ and $\varpi \leq J'$, we have $h + \nu \leq L$, so the equality holds at the ν consecutive positions $k = h, \ldots, h + \nu - 1$. These satisfy $k - \varpi \geq |u|$, so both k and $k - \varpi$ lie in the ν-periodic part of Ω'. Equality on ν consecutive positions of a ν-periodic word propagates to all later positions, so $\Omega'[k] = \Omega'[k - \varpi]$ for all $k \geq e$; that is, (s, e) is an equal-suffix pair of Ω' with $\mathrm{Adm}(q'_s, q'_e)$. This contradicts Lemma 6.6. □

Proof of Theorem 6.1. If the initial state is rejected, the empty word is a hole. If a cyclic vertex of G has two safe letters, Lemma 6.3 gives a hole with tail at most $4N - 2$. Otherwise, starting from any hole, either its tail is less than N, or Lemma 6.7 gives a hole with tail $L - J' = |u| + \nu \leq 2N$. Since $N \geq 1$, all these bounds are at most $4N - 2$. □

The essential choice is the *earliest* periodic onset: preserving its preceding letter excludes equal-suffix pairs starting in the replacement prefix. The admission cone reduces the rest to the finite window of Lemma 6.7, and betweenness lets that window start right after the replacement prefix. If the alphabet is empty, the only word is ε.

7 From Short Tails to Bounded Depth

A hole with a short rejected tail can still be arbitrarily long. The second bound measures its complexity by the depth of a canonical hierarchy of block decompositions, and shows that this depth is bounded in terms of the tail. The hierarchy is the device that makes comparisons tractable: each of its levels collects the starting points of the comparisons that end at one particular cut.

7.1 The Canonical Desubstitution Hierarchy

Let Γ be an alphabet and $u \in \Gamma^+$ a nonempty word with last letter ℓ, the *marker*. Then u factors uniquely as

$$u = X_0\,\ell\,X_1\,\ell\cdots X_{h-1}\,\ell \qquad \text{with } X_i \in (\Gamma \setminus \{\ell\})^*.$$

The *desubstitution* of u is the word $[X_0][X_1]\cdots[X_{h-1}]$ of length h over the alphabet of *blocks* $(\Gamma \setminus \{\ell\})^*$: each block $[X_i]$, a word over Γ, is a single letter of the new word. Expanding every block $[X]$ to $X\ell$ recovers u. The name follows the use of desubstitution and derived sequences in symbolic dynamics [Durand 1998].

Definition 7.1 (Hierarchy and depth). The *canonical hierarchy* of a nonempty word w is the sequence $u_0 = w$ and u_{j+1} = the desubstitution of u_j, with marker ℓ_j, the last letter of u_j. A word is *unary* if all its letters are equal; this notion concerns words and is unrelated to the arity of f. The *depth* of w is the least j such that u_j is unary.

If u_j is not unary, then $|u_{j+1}| < |u_j|$, because some letter of u_j differs from the marker. Hence the depth is well defined and $\mathrm{depth}(w) \le |w| - 1$.

Example 7.2 (A hierarchy of depth three). Let $w = 01001101$. Its marker is 1, and $w = (0)\,1\,(00)\,1\,(\varepsilon)\,1\,(0)\,1$, so $u_1 = \alpha_1\,\beta_1\,\gamma_1\,\alpha_1$ with the letters $\alpha_1 = [0]$, $\beta_1 = [00]$, and $\gamma_1 = [\varepsilon]$. The marker of u_1 is α_1, and $u_1 = (\varepsilon)\,\alpha_1\,(\beta_1\gamma_1)\,\alpha_1$, so $u_2 = \alpha_2\,\beta_2$ with $\alpha_2 = [\varepsilon]$ and $\beta_2 = [\beta_1\gamma_1]$. Finally, $u_2 = (\alpha_2)\,\beta_2$ gives $u_3 = \alpha_3$ with $\alpha_3 = [\alpha_2]$, which is unary. Thus $\mathrm{depth}(w) = 3$, with lengths 8, 4, 2, 1.

The hierarchy is compatible with comparisons: a comparison that does not end at the last cut of a word is a comparison of the next level, refined by a common suffix of two blocks.

Lemma 7.3 (Alignment). *Let u be the expansion of a nonempty block word $v = [X_0]\cdots[X_{h-1}]$ with marker ℓ, and let $s < e < |u|$. Then (s, e) is a comparison of u iff s and e lie in the expansions $X_{i_s}\ell$ and $X_{i_e}\ell$ of two blocks with $i_s < i_e$, at the same distance from the markers that end these blocks, the block suffixes beginning at these cuts are equal, and $(i_s + 1, i_e + 1)$ is a comparison of v.*

Proof. Let e' be the first marker position at or after e; it exists because u ends with ℓ. If (s, e) is a comparison, then $u[s+i] = u[e+i]$ for $0 \le i \le e' - e$, and since no block contains ℓ, $s' = s + (e' - e)$ is the first marker position at or after s. So s and e lie at the same distance from the markers ending their blocks, the letters between agree, and $i_s < i_e$ because $s' < e'$. The remaining suffixes $u[s'+1{:}]$ and $u[e'+1{:}]$ are the expansions of $[X_{i_s+1}]\cdots$ and $[X_{i_e+1}]\cdots$, and the second is a prefix of the first. An expansion determines its block word, since the markers delimit the blocks, so the block word after i_e is a prefix of the block word after i_s: $(i_s + 1, i_e + 1)$ is a comparison of v. The converse reverses these steps. □

7.2 Cores and Supports

We now relate the levels of the hierarchy to the cuts of the original word. Every position of u_j carries a *core*, which is either an original cut of w (an *ordinary* core) or a symbolic *start core*. At level 0, position i has core i. At level $j+1$, the first position has a new start core $\star_{j+1}$, and position $i \geq 1$ inherits the core of the $(i-1)$-st occurrence of the marker ℓ_j in u_j, counting occurrences from 0. For a level j, let C_j be the set of ordinary cores of u_j.

The core of a block is thus the core of the marker that precedes it, and the block is read from that marker. A block $[X]$ therefore has two expansions. Its *ordinary expansion* $X\ell$ places the marker after the block and reconstructs the word. Its *shifted expansion* ℓX places the preceding marker before the block: it is the factor of w that starts at the core of the block, read from the reader state at that core. The first block has no preceding marker, which is why it receives a symbolic start core. Iterating the shifted expansion down to level zero gives a map ξ_j from level-j letters to words: $\xi_0(a) = a$, and $\xi_{j+1}([X])$ is $\xi_j(\ell_j)$ followed by the images under ξ_j of the letters of X.

Lemma 7.4 (Cores). *Let w be nonempty.*

(1) $C_{j+1} \subseteq C_j$. The start cores of u_j occupy an initial segment of its positions, are pairwise distinct, and number at most j; the ordinary cores increase from left to right.

(2) If $C_j \neq \emptyset$ and $t_j = \max C_j$, then

$$C_{j+1} = \{\, s \mid s < t_j \text{ and } w[t_j{:}] \preceq w[s{:}] \,\}.$$

In particular, (s, t_j) is a comparison of w for every $s \in C_{j+1}$. If $C_j = \emptyset$, then $C_{j+1} = \emptyset$.

The cut t_j is the core of the last position of u_j, the *terminal core*, and we call $w[t_j{:}]$ the *terminal core word* of level j. Part 2 says that the next level retains exactly the earlier occurrences of the terminal core word. Hence each level of the hierarchy collects the left endpoints of the comparisons whose right endpoint is the terminal core of the previous level.

The first two levels of Example 7.2 are aligned below. The upper rows list the cuts of w and the letter after each cut; the markers $\ell_0 = 1$ are in bold. The lower rows place each letter of u_1 under its shifted expansion, which starts at its core.

cut of w	0	1	2	3	4	5	6	7	8
letter after the cut	0	**1**	0	0	**1**	**1**	0	**1**	
letter of u_1	α_1	β_1			γ_1	α_1		last ℓ_0	
core	$\star_1$	1			4	5		$t_0 = 7$	

The markers at the cuts $1, 4, 5$ are the occurrences $0, 1, 2$ of ℓ_0, so they become the cores of the positions $1, 2, 3$ of u_1; the first position receives $\star_1$. Thus $C_0 = \{0, \ldots, 7\}$ and $C_1 = \{1, 4, 5\}$. The shifted expansions are $\xi_1([X]) = 1X$, and indeed $w[1{:}4] = 100 = \xi_1(\beta_1)$ and $w[4{:}5] = 1 = \xi_1(\gamma_1)$, whereas the ordinary expansion of $\beta_1 = [00]$ is 001. The first letter has no preceding marker and covers only $w[0{:}1] = 0$. The last marker is not inherited: its cut is the terminal core $t_0 = 7$. The terminal core is the original cut attached to the last letter of a level; at level zero it is 7, whereas the terminal cut of w is 8. The terminal core word $w[7{:}] = 1$ occurs exactly at the earlier cuts $1, 4, 5$, as Part 2 predicts. At level 1, the terminal core is $t_1 = 5$, and $w[5{:}] = 101 = \xi_1(\alpha_1)\, w[t_0{:}]$ occurs nowhere earlier. Accordingly, $u_2 = \alpha_2\beta_2$ has cores $\star_2, \star_1$: its second position inherits $\star_1$ from the first occurrence of α_1, so $C_2 = \emptyset$.

The proof of Lemma 7.4 is by induction on j and is given in Appendix B. Its main invariant states that ξ_j is injective and that the factor of w from an ordinary core of level j to the next core is the image under ξ_j of the letter at that core. For the last core, set $t_{-1} = |w|$, so that $w[t_{-1}{:}] = \varepsilon$;

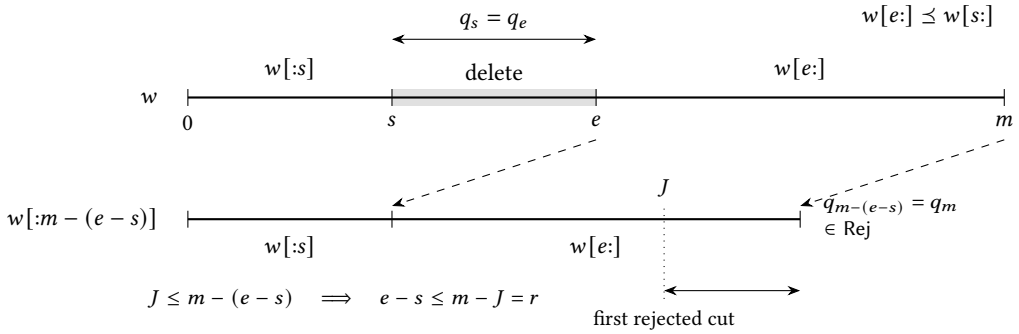


Fig. 9. The return gap (Lemma 7.5). A comparison makes loop deletion produce the prefix $w[:s]w[e:] = w[:m-(e-s)]$. Equal states at s and e preserve the final rejected state, so the shortened prefix cannot end before J. The position of J relative to s, e is schematic; only $J \le m-(e-s)$ is asserted.

then $w[t_j:] = \xi_j(\ell_j)\, w[t_{j-1}:]$. The injectivity of ξ_j together with Lemma 7.3 identifies the earlier occurrences of the terminal core word with the cores of the next level.

7.3 The Return Gap

Let $(\mathcal{M}, \mathrm{Adm}, \mathrm{Rej})$ be any N-state reader. For a rejected word w, let J be its first rejected cut and $r = |w| - J$. Its level-j *state support* is $\mathcal{S}_j = \{q_i : i \in C_j\}$, the set of original reader states at the ordinary cores. These supports decrease by Lemma 7.4.

Lemma 7.5 (Return gap). *If (s, e) is a comparison of a rejected word w and $q_s = q_e$, then $e - s \le r$.*

Proof. Put $m = |w|$ and $\varpi = e - s$. The comparison implies $w[:s]w[e:] = w[:m-\varpi]$. Deleting the equal-state loop therefore gives $q_{m-\varpi} = q_m \in \mathrm{Rej}$. By minimality of J, $m - \varpi \ge J$, so $\varpi \le m - J = r$. □

This holds for every reader, without the rejected-path property and without assuming that w is a hole. In particular, tail zero excludes equal-state comparisons. Figure 9 shows the loop deletion and the resulting rejected prefix.

7.4 The Depth Bound

Lemma 7.6 (Equal-support width). *Let w be a nonempty rejected word with tail r, and let $j \le j'$ be levels with $\mathcal{S}_j = \mathcal{S}_{j'} \ne \emptyset$. Then $j' - j + 1 \le r + 1$.*

Proof. For $j = j'$, the bound is immediate. Assume $j < j'$. Since $\mathcal{S}_{j'} \ne \emptyset$, all of $C_j, \dots, C_{j'}$ are nonempty. By Lemma 7.4, $t_{i+1} < t_i$ for $j \le i < j'$, so $j' - j \le t_j - t_{j'}$. The state q_{t_j} lies in $\mathcal{S}_j = \mathcal{S}_{j'}$, so $q_s = q_{t_j}$ for some $s \in C_{j'}$. As $C_{j'} \subseteq C_{j+1}$, Lemma 7.4 shows that (s, t_j) is a comparison, and Lemma 7.5 gives $t_j - s \le r$. Since $s \le t_{j'}$, we obtain $j' - j \le t_j - t_{j'} \le t_j - s \le r$. □

Theorem 7.7 (Depth from tail). *For every N-state reader, each nonempty rejected word with tail r has* $\mathrm{depth}(w) \le 2N(r+1) - 1$.

Proof. Let $d_w = \mathrm{depth}(w)$ and $R = r + 1$. The supports $\mathcal{S}_0 \supseteq \mathcal{S}_1 \supseteq \cdots \supseteq \mathcal{S}_{d_w}$ are nested, so the distinct nonempty ones form a strictly decreasing chain of nonempty subsets of an N-element set, and there are at most N of them. By Lemma 7.6, each of them is the support of at most R levels. Hence at most NR levels have nonempty support. If every level up to d_w does, then $d_w + 1 \le NR$. Otherwise, let j_0 be the first level with $\mathcal{S}_{j_0} = \emptyset$; then $1 \le j_0 \le NR$, since C_0 contains the cuts

$0, \ldots, |w| - 1$. All cores of u_{j_0} are start cores, so $|u_{j_0}| \le j_0$ by Lemma 7.4. Each further level up to d_w strictly decreases the length, so $d_w - j_0 \le |u_{j_0}| - 1 \le j_0 - 1$, and $d_w \le 2j_0 - 1 \le 2NR - 1$. □

Theorem 7.7 holds for every nonempty rejected word, hole or not; combined with Theorem 6.1, it bounds the depth of some hole by $2N(4N - 1) - 1$.

8 Bounded Depth Implies Bounded Length

The last bound turns a bound on the depth into a bound on the length. Like the depth bound, it holds for every reader: it uses neither the admission cone nor any property of non-structural subtype entailment.

Theorem 8.1 (Bounded-depth search). *Let $(\mathcal{M}, \mathsf{Adm}, \mathsf{Rej})$ be any reader with N states over a finite alphabet. If it has a nonempty hole of depth at most d, then it has a nonempty hole of length at most $\mathsf{Len}(N, d)$, where Len is the elementary function given in (48) below. The replacement can be chosen of depth at most d. The same statement holds for any lettered monitor with N states (defined in Section 8.1), even over an infinite alphabet.*

The proof first bounds the number of distinct letters at every positive level, then shortens a hole level by level along its hierarchy. Shortening each block of a level independently fails, because comparisons relate *different* blocks through equal suffixes. We therefore shorten all blocks of a level *simultaneously*, preserving a finite summary of the whole tuple. As Example 5.7 shows, preserving the final reader state alone does not preserve holes.

8.1 Lettered Monitors and Their Derivatives

To describe each level of the hierarchy as a hole problem in its own right, we generalize readers to *lettered monitors*. A lettered monitor $\mathcal{M}$ over an alphabet Γ, which may be infinite, consists of a finite set Q of states with a start state ζ_0, a transition function $\delta : Q \times \Gamma \to Q$, a *final target* predicate $T^{\mathrm{fin}} \subseteq Q \times \Gamma$, and two admission relations Λ and Λ^{fin} on *labels*, the second for comparisons that end at the final cut. Reading a word v assigns the state $\zeta_{i+1} = \delta(\zeta_i, v[i])$ to cut $i + 1$. The label of cut 0 is a symbol $\triangleright$, and the label of cut $i > 0$ is the pair $(\zeta_{i-1}, v[i-1])$ of the state at cut $i - 1$ and the letter read from it. A nonempty word v is a *hole* of $\mathcal{M}$ if its last label (ζ, a) satisfies $T^{\mathrm{fin}}(\zeta, a)$, no terminal comparison $(s, |v|)$ satisfies Λ^{fin}, and no other comparison (s, e) satisfies Λ, each relation being applied to the labels of the two cuts. A reader $(\mathcal{M}, \mathsf{Adm}, \mathsf{Rej})$ determines the lettered monitor with states Q, the same transition δ, $T^{\mathrm{fin}}(\zeta, a)$ iff $\delta(\zeta, a) \in \mathsf{Rej}$, and $\Lambda = \Lambda^{\mathrm{fin}}$ given by Adm on the states that the labels denote: $\triangleright$ denotes q_0 and (ζ, a) denotes $\delta(\zeta, a)$. The reader and this lettered monitor have the same nonempty holes. As with rejection, the final target only marks a candidate: a word whose last label satisfies T^{fin} is a hole only if no comparison is admitted. The predicate T^{fin} and the relations take labels, and not only the states they denote, because the monitors of higher levels must evaluate them at positions *inside* blocks, which requires the letters. Block alphabets $(\Gamma \setminus \{\ell\})^*$ are generally infinite, even for finite Γ, so lettered monitors allow infinite alphabets. The compression argument never uses finiteness of an alphabet: it works with the finitely many letters that one hole uses and with finitely many summary values.

Fix a marker $\ell \in \Gamma$, a block $X \in (\Gamma \setminus \{\ell\})^*$, and a state $\zeta \in Q \sqcup \{\star\}$. The *shifted block* of X read from $\zeta \in Q$ is the word ℓX read from ζ; from the new symbol $\star$, it is X read from ζ_0. It has a label $lab_p(\zeta, X)$ at each position $p = 0, \ldots, |X|$: for $\zeta \in Q$, the label of the cut after $\ell\, X[:p]$, so that position 0 carries the *marker label* (ζ, ℓ); for $\zeta = \star$, the label $\triangleright$ at position 0 and the label of the cut after $X[:p]$ at positions $p \ge 1$. The *exit state* $\delta_\ell(\zeta, X)$ is the state reached at the end of the shifted block.

The derivative $\mathcal{M}_\ell$ defined next reads the block word. Each of its letters represents an entire block, so the label of a derivative cut keeps both the block and its entry state, the state from which the shifted block is read; this lets the derivative recover every original label inside the block. With these labels, $\mathcal{M}_\ell$ checks three kinds of obligations of the expanded word. Its admission Λ_ℓ compares two positions in different blocks that lie at the same distance from the ends of their blocks, with equal letters after them. The second clause of its terminal admission $\Lambda_\ell^{\mathrm{fin}}$ compares a position anywhere in an earlier block with the final cut, which carries the label of the closing marker. Its target T_ℓ^{fin} checks the original target together with all terminal comparisons that start in the last block.

Definition 8.2 (Derivative). Let $\mathcal{M}$ be a lettered monitor over Γ with states Q, and $\ell \in \Gamma$. The *derivative* $\mathcal{M}_\ell$ is the lettered monitor over the block alphabet $(\Gamma \setminus \{\ell\})^*$ with states $Q \sqcup \{\star\}$, start state $\star$, transition function δ_ℓ, and the following relations on labels (ζ, X) and (ζ', X'):

- $\Lambda_\ell((\zeta, X), (\zeta', X'))$ holds if $\Lambda\big(lab_p(\zeta, X),\ lab_{p'}(\zeta', X')\big)$ for some positions $p \le |X|$ and $p' \le |X'|$ from which the two blocks have a common suffix, $X[p{:}] = X'[p'{:}]$;
- $\Lambda_\ell^{\mathrm{fin}}((\zeta, X), (\zeta', X'))$ holds if $\Lambda_\ell((\zeta, X), (\zeta', X'))$ holds, or if $\Lambda^{\mathrm{fin}}\big(lab_p(\zeta, X), (\delta_\ell(\zeta', X'), \ell)\big)$ for some $p \le |X|$;
- $T_\ell^{\mathrm{fin}}(\zeta, X)$ holds if $T^{\mathrm{fin}}(\delta_\ell(\zeta, X), \ell)$ holds and $\Lambda^{\mathrm{fin}}\big(lab_p(\zeta, X), (\delta_\ell(\zeta, X), \ell)\big)$ fails for every $p \le |X|$.

Every relation involving the label $\triangleright$ of $\mathcal{M}_\ell$ is false.

The derivative's cut zero is an artificial boundary. Original comparisons beginning at cut zero are represented inside the first shifted block, whose entry state is $\star$; they are still tested through the label $(\star, X_0)$. Setting relations at the derivative's new start label to false therefore discards no original comparison.

For a concrete instance, use the reader of Example 5.7 on 0101 and marker 1. Its block word is $[0][0]$. The derivative starts at $\star$ and reads $[0]$ to state a; from a, the shifted block 10 ends at c. The table shows which derivative label represents each original cut.

original cut	0	1	2	3	4
state	q	a	b	c	r
original label	$\triangleright$	$(q, 0)$	$(a, 1)$	$(b, 0)$	$(c, 1)$
derivative label		$(\star, [0])$		$(a, [0])$	final
position p in its block	0	1	0	1	

The two derivative labels sit at the derivative cuts 1 and 2, and each represents the original cuts of one shifted block: the first block is read from the start, and the second begins with the marker label $(a, 1)$. The final original cut carries the label $(c, 1)$ of the closing marker, read from the final derivative state c. For the two derivative labels, the common suffix 0 selects position zero in both blocks. Their original labels denote q and $\delta(a, 1) = b$, so Λ_1 holds. The derivative comparison $(1, 2)$ is terminal and its Λ_1^{fin} relation therefore holds as well. It detects the original nonterminal comparison $(0, 2)$, including its start at the original cut zero.

Lemma 8.3 (Holes under desubstitution). *Let v be a nonempty word of blocks that avoid ℓ, and let u be its expansion. Then u is a hole of $\mathcal{M}$ iff v is a hole of $\mathcal{M}_\ell$. Consequently, if $\mathcal{M}_0$ is a reader with N states and $\mathcal{M}_{j+1} = (\mathcal{M}_j)_{\ell_j}$ along the canonical hierarchy of w, then w is a hole of $\mathcal{M}_0$ iff u_j is a hole of $\mathcal{M}_j$, and $\mathcal{M}_j$ has $N + j$ states.*

Proof. Write $v = [X_0] \cdots [X_{h-1}]$, so $u = X_0 \ell X_1 \ell \cdots X_{h-1} \ell$, and let $\zeta'_0 = \star, \zeta'_1, \ldots, \zeta'_h$ be the derivative run. Let a_i be the cut where X_i begins and $b_i = a_i + |X_i|$ the position of its closing marker. Partition all nonfinal cuts of u into the intervals $[a_i, b_i]$. Induction on i gives the label $lab_p(\zeta'_i, X_i)$

at cut $a_i + p$, for $0 \le p \le |X_i|$; the final cut has label (ζ'_h, ℓ). The label of cut $i+1$ of v is (ζ'_i, X_i), so it determines every label in this block.

Suppose first that u is a hole. Its final target holds, and none of its terminal comparisons starting in the last block is admitted. These are exactly the two requirements of $T^{\mathrm{fin}}_\ell(\zeta'_{h-1}, X_{h-1})$. Consider a comparison (s', e') of v. If $s' = 0$, its relations are false by definition. Otherwise, $1 \le s' < e' \le h$. If Λ_ℓ held at its labels, it would supply positions p in $X_{s'-1}$ and p' in $X_{e'-1}$ with $X_{s'-1}[p{:}] = X_{e'-1}[p'{:}]$. The cuts $s = a_{s'-1} + p$ and $e = a_{e'-1} + p'$ have this common suffix before their markers. By Lemma 7.3, (s, e) is a comparison of u; it is nonterminal even when $e' = h$, since $e \le b_{h-1} < |u|$. Its labels would satisfy Λ, a contradiction. This excludes Λ_ℓ both at nonterminal comparisons of v and as the first disjunct of $\Lambda^{\mathrm{fin}}_\ell$ at terminal ones. The second disjunct, possible only for $e' = h$, gives an admitted terminal comparison $(a_{s'-1} + p, |u|)$ of u for some position p, also a contradiction. Thus v is a hole.

Conversely, suppose v is a hole. Its target implies the final target of u and excludes every admitted terminal comparison starting in the last block. Let (s, e) be any other comparison of u. If $e < |u|$, Lemma 7.3 locates s, e in blocks $s' - 1, e' - 1$ with $s' < e'$, a common suffix after them, and a comparison (s', e') of v. Admission by Λ would therefore give Λ_ℓ at its labels. For $e' < h$ this contradicts the nonterminal hole condition of v; for $e' = h$ it contradicts the first disjunct of its terminal relation $\Lambda^{\mathrm{fin}}_\ell$. If $e = |u|$, its start is in an earlier block $s' - 1$ with $s' < h$. Then (s', h) is a terminal comparison of v, and admission by Λ^{fin} would give the second disjunct of $\Lambda^{\mathrm{fin}}_\ell$. Again this is impossible. All comparisons of u are therefore unadmitted, so u is a hole. Iterating this equivalence proves the hierarchy assertion; each derivative adds exactly one fresh state. □

8.2 Isolated Exchange and Bounded Dictionaries

Shortening (Section 8.3) compresses all distinct blocks of a level at once, and its length bound grows quickly with the number of these blocks. We therefore need a bound on the number of different blocks at each positive level, not just on their lengths. We obtain it by merging letters that have the same effect on the derived monitor. Such a merge must preserve the comparisons of the level word as well as its transitions, targets, and admissions. We therefore protect the marker and the letters at symbolic start positions, and merge only letters whose unequal comparison partners are all protected. The behavior of such a letter can then be recorded in finite tables, its exchange type, whose size does not depend on the dictionary. The other letters are counted through the boundary pairs defined below, and these counts propagate from one level to the next.

Fix a nonempty hole w of depth $\Delta \le d$, its hierarchy u_i, and its derived monitors $\mathcal{M}_i$. For a word v, let $\mathrm{alph}(v)$ be its set of letters. Write $\mathcal{A}_i = \mathrm{alph}(u_i)$ for the level-i *dictionary*, and $g_i = |\mathcal{A}_i|$. These are letter dictionaries, distinct from the state supports $\mathcal{S}_i$ of Section 7. A comparison at level i is a comparison of u_i in the sense of Definition 4.12. It is *left maximal* if $1 \le s < e \le |u_i|$ and its predecessor letters $u_i[s-1]$ and $u_i[e-1]$ differ, so that it cannot be extended one letter to the left. Comparisons that begin at cut zero are excluded: they have no predecessor letter, and their admissions involve the start label, which the exchange types below record separately. The *boundary pairs* of level i are the ordered letter pairs

$$\mathrm{BP}_i = \{(u_i[s-1], u_i[e-1]) : (s, e) \text{ is left maximal},\ e < |u_i|\},$$

counted once regardless of how many comparisons produce them; put $b_i = |\mathrm{BP}_i|$. A left-maximal comparison that ends at the final cut has the marker ℓ_i as its second predecessor letter, since u_i ends with ℓ_i. Its pair, an endpoint pair, is needed only in the proofs (Appendix C).

The positions of u_i with start cores form a prefix, and their number $s_i \le i$ is the *start count* of level i (Lemma 7.4). In the derived monitor, ordinary positions are read from the embedded copy of the original state set Q, and start positions from symbolic states. Let Z_i be the union of that copy of Q and the symbolic states used at start positions; then $|Z_i| \le N + s_i$. The *protected letters* of level i

are

$$\Pi_i = \{\ell_i\} \cup \mathrm{alph}(u_i[:s_i]), \qquad |\Pi_i| \le s_i + 1.$$

A letter $a \in \mathcal{A}_i \setminus \Pi_i$ is *isolated* if no boundary pair has a as one component and a letter outside Π_i as the other. All unequal comparison partners of an isolated letter are therefore protected, including its partner in terminal comparisons, the marker.

Example 8.4 (Protected and isolated letters). At level 1 of Example 7.2, the word $u_1 = \alpha_1\beta_1\gamma_1\alpha_1$ has cores $\star_1, 1, 4, 5$ and marker $\ell_1 = \alpha_1$. Its start count is $s_1 = 1$ and its start prefix is α_1, so $\Pi_1 = \{\alpha_1\}$. The only comparison of u_1 that ends before the final cut is $(0, 3)$, which begins at cut zero. Hence $\mathrm{BP}_1 = \varnothing$, and β_1 and γ_1 are isolated. Their only unequal comparison partner is the marker, through the terminal comparisons $(2, 4)$ and $(3, 4)$. Merging β_1 and γ_1 would therefore preserve the comparisons of u_1; it would also require equal exchange types, defined next, which depend on the monitor.

For an isolated letter a, its *exchange type relative to* Π_i records the following finite tables in $\mathcal{M}_i$:

- the transition $q \mapsto \delta_i(q, a)$ and target $T_i^{\mathrm{fin}}(q, a)$ for every ordinary state $q \in Q$;
- for each $R \in \{\Lambda_i, \Lambda_i^{\mathrm{fin}}\}$, the self table $R((q, a), (q', a))$ for $q, q' \in Q$;
- for each such R, both $R((q, a), (\zeta, \pi))$ and $R((\zeta, \pi), (q, a))$ for $q \in Q$, $\zeta \in Z_i$, $\pi \in \Pi_i$;
- the two initial-label rows $R(\triangleright, (q, a))$ for $q \in Q$.

At positive levels the last rows are false. Original cut-zero obligations are still recorded in the protected rows at symbolic entry states (rows $R((q, a), (\zeta, \pi))$ and their reverse orientations, with ζ symbolic and π in the start prefix). Ordinary states transition to ordinary states, so the transition table has N^N possibilities, rather than $(N + i)^N$.

Put

$$\begin{aligned} \mathsf{Ty}(N, d) &= N^N \, 2^{2N^2+4N(N+d)(d+1)+3N}, \\ \mathsf{Al}(N, d) &= d + 1 + \mathsf{Ty}(N, d), \qquad \mathsf{Dict}(N, d) = 2 + \mathsf{Al}(N, d)3^d. \end{aligned} \tag{43}$$

Here Ty bounds the number of exchange types, Al adds the protected letters, and Dict will bound every positive dictionary. Shortening a tuple of h blocks produces blocks shorter than $col(M, h)$, the column count of (47) for a monitor with M states, which for fixed M is exponential in h^2. Bounding every positive dictionary by $\mathsf{Dict}(N, d)$ in advance makes each of at most d expansion steps multiply lengths by at most the fixed factor $K = col(N + d + 1, \mathsf{Dict}(N, d))$. The exchange argument achieves such a bound by merging only isolated letters: their types need rows only against the at most $d + 1$ protected letters, so $\mathsf{Ty}(N, d)$ is independent of dictionary sizes. Extending this rule to non-isolated letters would require protecting their unequal comparison partners as well; their number would enter the type-count exponent and could compound from level to level.

Lemma 8.5 (Exchange types). *At every positive level $i < \Delta \le d$, there are at most $\mathsf{Ty}(N, d)$ exchange types relative to Π_i.*

Lemma 8.6 (Isolated exchange). *Let $1 \le i < \Delta$, and let distinct isolated letters a, a' of u_i have the same exchange type. Replace every a by a' at that level and expand using the markers of the original hierarchy below level i. The result is a hole of the same canonical depth, with every start count unchanged and no dictionary larger. Its level-i dictionary is strictly smaller. Choosing the direction toward the letter with shorter level-zero expansion also makes the word no longer.*

Call a hole *dictionary-normal* if, at every $1 \le i < \Delta$, distinct isolated letters have distinct exchange types, computed in that hole's own hierarchy.

Lemma 8.7 (Dictionary normal form). *Every nonempty hole has a dictionary-normal hole of the same depth, no greater length, the same start counts, and no larger dictionary at any level.*

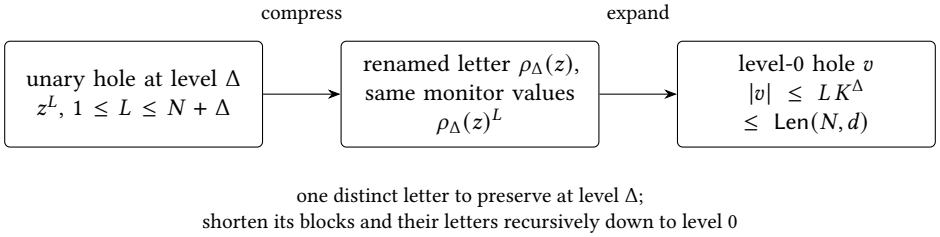


Fig. 10. Unary top level and compression. First shorten the unary power by deleting repeated-state loops, giving $L \le N + \Delta$. Compression then renames its single letter by the injective renaming ρ_Δ of Lemma C.4, under which the new level-Δ monitor agrees with the original one: it has the same transition, target, and admission values on the renamed letter. This preserves the run, target, and admissions of the power. With the dictionary-normal hierarchy unchanged and $K = col(N + d + 1, \mathsf{Dict}(N, d))$, supported block compression bounds that letter's expansion by K^Δ, giving $|v| \le LK^\Delta \le \mathsf{Len}(N, d)$. At $\Delta = 0$ the expansion is the identity.

Lemma 8.8 (Dictionary counting and propagation). *For a dictionary-normal hole of depth $\Delta \le d$, and $1 \le i < \Delta$, $g_i \le \mathsf{Al}(N, d) + 2b_i$. For every nonempty word and every level i,*

$$b_i \le b_{i+1} + g_{i+1} - 1. \tag{44}$$

Lemma 8.9 (Bounded dictionaries). *For a dictionary-normal hole of depth $\Delta \le d$, at $1 \le i < \Delta$,*

$$2b_i \le (\mathsf{Al}(N, d) - 1)(3^{\Delta-1-i} - 1), \qquad g_i \le 1 + (\mathsf{Al}(N, d) - 1)3^{\Delta-1-i}. \tag{45}$$

Consequently every nonempty hole of depth at most d has a hole of the same depth and no greater length with $g_i \le \mathsf{Dict}(N, d)$ for every $1 \le i \le \Delta$. No bound on g_0 is required.

Full proofs, including preservation of start counts under the induced level maps, appear in Appendix C. The counting idea is simple: dictionary normality bounds isolated letters by their types, while each remaining unprotected letter is a component of a boundary pair. This adds at most $2b_i$ letters. Those letters need not enter the protected set inside the type-count exponent: an isolated letter never compares with an unprotected unequal partner. Propagation therefore gives a linear recurrence with factor three, while the type bound remains fixed.

8.3 Shortening

Fix the dictionary-normal hole and its dictionaries $\mathcal{A}_j$. Throughout shortening, the original hierarchy means the hierarchy of this hole, which stays unchanged. Put $g = \mathsf{Dict}(N, d)$ and $K = col(N + d + 1, g)$, using the column count of (47). We shorten top-down. First the top level is shortened; then the blocks that form its letters; then the blocks that form the letters of those blocks; and so on down to level 0.

The Top Level. At the first unary level $\Delta \le d$, write $u_\Delta = z^L$. Every pair of its cuts is a comparison, and its states follow the orbit of $\delta(\cdot, z)$. Deleting the letters between two nonfinal cuts with equal states creates no new kind of label pair, so a unary hole of $\mathcal{M}_\Delta$ can be shortened to at most $N + \Delta$ letters, the number of states (details in Appendix C).

The Block Levels. At level $j + 1$, select distinct original blocks $X_1, \dots, X_h$ from $\mathcal{A}_{j+1}$. They are words over $\mathcal{A}_j$ avoiding the original marker ℓ_j. The *summary* of the tuple $(X_1, \dots, X_h)$ with respect to $\mathcal{M}_j$ and ℓ_j records:

- for each block and each state $\zeta \in Q \sqcup \{\star\}$, the exit state of its shifted block read from ζ;

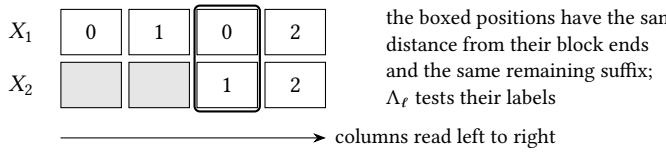


Fig. 11. The column encoding of a tuple of blocks, here $X_1 = 0102$ and $X_2 = 12$. The blocks are aligned at their right ends and padded on the left (shaded cells). A deterministic automaton reads the columns from left to right and maintains the summary of the tuple: the exit states of the shifted blocks, their tests against the final label, the equalities between blocks, and the bits of Λ_ℓ. Deleting the columns between two equal summaries shortens all blocks at once and preserves the summary (Appendix C).

- for each block, each such ζ, and each state $\zeta'' \in Q$, whether some label of the shifted block read from ζ is related by Λ^{fin} to the final label (ζ'', ℓ_j);
- which blocks are equal;
- for each ordered pair of blocks and each pair of states ζ, ζ', whether Λ_{ℓ_j} relates the two labels.

The summary determines every value of $\mathcal{M}_{j+1}$ on the letters $X_1, \ldots, X_h$, together with their equalities. Storing all equality bits preserves disequality as well: distinct old letters get distinct replacements, so the replacement is injective. It can be computed by a deterministic finite automaton reading the tuple as a single word of *columns*: align the blocks at their right ends, pad them on the left, and read the h-tuples of optional letters from left to right (Figure 11). Right alignment is what makes this possible: Λ_{ℓ_j} compares positions at equal distance from the ends of two blocks with equal letters after them, so appending a column either relates the two new last positions, or extends the common suffix of a pair related earlier by an equal letter. A column word is *valid* if each of its tracks consists of absent entries followed by letters, as padding on the left requires. If the column word has at least as many columns as there are summary values, two cuts have the same summary. Deleting the columns between them preserves the summary. It also preserves validity without any extra state in the automaton: deleting columns keeps every track of the form "absent entries, then letters", and each column itself records which tracks are present. Repeating this, we obtain blocks $\widetilde{X}_1, \ldots, \widetilde{X}_h$ of length less than the number $col(N + j, h)$ of summary values, with the same summary. Substituting $\widetilde{X}_i$ for X_i yields a word that is again a hole of $\mathcal{M}_{j+1}$.

The recursion keeps a set $\mathcal{E}_{j+1} \subseteq \mathcal{A}_{j+1}$ of distinct original letters, its *letter support*. After shortening its blocks to subsequences $\tau(X)$, form the lower letter support

$$\mathcal{E}_j = \{\ell_j\} \cup \bigcup_{X \in \mathcal{E}_{j+1}} \mathrm{alph}(\tau(X)) \subseteq \mathcal{A}_j. \tag{46}$$

The inclusion holds because every letter of an original upper block belongs to the original lower dictionary, and shortening only deletes letters. Thus each column shortening involves at most g blocks (tracks). At level zero no column shortening is needed, so its support need not have size at most g.

Compression proceeds top-down and renames letters as it goes. Shortening the level-$(j+1)$ blocks changes which level-j letters occur; those letters are shortened in turn, so the level-j marker and every lower derived monitor change as well. We therefore never compare the compressed word with the original hierarchy directly. Instead we keep, at each level j, an injective renaming ρ_j of the letter support $\mathcal{E}_j$ under which the new derived monitor *agrees* with the original one: it has the same transitions, targets, and both admission relations on the renamed letters, at every state, including the symbolic states and the start label (55). Agreement up to an injective renaming suffices because a monitor sees letters only through these values and through equality of letters,

and an injective renaming preserves both equality and inequality. Hence agreement preserves comparisons, runs, and holes. Since the support contains the marker, injectivity also preserves marker avoidance. Lemma C.4 composes the agreement given by the column summaries with agreement at the level below. Each level multiplies expansion lengths by at most K, so a renamed level-j letter expands to at most K^j letters. The original hierarchy and dictionaries stay unchanged throughout this induction; intermediate compressed words need not be dictionary-normal.

The Bound. For a lettered monitor with N states and a tuple of h blocks, put

$$col(N,h) = N^{(N+1)h} \cdot 2^{N(N+1)h} \cdot 2^{h^2} \cdot 2^{(N+1)^2h^2}. \tag{47}$$

This product counts all assignments to the summary fields and therefore bounds the number of reachable summary states.

Factor	Summary field
$N^{(N+1)h}$	Exit states: h blocks, $N+1$ entry states
$2^{N(N+1)h}$	Terminal-comparison bits: also N final-marker states
2^{h^2}	Equality bits for ordered pairs of blocks
$2^{(N+1)^2h^2}$	Derived-admission bits for block and entry-state pairs

Each shortened block has fewer than $col(N+j,g) \le K$ letters, so with its closing marker it has at most K letters. Expanding down one level therefore multiplies lengths by at most K. The unary top has at most $N+\Delta$ letters, and there are $\Delta \le d$ expansions. Define the single length bound

$$\mathsf{Len}(N,d) = (N+d+1)\, col\big(N+d+1, \mathsf{Dict}(N,d)\big)^d. \tag{48}$$

The resulting nonempty hole has depth at most d and length at most $(N+\Delta)K^\Delta \le \mathsf{Len}(N,d)$, proving Theorem 8.1. The alphabet size does not enter the bound. Section 10 gives a closed-form estimate and distinguishes hole length from exhaustive enumeration cost.

9 Deciding Entailment in the Three Domains

This section assembles the three bounds into decision procedures: a finite search for holes, a procedure for arbitrary trees built on it (Figure 12), and transfers to regular and finite trees.

Theorem 9.1 (Finite search for holes). *For an N-state reader, over any finite alphabet, that satisfies the rejected-path property, put*

$$B = 4N-2, \qquad d = 2N(B+1)-1, \qquad H = \mathsf{Len}(N,d). \tag{49}$$

A hole exists iff a hole of length at most H exists, where Len *is given by* (48)*. Moreover, $H \le 2^{2^{1280(N+1)^5}}$.*

Proof. By Theorem 6.1, choose a hole with tail at most B. If it is empty, it already meets the bound. Otherwise, Theorem 7.7 bounds its depth by d in the original N-state reader. Applying Theorem 8.1 gives a hole of length at most H. For an empty alphabet only ε exists, so no nonempty hierarchy is required. The reverse implication is immediate. The closed-form estimate is Lemma C.5. Each candidate can be checked by computing its run and examining the finitely many pairs $s < e \le |w|$. □

Theorem 9.2 (Correctness). *The procedure of Figure 12 decides arbitrary-tree entailment.*

Proof. By Lemma 4.5, (24), and Theorem 4.3, step 1 returns yes exactly when no solution exists. Otherwise, (11) and Theorem 5.4 give

$$\phi \models_c x \le y \quad \Longleftrightarrow \quad \text{neither label reader has a hole.}$$

Input: arity n, variance c, a flat system ϕ on k variables, and query $x \le y$.
Output: whether $\phi \models_c x \le y$.

(1) Form $\psi = \phi^{\pm}$, compute its closure, and test for a label clash. If a clash exists, return *yes*.
(2) Instantiate the reader of Section 5.2 on ψ, (x^+, y^+) and on $\psi^{\flat}$, (y^+, x^+). Use $N = 2 \cdot 4^{2k}$ and compute B, d, H from (49).
(3) Enumerate all words in $\mathbb{D}^*$ of length at most H, including ε. If either reader has a hole, return *no*; otherwise return *yes*. (Section 10 replaces this enumeration by the summary search of Proposition 10.2.)

For regular trees, use the same answer. For finite trees, first test finite satisfiability; if it fails, return *yes*, otherwise use the same answer.

Fig. 12. The decision procedure for flat constraints and a variable query.

Both readers satisfy the rejected-path property. By Theorem 9.1, the bounded search finds a hole whenever one exists. Thus each answer is correct, and the finite enumeration terminates. □

9.1 Regular and Finite Transfers

Finite trees need one test beyond the label clash. A *cycle clash* of a covariant system χ is a nonempty word η together with variables a, b such that

$$\mathrm{Lo}(\eta, a, a), \qquad D_\chi(a, b), \qquad \mathrm{Up}(\eta, b, b). \tag{50}$$

In every solution σ they give $\mathrm{emb}_\perp(\eta, \sigma(a)) \le \sigma(a) \le \sigma(b) \le \mathrm{emb}_\top(\eta, \sigma(b))$ by (16). Iterating the two self-bounds forces constructor paths along every power of η, so a cycle clash excludes finite solutions.

Proposition 9.3 (Transfer). *For every arity and variance,*

$$\phi \models_{\mathrm{reg},c} x \le y \iff \phi \models_c x \le y, \tag{51}$$
$$\phi \models_{\mathrm{fin},c} x \le y \iff \neg\, \mathrm{Sat}_{\mathrm{fin},c}(\phi) \ \vee\ \phi \models_c x \le y, \tag{52}$$
$$\mathrm{Sat}_{\mathrm{reg},c}(\phi) \iff \mathrm{Sat}_c(\phi). \tag{53}$$

Finite satisfiability is decidable.

Proof. The least shape is the unfolding of a finite graph of pairs of upper and lower supporting-variable sets. Selection and decoding are realized by graph products with polarity bits. Applying these operations to the same four-spine extension as in Theorem 4.9 gives a regular countermodel whenever an arbitrary countermodel exists; the reverse implication is immediate. The same operations applied to ψ itself, without spines, give a regular solution whenever a solution exists, which proves (53). The full graph construction and its equality with the pruned least shape are proved in Theorems D.1 and D.2.

For finite satisfiability, a covariant system must have neither a label clash nor a cycle clash (50). These conditions are sufficient by Theorem D.3. Without a cycle clash the least shape has depth at most $k_\chi^2 + 1$ for k_χ covariant variables; a longer constructor path repeats a pair of supporting variables and yields a cycle clash. A finite graph on such pairs decides the cycle test. Selection preserves the finite domain, and decoding gives the finite signed correspondence of Lemma D.4.

Finally, suppose the base has a finite solution and an arbitrary countermodel. Its unsafe word has a clash-free four-spine extension. Sources and sinks are separated. Assign old variables rank zero, new spine variables their cut indices, and fillers rank $|w| + 1$; ranks increase along new constructor edges, so every cycle clash would reflect to the base. The finite solution is used only to exclude cycle clashes in the base. The extension preserves that exclusion, and its least shape can therefore be

reconstructed independently with bounded depth. Its selected fixed solution restricts and decodes to a finite countermodel with the same unsafe word. This gives the finite entailment equivalence, including the stated vacuity case; all rank and depth details are proved in Theorem D.5. □

The finite vacuity case is necessary: at positive arity the constraint $f(x,\dots,x) \le x \le f(x,\dots,x)$ forces the infinite constructor tree. It has no finite solution. For $n = 0$, the reader search examines only ε, and the terminal child test still checks for a constructor bound. Together with Theorem 9.2, the transfer proves Theorem 1.1.

The effective graph-size and finite-depth bounds are collected in Appendix D.4.

The enumeration of Figure 12 establishes decidability. Section 10 implements its hole search differently: the summary search (Proposition 10.2) stores one level of the block hierarchy instead of a word and yields the space bound (Theorem 10.3).

10 Complexity

This section bounds the cost of deciding entailment. The input lists the arity n, the variance table $c(0),\dots,c(n-1)$, the variable set V, the query, and the literals of ϕ, each constructor literal listing all n child variables. Write $\|\phi\|$ for its size; thus $n \le \|\phi\|$ and $k = |V| \le \|\phi\|$. The readers of Section 5.2 have $N = 2\cdot 4^{2k} = 2^{4k+1}$ states. The constants hidden in O-notation below are absolute: they depend neither on n, nor on c, nor on the domain. We write 2-EXPSPACE for the problems decidable in deterministic space $2^{2^{p(\|\phi\|)}}$, and 3-EXPTIME for those decidable in deterministic time $2^{2^{2^{p(\|\phi\|)}}}$, for polynomials p. Deterministic classes, such as these and PSPACE, are closed under complement, so their bounds hold for entailment and non-entailment alike.

This section concentrates on constructors of arity $n \ge 2$. Section 10.1 proves an upper bound that holds for every arity: a search that stores one level of the block hierarchy at a time places entailment in 2-EXPSPACE and in 3-EXPTIME. For $n \le 1$ stronger bounds hold, PSPACE at arity one and polynomial time at arity zero (Appendix F). Section 10.2 proves lower bounds for a covariant constructor of each fixed arity $n \ge 2$: shortest holes can be exponentially long in k, and entailment is PSPACE-hard. All these bounds hold in each of the three domains.

10.1 Upper Bounds

Hole Length. The constants of (49) are $B = 4N - 2$, $d = 8N^2 - 2N - 1$, and $H = \mathsf{Len}(N, d)$. By Theorem 9.1 and Lemma C.5,

$$H \le 2^{2^{1280(N+1)^5}},$$

a triply exponential bound $2^{2^{2^{O(k)}}}$ in k. When ϕ is satisfiable in the domain considered and the entailment fails, a hole of length at most H of one of the two label readers certifies the failure (Theorem 5.4 and Proposition 9.3). It is an unsafe word, and it can be checked in time polynomial in its length and in $\|\phi\|$. The bound concerns the length of an unsafe word, not the node count of a fully expanded countermodel. Enumerating all words up to length H, as Figure 12 does, examines $\sum_{L\le H} n^L$ words per reader, four exponentials in k for fixed $n \ge 2$. The summary search below decides the same question without storing a word.

Polynomial-Time Tests. Every test of the reduction other than the search for holes is cheap.

Lemma 10.1 (Polynomial-time tests). *From a flat system ϕ with k variables and the query, the following are computable in time polynomial in $\|\phi\|$: the signed system $\psi = \phi^{\pm}$, its order dual, and their closures; whether either has a label clash or a cycle clash; and, for either label reader and states given as bit vectors, the initial state, the successor of a state under a letter, whether a state is rejected,*

Input: a lettered monitor $\mathcal{M}_0$ with N states over a finite alphabet Γ, given by its tables, and a depth d.
Stored: a level $j \le d$ and a current monitor $\mathcal{W}$ with $N + j$ states over a current alphabet $\Gamma_{\mathcal{W}}$; initially $j = 0$, $\mathcal{W} = \mathcal{M}_0$, and $\Gamma_{\mathcal{W}} = \Gamma$.
Repeat one of two nondeterministic moves:

(1) *Accept.* Guess a letter $z \in \Gamma_{\mathcal{W}}$ and a length $1 \le m' \le N + j$. Accept if $z^{m'}$ is a hole of $\mathcal{W}$.

(2) *Derive*, if $j < d$. Guess a marker $\ell \in \Gamma_{\mathcal{W}}$ and a number $1 \le h \le \mathrm{Dict}(N, d)$. Guess a valid column word with h tracks over $\Gamma_{\mathcal{W}} \setminus \{\ell\}$ and fewer than $col(N + j, h)$ columns, one column at a time. After each column, update the summary Θ of the tracks with respect to $\mathcal{W}$ and ℓ, and discard the column. Stop without accepting unless Θ declares the h blocks pairwise distinct. Then replace $\mathcal{W}$ by the monitor $\mathcal{W}[\ell, \Theta]$ over $\Gamma_{\mathcal{W}} = [h]$ that the summary determines, and j by $j + 1$.

Fig. 13. The summary search of Proposition 10.2. Here $[h] = \{1, \ldots, h\}$, and $\mathcal{W}[\ell, \Theta]$ has one new state (Lemma E.1). The track flags that keep the column word valid, the order of the summary updates, and the space accounting are in Appendix E.

and whether a pair of states is admitted. In particular, $\mathrm{Sat}_{\mathcal{D},c}(\phi)$ *is decidable in polynomial time for each of the three domains* $\mathcal{D}$.

The closure has at most $(2k)^2$ pairs and is computed by saturation. Every reader operation is a set operation on the $2k$ signed variables, computed from the closure. The clash tests decide satisfiability by (24), Theorem 4.3, (53), and Lemma D.4 and Theorem D.3. The proof is in Appendix E.

Searching Without Storing the Word. Theorem 9.1 bounds the length of a shortest hole, but a hole of that length takes triply exponential space to write down. The search below stores much less. It rests on three observations about Section 8. First, a lettered monitor with N states and a nonempty hole of depth at most d has such a hole whose hierarchy uses at most $\mathrm{Dict}(N, d)$ distinct letters at each positive level (Lemma 8.9). The h blocks of a level, over a monitor with $N + j$ states, can be shortened together below $col(N + j, h)$ letters without changing their summary, and a unary hole of such a monitor can be shortened to at most $N + j$ letters. Second, compression cuts a given hole top-down, but the derived monitors are built bottom-up: $\mathcal{M}_{j+1}$ is the derivative of $\mathcal{M}_j$ at the marker ℓ_j. Restricted to the blocks that are the letters of level $j+1$, and up to their names, it depends only on the summary of these blocks and on the targets of $\mathcal{M}_j$ at the marker, not on the blocks themselves (Lemma E.1). A search can therefore forget the blocks and call them $1, \ldots, h$. Third, the summary is computed by an automaton that reads the blocks column by column (Figure 11), so the columns can be guessed and discarded one at a time.

The search thus keeps only the tables of a current monitor over the abstract letters $1, \ldots, h$. It guesses the next level's blocks column by column, keeps their running summary, and accepts when a short unary word is a hole of the current monitor (Figures 13 and 14). The stored tables are doubly exponential in k, although the hole they describe may be triply exponentially long.

Proposition 10.2 (Summary search). *Let* $\mathcal{M}_0$ *be a lettered monitor with* $N \ge 1$ *states over a finite alphabet* Γ*, given by its tables, and let* $d \ge 0$*. Whether* $\mathcal{M}_0$ *has a nonempty hole of depth at most* d *is decidable in nondeterministic space*

$$O\big((N + d + 1)^2 \, (\mathrm{Dict}(N, d) + |\Gamma|)^2\big),$$

which is polynomial in $N + d + 1$*, the dictionary bound* $\mathrm{Dict}(N, d)$ *of* (43)*, and* $|\Gamma|$*. Hence it is decidable in deterministic space* $2^{O((N+d+1)^3)} \cdot (1 + |\Gamma|)^4$.

The search of Figure 13 has two moves. An accept move tests a bounded unary hole of the current monitor. A derive move chooses a marker and streams a bounded tuple of blocks that avoid it. The search maintains the following invariant. At each level $j \ge 1$, each current letter names a

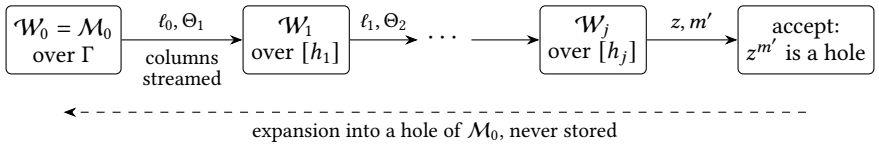


Fig. 14. The summary search builds the derived monitors bottom-up. A derive step chooses a marker ℓ_j of the current monitor $\mathcal{W}_j$, streams the columns of h_{j+1} distinct blocks that avoid it, keeps only their summary Θ_{j+1}, and moves to $\mathcal{W}_{j+1} = \mathcal{W}_j[\ell_j, \Theta_{j+1}]$ over the abstract letters $[h_{j+1}]$. Expanding the abstract letters level by level would turn an accepted unary word into a hole of $\mathcal{M}_0$ (dashed arrow). Compression in Section 8 instead starts from a given hole and shortens its hierarchy top-down.

distinct block over the letters of level $j-1$. The stored tables give the transitions, final targets, and admissions of the corresponding derived monitor on these letters. A derive move generates the blocks one column at a time, checks from the equality bits of the summary that they are distinct, and then retains only their names $1, \dots, h$ and the tables of $\mathcal{W}[\ell, \Theta]$. Higher levels use only this information.

An accepting computation could be expanded backward into a hole of $\mathcal{M}_0$: replace each abstract letter by the block it names, level by level, and close each block with its marker. Agreement (Lemma C.3) and desubstitution (Lemma 8.3) make each expansion a hole of the monitor one level below, which gives soundness; the search itself never performs or stores this expansion. Completeness follows from the dictionary bound (Lemma 8.9), column shortening, and unary shortening (Appendix C): the markers and the shortened blocks of a suitable hole provide the guesses. The largest stored items are the relations of the current monitor, the derived-admission bits of one summary, and the column counter, each of $O((N+d+1)^2(\mathrm{Dict}(N,d)+|\Gamma|)^2)$ bits. The deterministic bound then follows from Savitch's theorem [Savitch 1970]. The full proof is in Appendix E.

Theorem 10.3 (Upper bounds). *For every arity n, every variance c, and each of the three tree domains, entailment $\phi \models_{\mathcal{D},c} x \le y$ of a flat system with k variables is decidable in deterministic space $2^{2^{O(k)}} \cdot \mathrm{poly}(\|\phi\|)$. Hence entailment and non-entailment are in 2-EXPSPACE and in 3-EXPTIME in each domain, also for the general input syntax of Proposition 2.2.*

Proof sketch. By Lemma 10.1, decide in polynomial time whether ϕ is satisfiable in $\mathcal{D}$; if it is not, the entailment holds. Otherwise, by Proposition 9.3, (11), and Theorem 5.4, the entailment holds iff neither label reader has a hole. The empty word is a hole iff the initial state is rejected. By Theorems 6.1 and 7.7, a label reader has a hole iff the empty word is a hole or the reader has a nonempty hole of depth at most $d = 8N^2 - 2N - 1$. Its nonempty holes are those of its lettered monitor, whose tables are computed entry by entry by Lemma 10.1. Proposition 10.2, with $|\Gamma| = n$ and $N + d + 1 = 8N^2 - N \le 2^{8k+5}$, decides this in space $2^{2^{O(k)}}(1+n)^4$. Since $k \le \|\phi\|$, the whole procedure runs in space $2^{2^{O(k)}} \mathrm{poly}(\|\phi\|)$, hence in time $2^{2^{2^{O(\|\phi\|)}}}$. At positive arity, Proposition 2.2 reduces the general syntax in polynomial time to a flat system with $O(\|\phi\|)$ variables; at arity zero, Proposition F.3 in Appendix F applies. Appendix E gives the full accounting. □

Comparison with the Length Bound. Guessing a hole of length at most H and checking it places non-entailment in 3-NEXPTIME and entailment in co-3-NEXPTIME, hence in 3-EXPSPACE; these are the bounds that the length bound alone provides. Theorem 10.3 implies them, since

$$\text{2-EXPSPACE} \subseteq \text{3-EXPTIME} \subseteq \text{3-NEXPTIME} \cap \text{co-3-NEXPTIME},$$

and it lowers the space bound by one exponential; by the space hierarchy theorem [Stearns et al. 1965, Theorem 6], 2-EXPSPACE is a proper subclass of 3-EXPSPACE. The bound H on the hole length is unchanged. The saving comes from representing one level of the hierarchy by the tables of its derived monitor instead of storing a hole. For fixed $n \geq 2$:

Procedure	Stored object	Resulting bound
Enumeration (Figure 12)	a word of length at most H	time: four exponentials in k
Guessing a hole	a hole of length at most H	3-NEXPTIME, hence 3-EXPSPACE
Summary search (Proposition 10.2)	tables of one level	2-EXPSPACE and 3-EXPTIME

The search keeps tables of doubly exponential size in k, whereas the only known bound on the hole length is triply exponential.

10.2 Lower Bounds

We prove two lower bounds for a covariant constructor of arity $n \geq 2$: holes can be forced to be exponentially long in k, and entailment is PSPACE-hard, in each of the three domains. All systems in this subsection are covariant, so H_0 is the identity in (10). A word w is then left unsafe for (ϕ, x, y) iff some solution σ has $\widehat{\sigma(x)}(w) = \top \neq \widehat{\sigma(y)}(w)$, and right unsafe iff some solution has $\widehat{\sigma(y)}(w) = \bot \neq \widehat{\sigma(x)}(w)$. Proofs omitted here are given in Appendix G.

Overview. Both constructions simulate a nondeterministic automaton over $\{0,1\}$ by constructor literals along binary paths. For long holes, lower literals run an automaton on y: a transition $q \xrightarrow{a} q'$ places the a-child of Y_q above $Y_{q'}$, so Y_q has no bottom prefix along any word that the automaton accepts from q. All states of the automaton are final, so its language is prefix closed, and the remaining words form a union of sets $u\{0,1\}^*$. We use an automaton that accepts every word except the extensions of one exponentially long word, the record of a binary counter. Separate upper literals keep x top free, with constructor positions in $\{0,1\}^*$, so every unsafe word is right unsafe and extends the counter record. Truncating the trees of the automaton states, with top leaves at a depth beyond a chosen word, gives finite solutions, so these words are unsafe in all three domains. For hardness the roles of the two kinds of literals are reversed: upper literals run a given automaton on x, and lower literals keep a second variable r at least a constructor along every proper prefix of a code word.

Constructor Positions. For a tree t, let $\mathrm{Pos}(t) = \{w \in \mathrm{dom}(t) : t(w) = f\}$ be its set of *constructor positions*; it is prefix closed. A tree is *top free* if no node is labeled $\top$. Every prefix-closed set $P \subseteq \mathbb{D}^*$ is $\mathrm{Pos}(T_P)$ for exactly one top-free tree T_P: its domain is $\{\varepsilon\} \cup P \cup P\mathbb{D}$, its label is f on P, and $\bot$ elsewhere. For example, $T_\varnothing = \bot$, and $T_{\{0,1\}^*}$ is the full constructor tree F when $n = 2$. For a prefix-closed set P and $m \geq 1$, the *truncation* T_P^m is obtained from T_P by replacing the subtree at every word of P of length m by $\top$; it is a finite tree. For variables a_0, a_1, b we write $f(a_0, a_1, \bar{b})$ for $f(a_0, a_1, b, \ldots, b)$, with $n-2$ copies of b; for $n = 2$ it stands for $f(a_0, a_1)$.

Lemma 10.4 (Top-free trees). *Let $n \geq 2$.*

(1) For top-free trees t, u and every word w, $\widehat{t}(w) = f$ if $w \in \mathrm{Pos}(t)$ and $\widehat{t}(w) = \bot$ otherwise; and $t \leq u$ iff $\mathrm{Pos}(t) \subseteq \mathrm{Pos}(u)$.

(2) T_P is finite iff P is finite.

(3) Let σ satisfy $b = \bot$ and $g \leq f(g, g, \bar{b})$. Then $\sigma(g)$ is top free and $\mathrm{Pos}(\sigma(g)) \subseteq \{0,1\}^*$, and so is every tree $t \leq f(\sigma(g), \sigma(g), \bot, \ldots, \bot)$.

(4) Let P, P' be prefix closed, $\varepsilon \in P$, $a \in \mathbb{D}$, and $aP' \subseteq P$. Then $f(t_0, \ldots, t_{n-1}) \leq T_P^m$ for every $m \geq 1$, where $t_a = T_{P'}^m$ and $t_i = \bot$ for $i \neq a$.

Item (3) holds because a top leaf cannot lie below a constructor: by (9), a top leaf of $\sigma(g)$ at a word iw would force $i \in \{0,1\}$ and a top leaf at the shorter word w, and $\sigma(g)$ has no top leaf at ε. In item (4), the truncation at the same depth m on both sides is what makes the inequality hold: a top leaf of $T^m_{P'}$ at a word v of length m lies below the top leaf of T^m_P at the prefix of av of length m.

Long Holes. As noted in Section 10.1, a failed entailment on a satisfiable system is certified by a hole of one of the two label readers, and by Theorem 5.4 these holes are exactly the unsafe words. Such certificates can be forced to be exponentially long. The construction uses an automaton with $O(\beta)$ states for the binary words that do not begin with one word of length $2(\beta+1)2^\beta$, the record of a β-bit binary counter.

Fix $\beta \geq 1$. Over the *token alphabet* $\mathbb{T} = \{0, 1, \#, \$\}$, let $\langle j \rangle$ be the binary expansion of j with β bits, least significant bit first, and put

$$\mathsf{H}_\beta = \langle 0\rangle\#\langle 1\rangle\#\cdots\#\langle 2^\beta - 1\rangle\$, \qquad |\mathsf{H}_\beta| = (\beta+1)2^\beta.$$

The *token encoding* maps the letters of $\mathbb{T}$ to $0 \mapsto 00$, $1 \mapsto 01$, $\# \mapsto 10$, and $\$ \mapsto 11$. Let $\mathsf{W}_\beta \in \{0,1\}^*$ be the token encoding of H_β, of length $\lambda_\beta = 2(\beta+1)2^\beta$. For $\beta = 1$, $\mathsf{H}_1 = 0\#1\$$ and $\mathsf{W}_1 = 00100111$. In a nondeterministic automaton all of whose states are final, a word is accepted iff it has a run, that is, a path labeled by the word from an initial state.

Lemma 10.5 (Counter automaton). *For every $\beta \geq 1$ there is a nondeterministic automaton $\mathcal{B}_\beta$ over $\{0,1\}$ with $9\beta + 27$ states, all of them final, whose language is $\{0,1\}^* \setminus \mathsf{W}_\beta\{0,1\}^*$. It is computable in time polynomial in β.*

The automaton reads the encoded tokens. In parallel, its branches check the format of the blocks, that the first block is zero, that each block is the successor of the previous one, and that $ follows exactly the all-ones block. A branch that detects an error enters a state that loops on every letter, and on W_β itself every branch dies (Appendix G). For instance, for $\beta = 2$ the record is $\mathsf{H}_2 = 00\#10\#01\#11\$$. In a token word that begins with 00#10#11, the first bit of the third block is wrong: the successor of 10, that is, of 1, is 01. The branch spawned at the first bit of the second block predicts the bit 0 from that bit and the carry, counts $\beta + 1 = 3$ tokens, and reads 1 instead. It detects the error while storing only the predicted bit and a counter, not the history.

Let Q be the set of states of $\mathcal{B}_\beta$, $I \subseteq Q$ its initial states, and $q \xrightarrow{a} q'$ its transitions. Add a state q_{in} with $q_{\text{in}} \xrightarrow{a} q'$ iff $q \xrightarrow{a} q'$ for some $q \in I$, so that q_{in} accepts the language of $\mathcal{B}_\beta$. For $n \geq 2$, let $\Phi^{(n)}_\beta$ have the variables x, g, b, and Y_q for $q \in Q \cup \{q_{\text{in}}\}$, with $y = Y_{q_{\text{in}}}$, and the flat literals

$$b = \bot, \quad g \leq f(g,g,\bar{b}), \quad x \leq f(g,g,\bar{b}) \qquad \text{(confinement to binary paths)},$$

$$f(b,b,\bar{b}) \leq Y_q \qquad \text{(nonbottom roots)},$$

$$f(Y_{q'},b,\bar{b}) \leq Y_q \ \ (q \xrightarrow{0} q'), \quad f(b,Y_{q'},\bar{b}) \leq Y_q \ \ (q \xrightarrow{1} q') \qquad \text{(propagation along transitions)},$$

for all $q \in Q \cup \{q_{\text{in}}\}$. Thus y receives the lower literals of the initial states directly. Write $\Phi_\beta = \Phi^{(2)}_\beta$. For every n, $\Phi^{(n)}_\beta$ has $k = 9\beta + 31$ variables. The confining literals keep x top free, with constructor positions in $\{0,1\}^*$. The other two groups give each Y_q a constructor or a top prefix along every word that $\mathcal{B}_\beta$ accepts from q, so y has no bottom prefix along any binary word without the prefix W_β. The trees of y and of the Y_q may have top leaves, which permits finite solutions.

Proposition 10.6 (Exponentially long holes). *Let $n \geq 2$, let f be covariant of arity n, let $\beta \geq 1$, and let $\Phi = \Phi^{(n)}_\beta$, with the query $x \leq y$.*

(1) Φ has a finite solution, and $\Phi \models_{\mathcal{D}} x \leq y$ fails in each of the three domains $\mathcal{D}$.

(2) No word is left unsafe, and the right-unsafe words are exactly the words of $\mathrm{W}_\beta\{0,1\}^*$.
(3) The left label reader $(\mathcal{M}_\mathrm{l}, \mathsf{Adm}_\mathrm{l}, \mathsf{Rej}_\mathrm{l})$ *has no hole, and the holes of the right label reader* $(\mathcal{M}_\mathrm{r}, \mathsf{Adm}_\mathrm{r}, \mathsf{Rej}_\mathrm{r})$ *are exactly the words of* $\mathrm{W}_\beta\{0,1\}^*$. *In particular, every hole that the search of Figure 12 can find has length at least* $\lambda_\beta = 2(\beta+1)2^\beta$, *and* W_β *is a hole of this length.*

Hence, for every fixed arity $n \geq 2$ *and in each domain, shortest holes of systems with* k *variables can have length* $2^{\Omega(k)}$: *here* $\lambda_\beta \geq 2^{(k-22)/9}$.

Proof idea. For $n = 2$, let σ be a solution. By Lemma 10.4(3), $\sigma(x)$ is top free with $\mathrm{Pos}(\sigma(x)) \subseteq \{0,1\}^*$, so no word is left unsafe, and a right-unsafe word w has $w \in \mathrm{Pos}(\sigma(x))$ and $\widehat{\sigma(y)}(w) = \bot$ for some solution σ. Induction along runs of $\mathcal{B}_\beta$ shows that $\widehat{\sigma(y)}$ is not $\bot$ on the language of $\mathcal{B}_\beta$, so by Lemma 10.5 every right-unsafe word begins with W_β. Conversely, let $w \in \mathrm{W}_\beta\{0,1\}^*$ and $m = |w| + 1$. The truncations $Y_q = T^m_{L_q}$, with L_q the set of words accepted from q, together with $g = T_{\{0,1\}^{<m}}$, $b = \bot$, and $x = T_P$ for the set P of prefixes of w, form a finite solution with $w \in \mathrm{Pos}(x)$ and $\widehat{y}(w) = \bot$ (Lemma 10.4(4)). This gives (1) and (2), and (3) follows by Theorem 5.4. The case $n \geq 3$ reduces to $n = 2$ by an embedding of binary trees, described at the end of this subsection. The details are in Appendix G. □

Since $N = 2^{4k+1}$, the length λ_β is only polynomial in N, whereas the length bound H of Section 10.1 is triply exponential in k.

Hardness. Henglein and Rehof [1998, Theorems 1 and 2] proved PSPACE-hardness for a binary covariant constructor over arbitrary and over finite trees (Section 12). We give a reduction from universality of nondeterministic automata over $\{0,1\}$ that works in all three domains and for every fixed arity $n \geq 2$. An automaton $\mathcal{A} = (Q, q_0, F, \delta)$ has one initial state q_0, final states F, and transitions $\delta \subseteq Q \times \{0,1\} \times Q$; it is universal if it accepts every word. Universality of such automata is PSPACE-hard.[2]

The reduction runs an automaton on x by upper literals, and an accepting run forces the subtree of x at its end to be $\bot$; then x has no node below that word. A word at which x is to have a constructor while a second variable r is $\bot$ must therefore have no accepted prefix. We encode words by a prefix-free code, so that the code of a rejected word has no accepted code as a prefix. The variables r and r' track the position within an encoded letter (Figure 15). Their lower literals keep r at least a constructor at every proper prefix of a code, while at a completed code r may be $\bot$, where x can still have a constructor if the code is rejected.

For $w \in \{0,1\}^*$ let $\mathrm{code}(w) = e(w)1$, where e replaces each letter 0 by 00 and each 1 by 01; we call code the *prefix-free code*. On single bits, e agrees with the token encoding, but the two encodings serve different constructions. The code is prefix free: if $\mathrm{code}(w') \preceq \mathrm{code}(w)$ and $|w'| < |w|$, then position $2|w'|$ carries the final 1 of $\mathrm{code}(w')$ but the letter 0 in $\mathrm{code}(w)$. From $\mathcal{A}$ we obtain in linear time an automaton $\mathcal{A}'$, again with the initial state q_0, that accepts $\mathrm{code}(L(\mathcal{A}))$: replace each transition (q, a, q') by $q \xrightarrow{0} \mu \xrightarrow{a} q'$ through a new middle state μ, and add a transition labeled 1 from every final state to one new state q_f, the only final state of $\mathcal{A}'$.

[2] Krötzsch et al. [2016, Theorem 3] prove this for automata over $\{0,1\}$, even for partially ordered ones. Their automata may have several initial states; a fresh initial state that has the outgoing transitions of all of them, and is final iff one of them is, accepts the same language. For regular expressions over $\{0,1\}$ with union, concatenation, and star, the classical result is due to Stockmeyer and Meyer [1973, Section 3], who state that inequivalence is complete for nondeterministic linear space.

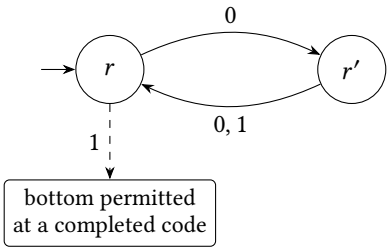


Fig. 15. The lower literals $f(r', b, \dots, b) \le r$ and $f(r, r, b, \dots, b) \le r'$ of $\chi(\mathcal{A}')$ as a diagram: an edge labeled a from v to v' means that the a-child of v lies above v'. Reading the encoding $e(w)$ of a word follows the solid edges, so the tree of r has a constructor or $\top$ at every proper prefix of a code. The final 1 of $\mathrm{code}(w)$ leaves the diagram (dashed): at a completed code the tree of r may have the label $\bot$, although r itself is never $\bot$.

For such an automaton $\mathcal{A}'$ with states Q', let $\chi(\mathcal{A}')$ have the variables z, b, r, r', and X_q for $q \in Q'$, write x for X_{q_0}, and take the flat literals

$$X_q \le f(X_{q'}, z, b, \dots, b) \;\; (q \xrightarrow{0} q'), \qquad X_q \le f(z, X_{q'}, b, \dots, b) \;\; (q \xrightarrow{1} q'), \qquad X_{q_f} = \bot,$$
$$b = \bot, \qquad f(r', b, \dots, b) \le r, \qquad f(r, r, b, \dots, b) \le r'.$$

Here b fills the arguments $2, \dots, n-1$ of the first two literals and the arguments $1, \dots, n-1$ of the first lower literal; for $n = 2$ the first two literals have no b. No literal bounds z, so z may be $\top$, and then, among the two binary children, an upper literal constrains only the child that the run follows; the added children are bounded by $b = \bot$. Along every run of $\mathcal{A}'$ from q_0 on a word u, the upper literals keep the subtree of x at u, if it exists, below the variable X_q of the reached state q; an accepting run therefore forces the subtree of x at its end, if it exists, to be $\bot$. The lower literals give r a constructor or $\top$ at every proper prefix of a code, and they allow r to be $\bot$ at a code. If $\mathcal{A}$ is universal, every proper prefix of a code lies on an accepting run, and $x \le r$ follows. If $\mathcal{A}$ rejects a word w, then a solution can give x a constructor at $\mathrm{code}(w)$ and r the label $\bot$ there.

Theorem 10.7 (PSPACE-hardness). *Let $n \ge 2$ be fixed and let f be covariant. For every automaton $\mathcal{A}$ over $\{0, 1\}$ and each domain $\mathcal{D}$,*

$$\chi(\mathcal{A}') \models_{\mathcal{D}} x \le r \quad \Longleftrightarrow \quad \mathcal{A} \text{ is universal.}$$

Consequently, in each of the three domains, entailment of flat systems over $\{\bot, f, \top\}$ is PSPACE-hard. So is the problem with the variance as part of the input, already on covariant instances.

Proof idea. Let P be the set of proper prefixes of codes, the words $e(w)$ and $e(w)0$. If $\mathcal{A}$ is universal, every word of P lies on an accepting run of $\mathcal{A}'$, so in every solution the subtree of x there, if present, lies below a constructor tree whose arguments $2, \dots, n-1$ are $\bot$; every code is accepted, so x is $\bot$ at codes. Along P, the lower literals give r only the labels f and $\top$. Every common position of x and r therefore compares a label at most f with one at least f, or has $x = \bot$. If $\mathcal{A}$ rejects w, no prefix of $\mathrm{code}(w)$ is accepted by $\mathcal{A}'$, and a path tree along $\mathrm{code}(w)$ together with finite approximants of r forms a finite countermodel, which lies in every domain. The system is satisfiable, has $|Q'| + 4$ variables, and is computed in polynomial time. The details are in Appendix G. □

For $n = 2$ and arbitrary or finite trees, Theorem 10.7 reproves, by a different reduction, the hardness theorems of Henglein and Rehof [1998, Theorems 1 and 2], which are stated for a binary covariant constructor over these two domains. It gives a uniform proof for all three domains and every fixed arity $n \ge 2$. Their theorems do not state the regular case, and we do not claim that it is

new. We claim no hardness for a fixed variance with a contravariant argument, nor for arity one (Appendix F).

Larger Arities. The hardness reduction works directly at every arity $n \geq 2$. The systems $\Phi_\beta^{(n)}$ for long holes are obtained from Φ_β by filling the new arguments of every constructor with the variable $b = \bot$. Such *bare padding* does not preserve lower bounds in general. The padded system also constrains trees that are not images of binary trees, and these may refute the entailment at short words that use the new letters.

Example 10.8 (Bare padding fails). For a binary covariant f_2, the system

$$\chi = \{x \leq f_2(y, b),\ f_2(z, b) \leq y,\ f_2(z, t) \leq z,\ b = \bot,\ t = \top\}$$

entails $x \leq y$. Indeed, every top leaf of x lies at a word $0v$ such that y has a top prefix along v. Then y also has one along $0v$: either $y = \top$, or the child 0 of y lies above z, which has a top prefix along every word that begins with 0^j1; and for $v \in 0^*$, the top prefix along v lies along $0v$. Moreover, y has bottom prefixes only along words that begin with 1, where x has one as well. Padding with a variable $o = \bot$ in the third argument, for $n = 3$, loses the entailment: the finite assignment $x = f(f(\bot, \bot, \top), \bot, \bot)$, $y = f(z, \bot, \top)$, $z = f(\top, \top, \bot)$, $b = o = \bot$, and $t = \top$ is a countermodel, since x has a top leaf at 02, where y has the bottom leaf of z.

Padding with $\bot$ does preserve solutions and unsafe words on binary paths: lifting a binary tree by adding $\bot$ children, and projecting a tree over $\mathbb{D}$ to its binary paths, map solutions to solutions in each domain and keep leaf extensions on $\{0, 1\}^*$ (Proposition G.1 in Appendix G). For $\Phi_\beta^{(n)}$, the confining literals $g \leq f(g, g, \bar{b})$ and $x \leq f(g, g, \bar{b})$ also exclude unsafe words that use an added child position, because they keep x top free with constructor positions in $\{0, 1\}^*$. Thus $\Phi_\beta^{(n)}$ and Φ_β have exactly the same unsafe words. The system of Example 10.8 has no confining literals, and the word 02 of its countermodel is left unsafe and uses the added letter 2. The embedding is therefore not a general reduction between arities, such as the equivalence of all arities at least two asserted by Niehren and Priesnitz [2003, Corollary 1] (Section 12.1). It transfers unsafe words only on binary paths, which suffices when confining literals exclude all other words.

Combined Bounds. Together with Theorem 10.3, entailment for a covariant constructor of each fixed arity $n \geq 2$ is PSPACE-hard and in 2-EXPSPACE in each domain. Section 14 discusses the open questions about its exact complexity.

11 Mechanization

The artifact, a development in Lean 4 [de Moura and Ullrich 2021] with Mathlib [The mathlib Community 2020], contains decision procedures for all three domains that search for holes in three ways: by bounded enumeration (Figure 12), by the summary search of Section 10.1, and, at arity one, by the single-word test of Appendix F. Their correctness is mechanized end to end for flat constraints. The development covers arbitrary arity and variance, signed constraints, satisfiability, the semantic reduction, the alphabet-generic hole bounds, and both domain transfers. The hole-bound modules include isolated exchange, induced level maps, normal forms, boundary-pair propagation, and compression within fixed original dictionaries. The artifact also contains a certifying solver whose answers are checked by proved checkers. Lean and Mathlib are both pinned to `v4.35.0`.

The trusted statement consists of the definitions of trees, the variance order, flat constraints, satisfaction, and entailment in each domain. A characterization module connects the path-based order to the greatest fixed point of the constructor rule and shows that the finite and regular representations denote the intended tree classes. The kernel checks the proof terms establishing

equivalence with finite search. The procedures are computable definitions. The proof dependencies and commands for checking them appear in Appendix H.2.

The final bounds are parameterized by the original reader-state count N, without a preliminary state refinement. Tail and depth use those states directly; compression uses derivative monitors with $N + j$ states at level j, as accounted for in (48). The paper and artifact have the same final bounds. Some intermediate statements and constants differ; Appendix H gives the public API, module layout, correspondence, and these differences. `ElementaryBound.lengthBound` defines $\mathrm{Len}(N, d)$; `ElementaryBound.exists_short_hole` proves the bounded-depth witness theorem. The back end uses `RejectedTail.holeLengthBound`, and `RejectedTail.holeLengthBound_le` proves the closed-form estimate $H \le 2^{2^{1280(N+1)^5}}$.

The module `Complexity` mechanizes the combinatorial content of Section 10 and Appendix F. The summary search is proved sound and complete, and entailment procedures built on it are proved correct in each domain. For the label readers, `Complexity.Summary.card_config_label_le` bounds the number of configurations so that one configuration fits in $2^{2^{O(k)}} \cdot \mathrm{poly}(n)$ bits, matching the space bound of Theorem 10.3; Appendix H.6 states the exact bound. Unlike the nondeterministic search of Proposition 10.2, which counts the columns of a derive move to bound its space, the mechanized search decides reachability of an accepting configuration in a finite graph and keeps no column counter. The configuration bound supports the space argument, but the executable search computes the reachable set by saturation and is not proved to run in that space. `Complexity.Unary.exists_hole_length_le` proves the bound $N - 1$ for every reader over a one-letter alphabet, and the unary procedures test only the word up to the first rejected cut. For every arity $n \ge 2$ and each domain, `Complexity.entails_hardnessSystem_iff` and its regular and finite versions prove the hardness reduction correct, with linearly many literals; the systems of Proposition 10.6 refute entailment, and `Complexity.expSystem_shortest_hole` gives their exact shortest hole length $2(\beta+1)2^{\beta}$. Not mechanized are the complexity classes themselves, that is, space-bounded machines, Savitch's theorem, and the space hierarchy theorem; the PSPACE-hardness of universality; the polynomial-time tests of Lemma 10.1; and the polynomial-time procedure at arity zero (Proposition F.3). Appendix H.6 gives the correspondence and the differences.

The artifact is included in the supplementary material. The NP03 counterexample is derived in full in Appendix J. A separate kernel-checked formalization agrees with that argument and is not part of this artifact.

12 Related Work

Table 3 summarizes selected results on subtype entailment.

Subtype Entailment and Constraint Simplification. Subtype constraints arise in type inference with subtyping [Aiken and Wimmers 1993; Eifrig et al. 1995; Fuh and Mishra 1990; Mitchell 1991; Palsberg et al. 1997], and the size of inferred constraint sets made simplification a central practical concern [Pottier 1996, 1998b, 2001; Trifonov and Smith 1996]. Sound but incomplete entailment tests based on syntactic closure are used in practice; Pottier [2001] discusses their limits. Satisfiability of subtype constraints is comparatively well understood [Frey 1997; Kozen et al. 1994; Niehren et al. 2005; Palsberg and O'Keefe 1995; Palsberg et al. 1997; Pratt and Tiuryn 1996; Simonet 2003; Tiuryn 1992]. Closure-based satisfiability characterizations are classical. Kozen et al. [1994] treat a language with $\top$ but without $\bot$; related characterizations appear in Palsberg and O'Keefe [1995]; Palsberg et al. [1997]. Here the additional ingredients are a common least shape under variable flip and polarity selection for variance. Subtyping between recursive types without variables is also well understood [Amadio and Cardelli 1993; Kozen et al. 1995]. For entailment, Henglein and Rehof [1997] proved coNP-completeness for structural subtyping over finite trees and Henglein and Rehof

Table 3. Selected results on subtype entailment. Earlier non-structural rows concern a covariant constructor; our decidability result allows arbitrary variance and arity. The complexity bounds for our results are proved in Section 10; Section 11 states which parts of them are mechanized.

Problem	Trees	Result	Reference
Structural entailment	finite	coNP-complete	Henglein and Rehof [1997]
Structural entailment	arbitrary	PSPACE-complete	Henglein and Rehof [1998]
Structural, first-order theory	finite	decidable	Kuncak and Rinard [2003]
NSSE, f unary	finite, regular, arbitrary	decidable	Niehren and Priesnitz [2003]
NSSE, f binary, no literals $x = \bot$, $x = \top$	finite, arbitrary	PSPACE-complete	Niehren and Priesnitz [1999]
NSSE, f binary	finite, arbitrary	PSPACE-hard	Henglein and Rehof [1998, Theorems 1–2]
NSSE, any arity and variance	finite, regular, arbitrary	decidable; in 2-EXPSPACE (stronger for $n \leq 1$, Appendix F)	this paper
NSSE, f covariant, any fixed arity $n \geq 2$	finite, regular, arbitrary	PSPACE-hard	this paper
Non-structural, first-order theory	finite, regular, arbitrary	undecidable	Su et al. [2002]

[1998] PSPACE-completeness over arbitrary trees. Their Theorems 1 and 2 establish non-structural PSPACE-hardness using a binary covariant constructor, over arbitrary and finite trees, respectively; this is the scope recorded in Table 3. Theorem 10.7 gives a different reduction, from universality of nondeterministic automata, with a uniform proof for all three domains and every fixed arity $n \geq 2$. Rehof [1997, 1998] studies minimal typings and the complexity of atomic and structural subtyping. Kuncak and Rinard [2003] show that the full first-order theory of structural subtyping of non-recursive types is decidable.

Non-Structural Subtype Entailment. Niehren and Priesnitz [1999] proved PSPACE-completeness for the fragment in which the constraints contain no literals $x = \bot$ and $x = \top$ (the constants may still occur in solutions), by a reduction to universality of finite automata; they introduced P-automata to explain why the nonregular sets of paths that arise in general can be ignored in this fragment. Niehren and Priesnitz [2001, 2003] introduced cap automata and cap set expressions, proved decidability of the unary case, and identified the role of word equations in the general problem. Their forward reduction from entailment to cap-automaton universality motivates our use of periodic suffix comparisons. Section 12.1 discusses the proof dependencies of their converse reduction and arity equivalence. Priesnitz's theses [Priesnitz 2000, 2004] develop the automata approach further. Su et al. [2002] proved that the first-order theory of non-structural subtyping is undecidable. This does not settle entailment, which is a restricted universal implication. They and Su [2002] also proposed a reduction of entailment to constrained tree automata on tuples of trees, whose emptiness problem is undecidable in general [Treinen 2000]; an extended version of the paper develops this reduction further [Su et al. 2006]. A preprint claimed membership in PSPACE for the finite and regular cases [Schubert 2021]; it was withdrawn by its author because of an error in a key lemma. The problem has been described as open in work on intersection types and on type inference [Dudenhefner et al. 2017; Jiang et al. 2025].

12.1 The Published Characterization

Proposition 16 of NP03 is false as stated: Appendix J gives a six-state complete restricted cap automaton that rejects 11, whereas the left automaton of its back-translated judgment accepts 11 under either finality convention for Table 3. The judgment is moreover a satisfiable entailment. Theorem 4 and the converse reduction underlying Theorem 1 rely on Proposition 16. Corollary 1 (equivalence of arities at least two and of tree classes) also relies on that converse, and its arity part on Proposition 17. Theorem 4, Theorem 1, and Corollary 1 themselves are not refuted; only their published proofs are incomplete. No defect in Proposition 17 is claimed here. The forward characterization (Theorem 2, with Propositions 8 and 11) is unaffected: it is all that NP03's unary decidability result and the earlier binary proof in Section 13.1 use. The proofs of Propositions 12 and 14 treat states ready by the bottom, top, or reflexivity rule as final. Table 3 must be read with that convention: read literally, it rejects the path 1 of NP03's unary example $z = \top \wedge f(z) \leq y \models^{?} x \leq y$ (Section 4), which NP03 shows to be both left safe and right safe. Appendix J gives the acceptance details. Our present proof uses neither the forward characterization nor any of these converse or alphabet reductions.

12.2 Other Connections

Set Constraints and Feature Trees. Entailment has been studied for neighboring constraint languages. Entailment of atomic set constraints is PSPACE-complete over infinite signatures [Niehren et al. 1999]. Ordering constraints over feature trees admit efficient entailment tests under an assumption of infinitely many features [Müller et al. 2000], while their full first-order theory is undecidable [Müller et al. 1998]. Componential set-based analysis uses entailment of set constraints for simplification [Flanagan and Felleisen 1999].

Type Inference Without General Entailment. Several modern approaches to type inference with subtyping avoid general entailment by changing the type language or the semantics. Algebraic subtyping [Dolan 2016; Dolan and Mycroft 2017; Parreaux 2020] interprets types in an algebraic model rather than as ground trees, and infers principal types by biunification on polarized types, treating subsumption between inferred types separately. MLstruct extends this approach to a Boolean algebra of structural types [Parreaux and Chau 2022], and Parreaux et al. [2024] retain subtyping constraints in polymorphic types. Semantic subtyping interprets types as sets of values and decides subtyping between types, including with polymorphism [Castagna and Xu 2011; Gesbert et al. 2015]. Orthologic type systems obtain efficient entailment for an open-world semantics [Guilloud and Kunčak 2025]. These approaches use different type languages or semantic models, so their decidability results do not directly decide the ground-tree entailment problem studied here.

Word Equations and Periodicity. Satisfiability of word equations is decidable [Makanin 1977] and in PSPACE [Plandowski 2004], but the positive $\forall\exists^3$ fragment is undecidable [Durnev 1995]. Niehren and Priesnitz [2003, Corollary 2] express NSSE in the positive $\forall\exists^*$ fragment with regular constraints. Existential word-equation algorithms do not directly decide this quantified formulation. Undecidability of the ambient fragment leaves open the restricted instances arising from NSSE, which our reader construction and compression argument decide. The periodicity arguments use the principle that sufficiently long agreement between two periods forces agreement between their residue classes [Fine and Wilf 1965]. Lemma 6.2 proves the special case it needs directly, and in Lemma 6.7, equality over one full period propagates along the periodic tail. These are instances of classical periodic-word reasoning [Lothaire 2002], without a general word-equation algorithm.

Back-References. A cap repeats a root word that is chosen during the run, which resembles a back-reference in an extended regular expression. For extended regular expressions, universality

is undecidable even with a single variable [Freydenberger 2011]. Cap set expressions are a much weaker mechanism: the only repetition is the prefix closure of the powers of one word, and that word must be followed to the end of the path. Even so, universality of unrestricted cap automata was left open by Niehren and Priesnitz [2003, Section 10]; our decision procedure applies to readers satisfying the rejected-path property and does not settle that question.

Desubstitution. Cutting a sequence at the occurrences of a word and recoding the blocks between them is the idea behind *return words* and *derived sequences* in symbolic dynamics [Durand 1998], and the question whether such decompositions are unique is the subject of recognizability theory [Mossé 1992]. Our hierarchy cuts a finite word at its last letter, and Lemma 7.4 identifies the retained positions with the occurrences of a terminal word. The connection is the recoding of intervals between repeated occurrences. Here the chosen marker directly delimits the blocks, so unique parsing needs no recognizability theorem. We use the hierarchy as a measure of the complexity of a hole whose levels can be bounded (Theorem 7.7) and compressed (Theorem 8.1).

Automata with Constraints. Our problem resembles emptiness for automata with equality and disequality constraints, where pumping must preserve the constraints. For tree automata with constraints between brothers [Bogaert and Tison 1992], reduction automata [Dauchet et al. 1995], and global constraints [Barguñó et al. 2010; Comon et al. 2008; Comon and Jacquemard 2003; Filiot et al. 2010], emptiness proofs replace subterms *simultaneously* while preserving states and equality patterns, rather than pumping one subterm at a time. The column summaries of Section 8 follow the same principle for the blocks of one hierarchy level. The new difficulty in NSSE is that the constraints compare suffixes of a single word at unbounded offsets; the admission cone and the depth bound are what reduce it to a setting where simultaneous replacement suffices.

13 From the Binary Proof to the General Proof

An earlier version of this work treated only the binary covariant signature, through cap automata. The present proof constructs readers directly from subtype constraints and handles arbitrary arity and variance. This section explains the change. Appendix I lists the distinctions on which both proofs rely, with the places where the present proof handles them.

13.1 An Earlier Proof for the Binary Covariant Case

The earlier version decided NSSE for the binary covariant interpretation of the TLCA problem, over the same three domains. It was based on the *cap automata* of Niehren and Priesnitz [2003]: finite automata with additional *P-edges*. A P-edge from a state q' back to a state q makes the automaton accept πu whenever π leads to q, a word ρ leads from q to q', and u is a prefix of a power of ρ (Appendix J gives the full definition). The earlier version mechanized NP03's forward characterization: for a satisfiable system, the left and right cap automata of a judgment accept exactly the safe paths, once ready states are counted as final (Section 12.1). Holes were then read from transition images. The transition relations of each cap automaton form a finite monoid. A deterministic reader over this monoid, with an admission relation derived from the P-edges, has as its holes exactly the words that the cap automaton rejects. The tail, depth, and compression arguments of Sections 6 to 8 were first proved for these readers over the binary alphabet. The completeness of the characterization and the countermodel transfers both used chains of fresh variables that force a top or a bottom label along an unsafe path.

13.2 Limits of the Earlier Proof

The cap-automaton proof covers only its binary covariant signature. Extending it to larger arities through NP03's arity equivalence would rely on a published proof that is incomplete (Section 12.1),

and contravariance would require new automata with a new completeness proof. We instead derived the reader directly from the semantics, which has three advantages.

- *One construction replaces several.* The chains that the earlier proof used inside its completeness argument and its transfers become the central object. The four-spine extension characterizes unsafety (Theorem 4.9), and its clash analysis (Theorem 4.13) defines the reader. The selected least shape of the same extension supplies the countermodels in all three domains.
- *Admissions have a semantic reading.* Each admitted comparison is a cross or a self event: a constructor bound at one cut meets an incompatible bound at a later cut across a repeated suffix. In the earlier proof the same comparisons appeared as closed P-edges in a transition monoid, which hides the constraints responsible for them.
- *Readers are smaller.* The label reader has $2 \cdot 4^{2k}$ states, and one construction serves both sides by order duality. A cap automaton has at most $N_{\mathrm{cap}} = (k+1)^2 + 1$ states, and its transition monoid can have up to $2^{N_{\mathrm{cap}}^2}$ elements. The rejected-tail bound therefore drops from $2^{O(k^4)}$ to $2^{O(k)}$. Either reader construction, followed by the same compression, gives a triply exponential length bound; the smaller direct reader improves the top exponent from $O(k^4)$ to $O(k)$.

The new route has a cost: its semantic reduction is longer. The four spines with polarity-dependent signs, the signed fillers, and the possible aliasing of roots at cut zero all need care, whereas the earlier proof could follow the published forward construction with its ready-state finality convention.

In summary, the combinatorial core carries over unchanged. The tail and depth arguments are alphabet-generic under their respective hypotheses, and bounded-depth compression applies to every finite-state lettered monitor over any finite alphabet, including the cap-monoid readers, once a depth bound is supplied. The direct reader replaces the cap automata, their transition monoids, and the published forward characterization. In exchange, the semantic reduction must itself establish the rejected-path property, from which the depth bound follows.

14 Discussion and Open Problems

Non-structural subtype entailment with one constructor of any arity and variance is decidable over arbitrary, regular, and finite trees. The proof reduces non-entailment to holes of finite readers, uses the rejected-path property of these readers to bound the rejected tail and hence the depth of a block hierarchy, and then shortens all blocks of each level simultaneously. The questions below concern the cost of this procedure, the role of the rejected-path property, and richer signatures and conclusions.

14.1 Complexity and Sufficient Tests

Section 10 places entailment in 2-EXPSPACE for every arity and variance, in each domain, and for a covariant constructor of each fixed arity $n \geq 2$ it is PSPACE-hard (Theorem 10.7). Three questions remain open. First, whether entailment for $n \geq 2$ is in PSPACE, or at least below 2-EXPSPACE. Second, whether the PSPACE bound at arity one (Appendix F) is tight. Third, whether a left or right label reader that has a hole always has one of length singly exponential in its state count N. The long holes of Proposition 10.6 do not settle the third question: their length $2^{\Omega(k)}$ is only polynomial in $N = 2^{4k+1}$, whereas the length bound H is triply exponential in k. In the fully covariant case, self events vanish, but the dictionary bound does not exploit this to improve the estimate. Reducing the depth, sharpening the dictionary bound, or using the label readers' stronger restrictions on admission (Section 5.3) may narrow the gap between PSPACE and 2-EXPSPACE.

The sufficient test of Lemma 6.3 could also serve as an early non-entailment test in a simplifier; its cost is polynomial in the explicit reader graph, which can itself be exponential in the input.

14.2 The General Hole Problem

For arbitrary readers, the remaining question can be stated precisely: is there a computable function $B(N)$ such that every N-state reader over any finite alphabet with a hole has one with rejected tail at most $B(N)$? Under the rejected-path property, $B(N) = 4N - 2$ works (Theorem 6.1). The depth and compression theorems hold for every reader, so such a function would decide hole existence by searching to $\mathsf{Len}(N, 2N(B(N) + 1) - 1)$. Conversely, a decision procedure for hole existence would give such a function, even uniformly over finite alphabets.

The uniformity rests on an alphabet reduction. There are at most N^N letter transitions $Q \to Q$, and for hole existence it suffices to retain at most two letters with each transition. Indeed, let w be a hole. If no letter of w shares its transition with another letter of the alphabet, w already uses only retained letters. Otherwise, consider the last position whose letter shares its transition. Replace every letter by the first retained letter with the same transition, except at that position, where a second letter with that transition serves as a marker occurring nowhere else. The run is unchanged. The marker excludes every nonterminal comparison ending at or before the marked position. Every comparison ending after it compares segments made of letters with unique transitions, which the replacement leaves unchanged, so it already existed in w with the same states. The new word is therefore a hole with the same rejected tail. Conversely, injecting the retained alphabet into the original one preserves holes. Up to renaming, there are thus finitely many N-state readers to consider, each with at most $2N^N$ letters. Given a hole-existence decision procedure, decide which of them have holes, enumerate a hole for each positive answer, and take the maximum rejected-tail length, or zero if there are none. This maximum is a computable bound for every N-state reader. Thus decidability of hole existence for all finite readers and the computable-tail question are equivalent. The linear bound of Theorem 6.1 answers the question only under the rejected-path property.

Hole languages need not be regular, even for label readers. Take the binary covariant constructor, the single literal $f(y, x) \le y$, and the query $x \le y$ from Section 1. A word is right unsafe exactly when it has the form $0^i 1z$ with $z \not\preceq (0^i 1)^\omega$. Indeed, let σ be a solution in which σy has a bottom prefix on $0^i 1z$. Since σy lies below its 0-child, its nodes on 0^i are constructors, each lying below its own 0-child, so its subtree at $0^i 1$ lies above σx. The bottom prefix lies at $0^i 1u$ with $u \preceq z$, and then σx has a bottom prefix on u. If $z \preceq (0^i 1)^\omega$, then u and $0^i 1z$ prefix the same infinite word, so $u \preceq 0^i 1z$ and the word is not right unsafe. Conversely, let σx have constructors exactly at the prefixes of $w = 0^i 1z$ and bottom leaves elsewhere, and let $\sigma y = f(\sigma y, \sigma x)$. Then σy has a constructor at w iff $z \preceq 0^i 1z$, that is, iff $z \preceq (0^i 1)^\omega$; otherwise it has a bottom prefix on w. The system is satisfiable, so by Theorem 5.4 these words are exactly the holes of the right label reader $(\mathcal{M}_\mathsf{r}, \mathsf{Adm}_\mathsf{r}, \mathsf{Rej}_\mathsf{r})$. Their intersection with 0^*10^* is $\{0^i 10^j : j > i\}$, which is not regular. Our finite search decides existence under the rejected-path property without constructing an automaton for the whole hole language.

14.3 Several Constructors

With several constructor labels, the label order need no longer be the chain $\bot < f < \top$: different constructors may be incomparable. The label-clash analysis must then distinguish incompatible constructors, and a least-shape construction must choose a compatible constructor as well as decide where a constructor is needed. Path polarity depends on which constructor occurs at each node, and available child positions depend on that constructor's arity. The normalization and path decompositions would have to track this additional information.

Two parts of the present proof may transfer more readily. Signed translation with polarity selection is a general device for variance: solve a covariant system on two copies of each variable, then select a solution satisfying the required relation between the copies. The alphabet-generic

word bounds remain available for any semantic reduction that produces finite readers with the rejected-path property, for instance from string constraints or automata with equality constraints that yield suffix-periodicity conditions. The unresolved part is obtaining such a reduction. Several nullary base types together with one binary constructor form a natural next case for type inference; no result for that signature is claimed here.

14.4 Richer Entailment Fragments

The judgment studied here has one variable inequality as its conclusion; model-theoretically, it is the validity of a universal implication $\forall \vec{x}\,(\phi \Rightarrow x \leq y)$. Natural extensions include existential entailment $\phi \models \exists \vec{z}.\, \psi$ and entailment of a disjunction of inequalities. These conclusions ask either for suitable values of $\vec{z}$ in each solution or for one of several inequalities to hold; neither is decided by testing the component inequalities independently. Undecidability of the full first-order theory [Su et al. 2002] does not settle these individual fragments. Determining which such fragments admit finite reader reductions remains an open direction.

Use of LLM-Based Tools

Tools based on large language models (LLMs) assisted proof attempts, checks of arguments, exploratory programs, and the formalization. Kernel-checked proofs support the decision procedures (Section 11). Search results were treated as evidence, not as proofs of universal claims.

References

Alexander Aiken and Edward L. Wimmers. 1993. Type Inclusion Constraints and Type Inference. In *Proceedings of the Conference on Functional Programming Languages and Computer Architecture (FPCA)*. 31–41.

Roberto M. Amadio and Luca Cardelli. 1993. Subtyping Recursive Types. *ACM Transactions on Programming Languages and Systems* 15, 4 (1993), 575–631.

Luis Barguñó, Carles Creus, Guillem Godoy, Florent Jacquemard, and Camille Vacher. 2010. The Emptiness Problem for Tree Automata with Global Constraints. In *Proceedings of the 25th Annual IEEE Symposium on Logic in Computer Science (LICS)*. 263–272. doi:10.1109/LICS.2010.28

Bruno Bogaert and Sophie Tison. 1992. Equality and Disequality Constraints on Direct Subterms in Tree Automata. In *STACS 92 (Lecture Notes in Computer Science, Vol. 577)*. Springer, 161–171. doi:10.1007/3-540-55210-3_181

Giuseppe Castagna and Zhiwu Xu. 2011. Set-Theoretic Foundation of Parametric Polymorphism and Subtyping. In *Proceedings of the 16th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. 94–106. doi:10.1145/2034773.2034788

Hubert Comon, Max Dauchet, Rémi Gilleron, Florent Jacquemard, Denis Lugiez, Christof Löding, Sophie Tison, and Marc Tommasi. 2008. Tree Automata Techniques and Applications. Online book, 2008 release.

Hubert Comon and Florent Jacquemard. 2003. Ground Reducibility Is EXPTIME-Complete. *Information and Computation* 187, 1 (2003), 123–153. doi:10.1016/S0890-5401(03)00134-2

Max Dauchet, Anne-Cécile Caron, and Jean-Luc Coquidé. 1995. Automata for Reduction Properties Solving. *Journal of Symbolic Computation* 20, 2 (1995), 215–233. doi:10.1006/jsco.1995.1048

Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction – CADE 28 (Lecture Notes in Computer Science, Vol. 12699)*. Springer, 625–635. doi:10.1007/978-3-030-79876-5_37

Stephen Dolan. 2016. *Algebraic Subtyping*. Ph. D. Dissertation. University of Cambridge.

Stephen Dolan and Alan Mycroft. 2017. Polymorphism, Subtyping, and Type Inference in MLsub. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL)*. 60–72. doi:10.1145/3009837.3009882

Andrej Dudenhefner, Moritz Martens, and Jakob Rehof. 2017. The Algebraic Intersection Type Unification Problem. *Logical Methods in Computer Science* 13, 3 (2017). doi:10.23638/LMCS-13(3:9)2017

Fabien Durand. 1998. A Characterization of Substitutive Sequences Using Return Words. *Discrete Mathematics* 179, 1–3 (1998), 89–101. doi:10.1016/S0012-365X(97)00029-0

Valery G. Durnev. 1995. Undecidability of the Positive $\forall\exists^3$-Theory of a Free Semigroup. *Siberian Mathematical Journal* 36, 5 (1995), 917–929.

Jonathan Eifrig, Scott Smith, and Valery Trifonov. 1995. Sound Polymorphic Type Inference for Objects. In *Proceedings of the Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*. 169–184.

Emmanuel Filiot, Jean-Marc Talbot, and Sophie Tison. 2010. Tree Automata with Global Constraints. *International Journal of Foundations of Computer Science* 21, 4 (2010), 571–596. doi:10.1142/S012905411000743X

Nathan J. Fine and Herbert S. Wilf. 1965. Uniqueness Theorems for Periodic Functions. *Proc. Amer. Math. Soc.* 16 (1965), 109–114. doi:10.1090/S0002-9939-1965-0174934-9

Cormac Flanagan and Matthias Felleisen. 1999. Componential Set-Based Analysis. *ACM Transactions on Programming Languages and Systems* 21, 2 (1999). doi:10.1145/316686.316703

Alexandre Frey. 1997. Satisfying Subtype Inequalities in Polynomial Space. In *Static Analysis Symposium (SAS) (Lecture Notes in Computer Science, Vol. 1302)*. Springer, 265–277.

Dominik D. Freydenberger. 2011. Extended Regular Expressions: Succinctness and Decidability. In *28th International Symposium on Theoretical Aspects of Computer Science (STACS 2011) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 9)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 507–518. doi:10.4230/LIPIcs.STACS.2011.507

You-Chin Fuh and Prateek Mishra. 1990. Type Inference with Subtypes. *Theoretical Computer Science* 73, 2 (1990), 155–175.

Nils Gesbert, Pierre Genevès, and Nabil Layaïda. 2015. A Logical Approach to Deciding Semantic Subtyping. *ACM Transactions on Programming Languages and Systems* 38, 1 (2015), 3:1–3:31. doi:10.1145/2812805

Simon Guilloud and Viktor Kunčak. 2025. Orthologic Type Systems. arXiv:2507.10482.

Fritz Henglein and Jakob Rehof. 1997. The Complexity of Subtype Entailment for Simple Types. In *Proceedings of the 12th Annual IEEE Symposium on Logic in Computer Science (LICS)*. 352–361. doi:10.1109/LICS.1997.614961

Fritz Henglein and Jakob Rehof. 1998. Constraint Automata and the Complexity of Recursive Subtype Entailment. In *Automata, Languages and Programming (ICALP 1998) (Lecture Notes in Computer Science, Vol. 1443)*. Springer, 616–627. doi:10.1007/BFb0055089

Shengyi Jiang, Chen Cui, and Bruno C. d. S. Oliveira. 2025. Bidirectional Higher-Rank Polymorphism with Intersection and Union Types. *Proceedings of the ACM on Programming Languages* 9, POPL (2025). doi:10.1145/3704907

Dexter Kozen, Jens Palsberg, and Michael I. Schwartzbach. 1994. Efficient Inference of Partial Types. *J. Comput. System Sci.* 49, 2 (1994), 306–324.

Dexter Kozen, Jens Palsberg, and Michael I. Schwartzbach. 1995. Efficient Recursive Subtyping. *Mathematical Structures in Computer Science* 5 (1995), 1–13.

Markus Krötzsch, Tomáš Masopust, and Michaël Thomazo. 2016. On the Complexity of Universality for Partially Ordered NFAs. In *41st International Symposium on Mathematical Foundations of Computer Science (MFCS 2016) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 58)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 61:1–61:14. doi:10.4230/LIPIcs.MFCS.2016.61

Viktor Kuncak and Martin Rinard. 2003. Structural Subtyping of Non-Recursive Types Is Decidable. In *Proceedings of the 18th Annual IEEE Symposium on Logic in Computer Science (LICS)*. doi:10.1109/LICS.2003.1210049

M. Lothaire. 2002. *Algebraic Combinatorics on Words*. Encyclopedia of Mathematics and its Applications, Vol. 90. Cambridge University Press.

Gennadiĭ S. Makanin. 1977. The Problem of Solvability of Equations in a Free Semigroup. *Mathematics of the USSR-Sbornik* 32, 2 (1977), 129–198.

John C. Mitchell. 1991. Type Inference with Simple Subtypes. *Journal of Functional Programming* 1, 3 (1991), 245–285.

Brigitte Mossé. 1992. Puissances de mots et reconnaissabilité des points fixes d'une substitution. *Theoretical Computer Science* 99, 2 (1992), 327–334.

Martin Müller, Joachim Niehren, and Andreas Podelski. 2000. Ordering Constraints over Feature Trees. *Constraints* 5, 1–2 (2000), 7–42.

Martin Müller, Joachim Niehren, and Ralf Treinen. 1998. The First-Order Theory of Ordering Constraints over Feature Trees. In *Proceedings of the 13th Annual IEEE Symposium on Logic in Computer Science (LICS)*. 432–443. doi:10.1109/LICS.1998.705677

Joachim Niehren, Martin Müller, and Jean-Marc Talbot. 1999. Entailment of Atomic Set Constraints Is PSPACE-Complete. In *Proceedings of the 14th Annual IEEE Symposium on Logic in Computer Science (LICS)*. 285–294. doi:10.1109/LICS.1999.782623

Joachim Niehren and Tim Priesnitz. 1999. Entailment of Non-structural Subtype Constraints. In *Advances in Computing Science – ASIAN'99 (Lecture Notes in Computer Science, Vol. 1742)*. Springer, 251–265. doi:10.1007/3-540-46674-6_22

Joachim Niehren and Tim Priesnitz. 2001. Non-structural Subtype Entailment in Automata Theory. In *Theoretical Aspects of Computer Software (TACS 2001) (Lecture Notes in Computer Science, Vol. 2215)*. Springer, 360–384.

Joachim Niehren and Tim Priesnitz. 2003. Non-structural Subtype Entailment in Automata Theory. *Information and Computation* 186, 2 (2003), 319–354. doi:10.1016/S0890-5401(03)00140-8

Joachim Niehren, Tim Priesnitz, and Zhendong Su. 2005. Complexity of Subtype Satisfiability over Posets. In *Programming Languages and Systems (ESOP 2005) (Lecture Notes in Computer Science, Vol. 3444)*. Springer, 357–373. doi:10.1007/978-3-540-31987-0_25

Jens Palsberg and Patrick O'Keefe. 1995. A Type System Equivalent to Flow Analysis. *ACM Transactions on Programming Languages and Systems* 17, 4 (1995), 576–599.

Jens Palsberg, Mitchell Wand, and Patrick O'Keefe. 1997. Type Inference with Non-structural Subtyping. *Formal Aspects of Computing* 9 (1997), 49–67. doi:10.1007/BF01212524

Lionel Parreaux. 2020. The Simple Essence of Algebraic Subtyping: Principal Type Inference with Subtyping Made Easy (Functional Pearl). *Proceedings of the ACM on Programming Languages* 4, ICFP (2020), 124:1–124:28. doi:10.1145/3409006

Lionel Parreaux, Aleksander Boruch-Gruszecki, Andong Fan, and Chun Yin Chau. 2024. When Subtyping Constraints Liberate: A Novel Type Inference Approach for First-Class Polymorphism. *Proceedings of the ACM on Programming Languages* 8, POPL (2024). doi:10.1145/3632890

Lionel Parreaux and Chun Yin Chau. 2022. MLstruct: Principal Type Inference in a Boolean Algebra of Structural Types. *Proceedings of the ACM on Programming Languages* 6, OOPSLA2 (2022). doi:10.1145/3563304

Wojciech Plandowski. 2004. Satisfiability of Word Equations with Constants Is in PSPACE. *Journal of the ACM* 51, 3 (2004), 483–496.

François Pottier. 1996. Simplifying Subtyping Constraints. In *Proceedings of the First ACM SIGPLAN International Conference on Functional Programming (ICFP)*. 122–133. doi:10.1145/232627.232642

François Pottier. 1998a. A Framework for Type Inference with Subtyping. In *Proceedings of the Third ACM SIGPLAN International Conference on Functional Programming (ICFP)*. 228–238. doi:10.1145/289423.289448

François Pottier. 1998b. *Type Inference in the Presence of Subtyping: From Theory to Practice.* Ph. D. Dissertation. Université Paris 7.

François Pottier. 2001. Simplifying Subtyping Constraints: A Theory. *Information and Computation* 170, 2 (2001), 153–183. doi:10.1006/inco.2001.2963

Vaughan Pratt and Jerzy Tiuryn. 1996. Satisfiability of Inequalities in a Poset. *Fundamenta Informaticae* 28, 1–2 (1996), 165–182. doi:10.3233/FI-1996-281211

Tim Priesnitz. 2000. *Entailment von nicht-strukturellen Teiltyp-Constraints.* Diplomarbeit. Universität des Saarlandes.

Tim Priesnitz. 2004. *Subtype Satisfiability and Entailment.* Ph. D. Dissertation. Saarland University. doi:10.22028/D291-25888

Jakob Rehof. 1996. Problem #16: Is the Subtype Entailment Problem Decidable? TLCA List of Open Problems. https://tlca.di.unito.it/opltlca/opltlcasu23.html

Jakob Rehof. 1997. Minimal Typings in Atomic Subtyping. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 278–291. doi:10.1145/263699.263738

Jakob Rehof. 1998. *The Complexity of Simple Subtyping Systems.* Ph. D. Dissertation. DIKU, University of Copenhagen.

Walter J. Savitch. 1970. Relationships between Nondeterministic and Deterministic Tape Complexities. *J. Comput. System Sci.* 4, 2 (1970), 177–192. doi:10.1016/S0022-0000(70)80006-X

Aleksy Schubert. 2021. On Non-structural Subtype Entailment. arXiv:2109.02458. Withdrawn by the author in December 2021.

Vincent Simonet. 2003. Type Inference with Structural Subtyping: A Faithful Formalization of an Efficient Constraint Solver. In *Programming Languages and Systems (APLAS 2003) (Lecture Notes in Computer Science, Vol. 2895)*. Springer, 283–302.

R. E. Stearns, J. Hartmanis, and P. M. Lewis, II. 1965. Hierarchies of Memory Limited Computations. In *6th Annual Symposium on Switching Circuit Theory and Logical Design (SWCT 1965)*. IEEE, 179–190. doi:10.1109/FOCS.1965.11

Larry J. Stockmeyer and Albert R. Meyer. 1973. Word Problems Requiring Exponential Time: Preliminary Report. In *Proceedings of the 5th Annual ACM Symposium on Theory of Computing (STOC)*. 1–9. doi:10.1145/800125.804029

Zhendong Su. 2002. *Algorithms for and the Complexity of Constraint Entailment.* Ph. D. Dissertation. University of California, Berkeley.

Zhendong Su, Alexander Aiken, Joachim Niehren, Tim Priesnitz, and Ralf Treinen. 2002. The First-Order Theory of Subtyping Constraints. In *Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 203–216. doi:10.1145/503272.503292

Zhendong Su, Alexander Aiken, Joachim Niehren, Tim Priesnitz, and Ralf Treinen. 2006. The First-Order Theory of Subtype Constraints. Extended version of the POPL 2002 paper. https://www.ps.uni-saarland.de/Publications/documents/fot_04.pdf

The mathlib Community. 2020. The Lean Mathematical Library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*. 367–381. doi:10.1145/3372885.3373824

Jerzy Tiuryn. 1992. Subtype Inequalities. In *Proceedings of the Seventh Annual IEEE Symposium on Logic in Computer Science (LICS)*. 308–315.

Ralf Treinen. 2000. Predicate Logic and Tree Automata with Tests. In *Foundations of Software Science and Computation Structures (FoSSaCS 2000) (Lecture Notes in Computer Science, Vol. 1784)*. Springer, 329–343.

Valery Trifonov and Scott Smith. 1996. Subtyping Constrained Types. In *Static Analysis Symposium (SAS) (Lecture Notes in Computer Science, Vol. 1145)*. Springer, 349–365. doi:10.1007/3-540-61739-6_52

A Details of the Spine Proofs

A.1 The Bridge Invariant

We complete the left-to-right direction of Lemma 4.11, using its four simultaneous obligations: old-to-old reflection, the two path memberships, and the bridge formula. All path judgments are in ψ. Reflexivity gives the old-to-old obligation and no pair between two distinct fresh spine variables. For transitivity, an intermediate source forces the incoming pair to be its identity, and an intermediate sink forces the outgoing pair to be its identity, by (28). These cases reduce to a subderivation. An old intermediate variable preserves (30) by upward or downward closure. For an R_j-to-$Z_{j'}$ pair through an old v, (30) gives $v \in U_j^R \cap L_{j'}^Z$, which is (31) with $\alpha = j, \gamma = j'$.

In a decomposition concluding at R_j, $j > 0$, the lower bounding literal must be the unique R-link in which R_j is a child, and the selected child position must be $w[j-1]$. For an old right endpoint, the upper literal is old. The induction hypothesis at the root pair and the one-step rule (15) give membership in U_j^R. The old-to-$Z_{j'}$ case is dual. If both endpoints are $R_j, Z_{j'}$, their two unique parent links force $w[j-1] = w[j'-1]$. Apply induction to their roots and append this common letter to the two equal factors in (31). When one parent index is zero, its root is old. If the lower parent is R_0 and the upper parent is $Z_{j'-1}$, (30) puts R_0 in $L_{j'-1}^Z$, while reflexivity puts it in U_0^R. This supplies a bridge with empty descent before the final letter. The other orientation is symmetric. If both parents have index zero, separation reflects their pair to $D_\psi(R_0, Z_0)$, and Z_0 lies in $U_0^R \cap L_0^Z$. An old-to-old decomposition has already been dealt with in (28). If the selected child is off-spine, the conclusion has a bottom filler on the left or a top filler on the right, so no membership or bridge obligation applies. The terminal lower bound at W_m has only bottom-filler children; the terminal upper bound at Y_m has only top-filler children. Thus their decomposition consequences are covered by the same cases.

This covers every closure rule. Because positive-index Q- and Y-variables are sinks, no bridge passes through them, and the fillers create no further bridges.

It remains to justify zero endpoints in (31). If $j = 0$, (30) says $R_0 \in L_{j'}^Z$; this gives the formula with $\alpha = 0, \gamma = j', v = R_0$. Conversely, its factor-length equation forces $\gamma = j'$, and a member of $U_0^R \cap L_{j'}^Z$ yields that same membership by downward closure. The case $j' = 0$ is symmetric. Thus the induction covers every cut.

A.2 The Remaining Clash Cases

We complete the classification in Theorem 4.13. Bound families refer to literal provenance, including new literals whose cut-zero roots are old. If the lower bound is old and the upper bound is on $Q_{j'}$, an old top gives a member of $\mathrm{Top} \cap L_{j'}^Q$. By (19) and flip closure, its variable flip is in $\mathrm{Bot} \cap U_{j'}$, giving bot-readiness. An old constructor can clash only against $Q_m = \bot$; variable flip and order reversal turn its membership in L_m^Q into an upper constructor in U_m, giving (b). An old lower bound clashes against a $Y_{j'}$-bound only when it is top; (30) then gives $\mathrm{top}(L_{j'})$. For a W_j-bound against an old upper bound, the latter must be bottom. Its root belongs to $U_j^W = \iota L_j$; flipping gives an old top in L_j, again top-readiness.

For a W_j-bound against a $Q_{j'}$-bound, incompatibility forces $j' = m$. The bridge gives cuts α, γ with

$$j - \alpha = m - \gamma, \quad w[\alpha : j] = w[\gamma : m], \quad \iota L_\alpha \cap \iota U_\gamma \neq \varnothing.$$

Flipping a common element gives $L_\alpha \cap U_\gamma \neq \varnothing$. Since $j \leq m$, $\alpha \leq \gamma$. Equality gives reflexivity readiness; otherwise take $s = \alpha, e = \gamma$. Then $j = s + m - e$ and the factor equality is exactly $w[e:] \preceq w[s:]$, with cross admission. This includes $e = m$, when descent is empty. A W-bound against a Y-bound always has the compatible labels f, f, even at terminal cuts. Its closure bridges

may exist, but their decomposition consequences are already covered by the full invariant in Lemma 4.11.

A.3 Readiness and Child Events Give Clashes

Suppose a readiness or child event occurs. If $v \in U_j \cap \mathrm{Bot}$, (30) gives $D(P_j, v)$, and the lower label on P_j, either f or $\top$, clashes with bottom. If $v \in L_j \cap \mathrm{Top}$, then $D(v, Y_j)$ clashes with the upper constructor on Y_j. If $v \in U_j \cap L_j$, then $D(P_j, Y_j)$; decomposing down the common remaining suffix gives $D(P_m, Y_m)$, a top/constructor clash. An upper constructor in U_m likewise clashes with $P_m = \top$.

B Proof of the Cores Lemma

We prove Lemma 7.4. Let w be nonempty with $m = |w|$, let u_j and ℓ_j be its hierarchy and markers, and let $\mathit{core}_j(x)$ be the core of position x of u_j.

Part 1. The cores of u_{j+1} are a new start core followed by the cores of the nonfinal occurrences of ℓ_j in u_j, in their order in u_j. By induction on j, the start cores of u_j therefore form an initial segment of pairwise distinct cores, the ordinary cores increase from left to right, and $C_{j+1} \subseteq C_j$. The start cores of u_{j+1} are the new one and those inherited from the start segment of u_j, so there are at most $1 + j$ of them.

Shifted Expansions. Let Γ_0 be the alphabet of w and $\Gamma_{j+1} = (\Gamma_j \setminus \{\ell_j\})^*$, so that u_j is a word over Γ_j. Define $\xi_0(a) = a$ and

$$\xi_{j+1}([X]) = \xi_j(\ell_j)\, \xi_j(X[0]) \cdots \xi_j(X[b-1]) \qquad \text{for } X = X[0] \cdots X[b-1].$$

A set of nonempty words is a *code* if every concatenation of its elements has a unique factorization into those elements; prefix-freeness is not required.

(a) *ξ_j is injective, and $\xi_j(\Gamma_j)$ is a code.* At level zero, the image consists of one-letter words. For the induction step, the map $[X] \mapsto \ell_j X$ has a code as its image, since each of its words starts with ℓ_j and contains no other ℓ_j. The extension of ξ_j to words is an injective morphism because $\xi_j(\Gamma_j)$ is a code, and the image of a code under an injective morphism is a code.

(b) *Invariant.* Put $t_{-1} = m$. If $C_j \neq \emptyset$, then every position after an ordinary position of u_j is ordinary; for every ordinary position x of u_j other than the last, $w[\mathit{core}_j(x){:}\mathit{core}_j(x+1)] = \xi_j(u_j[x])$; and for the last position, $w[t_j{:}t_{j-1}] = \xi_j(\ell_j)$.
The case $j = 0$ is immediate. For the step, let $x_0 < \cdots < x_{h-1}$ be the positions of ℓ_j in u_j, so that x_{h-1} is its last position and position $i \geq 1$ of u_{j+1} holds the block formed by the positions $x_{i-1} + 1, \ldots, x_i - 1$ and has the core $\mathit{core}_j(x_{i-1})$. If that core is ordinary, the positions $x_{i-1}, \ldots, x_i - 1$ of u_j are ordinary and not last, so by induction the factor of w from $\mathit{core}_j(x_{i-1})$ to $\mathit{core}_j(x_i)$ is $\xi_j(\ell_j)\, \xi_j(u_j[x_{i-1}+1]) \cdots \xi_j(u_j[x_i - 1]) = \xi_{j+1}(u_{j+1}[i])$. For $i < h-1$, the endpoint $\mathit{core}_j(x_i)$ is $\mathit{core}_{j+1}(i+1)$; for $i = h-1$, it is $\mathit{core}_j(x_{h-1}) = t_j$, and the letter $u_{j+1}[h-1]$ is the marker ℓ_{j+1}.

Write $r_j = w[t_j{:}]$ for the terminal core word of level j and $r_{-1} = \varepsilon$. By (b), $r_j = \xi_j(\ell_j)\, r_{j-1}$.

Part 2. We show by induction on j that, if $C_j \neq \emptyset$, then $C_{j+1} = \{s \mid s < t_j,\ r_j \preceq w[s{:}]\}$. (If $C_j = \emptyset$, then $C_{j+1} = \emptyset$ by Part 1.)

($\subseteq$) Let $s \in C_{j+1}$. Then s is the core of a nonfinal occurrence x of ℓ_j in u_j, so $s < t_j$, and by (b) the factor of w from s to $\mathit{core}_j(x+1)$ is $\xi_j(\ell_j)$. For $j = 0$ this factor is r_0. For $j > 0$, the core $\mathit{core}_j(x+1)$ belongs to C_j and hence, by the induction hypothesis, is an occurrence of r_{j-1}. So $w[s{:}]$ starts with $\xi_j(\ell_j)\, r_{j-1} = r_j$.

($\supseteq$) Let $s < t_j$ with $r_j \preceq w[s{:}]$. For $j = 0$, $r_0 = \ell_0$ is the last letter of w, and s is a nonfinal occurrence of it, so $s \in C_1$. Let $j > 0$. By the induction hypothesis, the occurrences of r_{j-1} in w are

exactly the elements of $C_j \cup \{t_{j-1}\}$ (an occurrence i satisfies $i \le t_{j-1}$, since $|r_{j-1}| = m - t_{j-1}$). As $t_j \in C_j$, the word r_{j-1} is a prefix of r_j, so s is an occurrence of r_{j-1} with $s < t_{j-1}$; thus $s \in C_j$, say $s = \mathit{core}_j(x)$, and x is not the last position because $s < t_j$. Inside the suffix r_j of w, the occurrences of r_{j-1} are at t_j and t_{j-1} only, because $t_j = \max C_j$. Hence the word r_j contains r_{j-1} exactly at the offsets 0 and $\varpi = t_{j-1} - t_j$. Consequently the occurrences of r_{j-1} in w that start in $[s, s+\varpi]$ are exactly s and $s+\varpi$: an occurrence starting strictly between them ends before $s + |r_j|$ and would be a third occurrence inside the copy of r_j at s. Since $s + \varpi < t_{j-1}$, the next element of C_j after s is $s + \varpi$, and by Part 1 it is $\mathit{core}_j(x+1)$. By (b), $\xi_j(u_j[x]) = w[s{:}s+\varpi] = w[t_j{:}t_{j-1}] = \xi_j(\ell_j)$, and injectivity gives $u_j[x] = \ell_j$. Since x is not the last position, $s \in C_{j+1}$. □

C Details of the Bounded-Depth Compression

We prove the dictionary normal form and the compression statements of Section 8 for an arbitrary lettered monitor with $N \ge 1$ states. Its nonempty holes supply every marker used below; no finiteness assumption on the alphabet is needed. All dictionaries and tables considered here are finite.

C.1 Comparisons and Isolated Exchange

Lemma C.1 (Comparison-preserving maps). *Let v be a nonempty word and θ a letter map. Suppose that θ identifies no unequal predecessor pair of any comparison, including terminal ones:*

$$1 \le s < e \le |v|, \quad v[e:] \preceq v[s:], \quad \theta(v[s-1]) = \theta(v[e-1]) \implies v[s-1] = v[e-1].$$

Then v and $\theta(v)$ have exactly the same comparisons. Moreover, on the letters of v, $\theta(a) = \theta(\ell)$ implies $a = \ell$, where ℓ is the last letter of v.

Proof. A letter map preserves every old comparison. If (s, e) is a new one, choose the largest offset $t < |v| - e$ at which $v[s+t] \ne v[e+t]$. The remaining suffixes agree, so $(s+t+1, e+t+1)$ is an old comparison. Its unequal predecessors have equal images, contrary to the hypothesis. To prove the last assertion, an occurrence of a before the final position gives a terminal comparison between the cuts following that occurrence and the last letter. Its predecessors cannot be identified unless they agree. The final occurrence itself is immediate. □

We first justify the state information used by the exchange types. The state set at level i is Q with i successive symbolic states adjoined. Ordinary states form a closed copy of Q: at level zero this is immediate, and a derived transition from an ordinary state runs the previous monitor over ℓX, hence stays ordinary by induction. The shifted-run definition also identifies the entry state of a position with its core: an ordinary core p carries the original state at cut p; a symbolic core carries the corresponding adjoined state. Indeed, the first block starts at the new symbolic state, and each later block starts at the entry state of its preceding marker. This is precisely the core recursion of Lemma 7.4. Thus all occurrences of an unprotected letter have ordinary entry states, and every entry state occurring in u_i belongs to Z_i.

Proof of Lemma 8.5. The transition table has N^N possibilities. The target table uses N bits, the two self tables $2N^2$ bits, the protected tables $4N\,|Z_i|\,|\Pi_i|$ bits, and the initial rows $2N$ bits. Hence the number of types is at most

$$N^N 2^{2N^2 + 4N|Z_i||\Pi_i| + 3N}.$$

Using $|\Pi_i| \le s_i + 1 \le d + 1$ and $|Z_i| \le N + s_i \le N + d$ gives the bound $\mathrm{Ty}(N, d)$ of (43). □

For the local part of Lemma 8.6, fix i, put $v = u_i$, and let θ identify a with a' and fix every other letter. An unequal predecessor pair identified by θ would be (a, a') or (a', a). A nonterminal such pair contradicts isolation; a terminal one would contain the protected marker. Lemma C.1

therefore preserves comparisons. Induction on positions preserves the full state run: the only changed transitions read a at ordinary states, where its transition table equals that of a'. The final letter is protected and unchanged, so its entry state and target test are unchanged.

For each comparison, consider its predecessor labels. If neither letter is a, the labels are unchanged. If both are a, the relevant self table preserves admission. If exactly one is a, the other is protected: for a nonterminal comparison this follows from isolation, and for a terminal comparison its later predecessor is the marker. The appropriate protected row, in the appropriate orientation, preserves either admission predicate. A comparison starting at zero uses the initial-label row; at positive levels this row is false. Protected letters at symbolic states are covered by the Z_i columns. These cases cover all comparisons, so $\theta(v)$ is a hole of the same level-i monitor. Expanding it with the markers of the original hierarchy below level i gives a nonempty base hole by Lemma 8.3. It remains to prove the asserted structural properties of that expansion.

C.2 Induced Level Maps and Start Counts

Continue a hierarchy past its first unary level when discussing all levels; desubstitution preserves unary words. For a level map θ define $\theta_0 = \theta$ and $\theta_{r+1}(X) = \theta_r^*(X)$, where the star denotes letterwise application to a block body.

Lemma C.2 (Shape under a level map). *Let w be a nonempty hole, and let θ map the dictionary of u_i into itself. Suppose that it identifies no unequal predecessor pair, as in the hypothesis of Lemma C.1, fixes the start prefix $u_i[:s_i]$ letterwise, preserves the level-i state run, and makes $\theta(u_i)$ a hole of $\mathcal{M}_i$. Expand $\theta(u_i)$ with the markers of the original hierarchy below level i to obtain w'. Then w' is a hole, its markers below i are unchanged, and its dictionaries below i are subsets of the old ones. At every $r \geq 0$,*

$$u'_{i+r} = \theta_r(u_{i+r}), \qquad \ell'_{i+r} = \theta_r(\ell_{i+r}),$$

and θ_r identifies no unequal predecessor pair of a comparison of u_{i+r}. Every start count is unchanged, and exactly the same levels are unary. In particular, depth is preserved and no dictionary grows.

Proof. Every image letter occurs in the old dictionary, so it has a valid expansion avoiding each lower marker. Expansion and cutting are inverse at every lower level: a block body avoids its marker and its following marker is present. Thus the decoded hierarchy reaches $f(u_i)$ exactly, with unchanged lower markers. Repeated use of Lemma 8.3 proves holehood. Descending through these expansions also proves dictionary containment below i.

We verify the upward induction. For any map F satisfying the hypothesis of Lemma C.1 on a word u ending in ℓ, Lemma C.1 reflects the marker: no other letter maps to $F(\ell)$. Hence cutting at markers commutes with F. Write V for the block word of u. The induced map F^* on its letters identifies no unequal predecessor pair of an upper comparison. To see this, suppose the cuts following blocks X, Y form a comparison of V, and $F^*(X) = F^*(Y)$. In the image of u, the cuts at the beginnings of these blocks form a comparison: their first blocks agree and the following block suffixes compare. Comparison preservation by F gives the same lower comparison before mapping. Since X, Y avoid ℓ, its first aligned markers force $X = Y$. This proves the required nonidentification property upstairs, including when the upper comparison is terminal. Induction gives the claimed level words, markers, and comparison sets at every higher level.

We show that every start count is unchanged. For any nonempty hierarchy let $m_j = |u_j|$ and write $\#_\ell(v)$ for the number of occurrences of ℓ in v. The core recursion gives

$$s_0 = 0, \qquad s_{j+1} = \min\{1 + \#_{\ell_j}(u_j[:s_j]),\ m_{j+1}\}. \tag{54}$$

There is one new start core, followed by those inherited from nonfinal markers in the old start prefix. The minimum removes the final marker when that whole level is a start prefix. In particular,

if $s_j = m_j$, then $s_{j+1} = m_{j+1}$, since $\#_{\ell_j}(u_j) = m_{j+1}$. At level i the unchanged run and length give $s'_i = s_i$, and the start prefixes agree by hypothesis. Above i, marker reflection and the equal lengths of mapped words preserve both arguments of the minimum in (54); induction gives $s'_j = s_j$.

Below level i we argue as follows. Suppose two hierarchies have the same markers below i, equal start counts at i, and equal start prefixes there. Expanding that common prefix downwards gives, at every $j \le i$, a common prefix of length λ_j with

$$s_j \le \lambda_j \le m_j, \qquad j < i \implies (s_j < \lambda_j \text{ or } \lambda_j = m_j).$$

To check these inequalities, if the upper start prefix is not the whole upper word, (54) says it has one more block than the number of markers in the lower start prefix. Its full expansion therefore ends strictly after that lower prefix. If it is the whole word, its expansion is the whole lower word. Apply this observation inductively; expanding a longer common upper prefix only increases the lower length. Equal lower markers give equal derived monitors through level i, so runs coincide along each common prefix. Every old start position is therefore still a start position: $s'_j \ge s_j$.

For $j < i$, if $s_j < m_j$, an excess $s'_j > s_j$ is impossible. The common prefix contains position s_j, which is ordinary in the old run and would be symbolic in the new one. The remaining case is the cap $s_j = m_j$. Assume $s'_j > s_j$. The old whole word is a prefix of the new one, and the new word is strictly longer. The old upper level is also wholly symbolic and is a common prefix of the new upper level. That upper level must be strictly longer too, since equal upper words with the same marker expand to equal lower words. The new start prefix contains the entire old lower word, hence at least m_{j+1} markers. Formula (54) now gives

$$s'_{j+1} \ge \min\{m_{j+1} + 1, m'_{j+1}\} > m_{j+1} = s_{j+1}.$$

The excess propagates to level i, contradicting $s'_i = s_i$. Thus no cap creates an extra start position, and all lower counts agree.

Finally, marker reflection implies that a higher mapped word is unary iff the old word is unary: if every image letter equals the image marker, every old letter is the marker. If u_i is nonunary, no lower word in either hierarchy is unary, since a unary word has only unary derived levels. If u_i is unary, its dictionary is a singleton and θ maps it into itself, so decoding makes $w' = w$. This proves the claim at lower levels as well. Higher dictionaries are images of old dictionaries, so their sizes cannot increase. □

Completion of Lemma 8.6. The local argument already proved comparison preservation, run preservation, and holehood at level i. The map fixes the start prefix and maps the dictionary into itself, because a, a' occur and are unprotected. Apply Lemma C.2. The new level-i dictionary is the old one with a removed, so its size strictly decreases. Write $|a|_0$ and $|a'|_0$ for the lengths of the ordinary level-zero expansions of a and a' with the markers of the original hierarchy. The decoded length changes by

$$\#_a(u_i)(|a'|_0 - |a|_0).$$

Both merge directions satisfy the hypotheses. Choose the direction with $|a'|_0 \le |a|_0$ to make this difference nonpositive. □

The level maps establish hierarchy geometry, not agreement of all the higher derived predicates under noninjective maps. After decoding, derive the monitors again from the new word. Holehood in those monitors follows from Lemma 8.3. Protected sets, isolation, and exchange types are likewise computed afresh for the next merge.

C.3 Dictionary Normal Form, Boundary Pairs, and the Recurrence

Proof of Lemma 8.7. Use the natural-number potential $\sum_{j=0}^{\Delta} g_j$, the total size of the dictionaries. Whenever dictionary normality fails, merge the offending isolated pair in the direction of smaller expansion. By Lemma 8.6, depth stays Δ, no dictionary grows, and one strictly shrinks, so the potential strictly decreases. The process terminates. Recompute the entire hierarchy and its types after every step; termination therefore means dictionary normality at every positive nonunary level of the final word simultaneously. Length and dictionary nonincrease and start-count equalities compose through the finite sequence of merges. □

Counting part of Lemma 8.8. Partition $\mathcal{A}_i$ into protected letters, isolated unprotected letters, and the remaining letters. The first part has size at most $d+1$. Dictionary normality injects the second part into the type space, so it has size at most $\mathrm{Ty}(N,d)$. Each letter in the third part is a component of some ordered boundary pair with both components unprotected. The union of the two projections of BP_i has size at most $2b_i$. Adding gives $g_i \le \mathsf{Al}(N,d) + 2b_i$. □

Propagation part of Lemma 8.8. For distinct blocks X, Y avoiding a marker ℓ, define $\mu_\ell(X,Y)$ as their first unequal letters when scanned from right to left, using ℓ when one body is exhausted. Equivalently, remove their longest common suffix; take the last remaining letter in each body, or ℓ for an empty remainder. These two letters are unequal. Set $\mu_\ell(X,X) = (\ell,\ell)$ to make the map total.

The *endpoint pairs* of level i are the predecessor pairs of the left-maximal comparisons that end at the final cut:

$$\mathrm{EP}_i = \{(u_i[s-1], u_i[e-1]) : (s,e) \text{ is left maximal},\ e = |u_i|\}.$$

Their second component is the marker ℓ_i.

Take a lower boundary pair witnessed by (s,e) in the expansion of a block word V, with e nonterminal. By Lemma 7.3, the cuts lie in blocks $X = V[i_s]$ and $Y = V[i_e]$, with $i_s < i_e$, the same remaining body suffix, and an upper comparison (i_s+1, i_e+1). Because $s \ge 1$, the predecessor of a cut at a block's beginning is the preceding marker; a cut inside a body has its ordinary predecessor. The unequal lower predecessors are exactly $\mu_\ell(X,Y)$: the common suffix extends right to the marker and cannot extend one more letter left. In particular $X \ne Y$. Thus (X,Y) belongs either to the upper nonterminal boundary pairs or to its endpoint pairs, and

$$\mathrm{BP}_i \subseteq \mu_{\ell_i}(\mathrm{BP}_{i+1} \cup \mathrm{EP}_{i+1}).$$

This also covers lower pairs with one component equal to ℓ_i and upper comparisons ending at the final cut. Every upper endpoint pair has the form (a, ℓ_{i+1}) with $a \ne \ell_{i+1}$. There are at most $g_{i+1} - 1$ such pairs. Taking cardinalities proves (44). □

Proof of Lemma 8.9. All counts now refer to one fixed dictionary-normal hole; abbreviate $\mathsf{Al} = \mathsf{Al}(N,d)$. Its unary top has $g_\Delta = 1$ and $b_\Delta = 0$. If $\Delta \ge 1$, propagation gives $b_{\Delta-1} = 0$. For $1 \le i < \Delta - 1$, counting at level $i+1$ and propagation give

$$b_i \le b_{i+1} + g_{i+1} - 1 \le 3b_{i+1} + \mathsf{Al} - 1.$$

Induct downwards from $\Delta - 1$. If $r = \Delta - 1 - i$ and the bound holds at $i+1$,

$$2b_i \le 3(\mathsf{Al}-1)(3^{r-1}-1) + 2(\mathsf{Al}-1) = (\mathsf{Al}-1)(3^r - 1).$$

The base case has both sides zero. Adding Al gives $g_i \le \mathsf{Al} + 2b_i \le 1 + (\mathsf{Al}-1)3^r$, as claimed. Since $r \le d$ and $\mathsf{Al} \ge 1$, this is at most $\mathrm{Dict}(N,d)$. The unary dictionary has size one, also at most $\mathrm{Dict}(N,d)$. Put an arbitrary hole in dictionary normal form using Lemma 8.7 to obtain the existential assertion. For $\Delta = 0$ there are no positive levels to bound; for $\Delta = 1$ only the unary dictionary is needed. □

C.4 Unary and Column Shortening

Fix a lettered monitor $\mathcal{M}$ with state set Q and start state ζ_0, and a marker ℓ. Write $lab_0(\zeta)$ for the initial shifted-block label: (ζ, ℓ) for any $\zeta \in Q$, and $\triangleright$ at the fresh entry symbol $\star$.

Unary Words. Let $v = z^L$ be a hole of $\mathcal{M}$, with states $\zeta_0, \zeta_1, \dots$ where $\zeta_{p+1} = \delta(\zeta_p, z)$. Since every pair of cuts of a unary word is a comparison, v is a hole iff $T^{\text{fin}}(\zeta_{L-1}, z)$ holds, no label of a cut $s < L$ is related by Λ^{fin} to the last label, and no two labels of cuts $s < e < L$ are related by Λ. If $\zeta_i = \zeta_{i'}$ for some $i < i' \le L - 1$, delete the letters between cuts i and i'. The cut $p > i$ of the new word has the state and label of the cut $p + (i' - i)$ of v, so the labels of the new word form a subsequence of those of v with the same last label. Every condition above is therefore preserved. Repeating, we obtain a unary hole of length at most $|Q|$.

The Column Automaton. Let $X_1, \dots, X_h$ be blocks avoiding ℓ. The summary of Section 8 is a tuple of four functions, where $\mathbb{B}$ is the set of Boolean values:

$$\begin{aligned} exit_a(\zeta) &\in Q, && \text{the exit state of the shifted block of } X_a \text{ read from } \zeta,\\ term_a(\zeta, \zeta'') &\in \mathbb{B}, && \text{whether some label of it is related by } \Lambda^{\text{fin}} \text{ to } (\zeta'', \ell),\\ eq(a, b) &\in \mathbb{B}, && \text{whether } X_a = X_b,\\ adm(a, b, \zeta, \zeta') &\in \mathbb{B}, && \text{whether } \Lambda_\ell \text{ relates } (\zeta, X_a) \text{ and } (\zeta', X_b), \end{aligned}$$

for blocks a, b, states $\zeta, \zeta' \in Q \sqcup \{\star\}$, and states $\zeta'' \in Q$. Its number of values is the function $col(|Q|, h)$ of (47).

Right-align the blocks, pad them on the left, and read the resulting word of columns, each an h-tuple of optional letters, from left to right (Figure 11). The tuple of h empty blocks has the initial summary with $exit_a(\zeta) = \delta(\zeta, \ell)$ for $\zeta \in Q$ and $exit_a(\star) = \zeta_0$, with $term_a(\zeta, \zeta'') = \Lambda^{\text{fin}}(lab_0(\zeta), (\zeta'', \ell))$, with all blocks equal, and with $adm(a, b, \zeta, \zeta') = \Lambda(lab_0(\zeta), lab_0(\zeta'))$. Reading a column γ updates all components simultaneously, using the old values on the right-hand sides. If γ_a is absent, block a has not started and its components are unchanged. If $\gamma_a = z$, then

$$exit'_a(\zeta) = \delta(exit_a(\zeta), z), \qquad term'_a(\zeta, \zeta'') = term_a(\zeta, \zeta'') \vee \Lambda^{\text{fin}}\big((exit_a(\zeta), z), (\zeta'', \ell)\big).$$

Equality is updated to $eq(a, b) \wedge \gamma_a = \gamma_b$. For the admission bit, if $\gamma_a = z$ and $\gamma_b = z'$ are both present,

$$adm'(a, b, \zeta, \zeta') = \Lambda\big((exit_a(\zeta), z), (exit_b(\zeta'), z')\big) \;\vee\; \big(z = z' \wedge adm(a, b, \zeta, \zeta')\big).$$

If only $\gamma_a = z$ is present, set

$$adm'(a, b, \zeta, \zeta') = \Lambda\big((exit_a(\zeta), z), lab_0(\zeta')\big).$$

If only $\gamma_b = z'$ is present, set

$$adm'(a, b, \zeta, \zeta') = \Lambda\big(lab_0(\zeta), (exit_b(\zeta'), z')\big).$$

In these cases the absent track is still empty, so only the empty common suffix is possible and no old admission bit is retained. If both entries are absent, retain the old admission bit. The update is correct because Λ_ℓ compares positions at the *same distance from the ends* of the two blocks, with equal letters after them: appending a column either relates the two new last positions, or extends the common suffix of an earlier related pair by the letter $z = z'$. This is why the blocks are aligned at their right ends.

Shortening. A word of columns is *valid* if each track consists of absent entries followed by letters. Deleting the columns between two cuts with the same summary preserves validity, since a subsequence of a valid word is valid, and preserves the final summary. Hence every tuple of blocks can be replaced by one with the same summary and all lengths below $col(|Q|, h)$.

The summary determines the derived transition δ_ℓ, the target predicate T^{fin}_ℓ, both derived relations on the labels with these letters, and the equalities between the letters; that is, it determines the restriction of $\mathcal{M}_\ell$ to the letters in use up to renaming. Replacing the blocks therefore maps holes of $\mathcal{M}_\ell$ to holes. Since the replacement is injective on the letters in use and preserves marker avoidance, it keeps a unary level of the canonical hierarchy unary.

C.5 Agreement Under Derivation

For lettered monitors $\mathcal{U}, \mathcal{V}$ with the same state set, a finite letter support $\mathcal{E}$, and a map ρ from $\mathcal{E}$ into the alphabet of $\mathcal{V}$, write $\mathcal{U} \equiv_{\mathcal{E},\rho} \mathcal{V}$ for *agreement*: their start states coincide, ρ is injective on $\mathcal{E}$, and, for every $a \in \mathcal{E}$ and every state ζ,

$$\begin{aligned} \delta_{\mathcal{V}}(\zeta, \rho(a)) &= \delta_{\mathcal{U}}(\zeta, a), \\ T^{\mathrm{fin}}_{\mathcal{V}}(\zeta, \rho(a)) &\iff T^{\mathrm{fin}}_{\mathcal{U}}(\zeta, a), \\ \Lambda_{\mathcal{V}}(\widehat{\rho}L, \widehat{\rho}L') &\iff \Lambda_{\mathcal{U}}(L, L'), \\ \Lambda^{\mathrm{fin}}_{\mathcal{V}}(\widehat{\rho}L, \widehat{\rho}L') &\iff \Lambda^{\mathrm{fin}}_{\mathcal{U}}(L, L'). \end{aligned} \tag{55}$$

The last two lines range over all labels whose letters lie in $\mathcal{E}$, as well as the start label. Here $\widehat{\rho}(\triangleright) = \triangleright$ and $\widehat{\rho}(\zeta, a) = (\zeta, \rho(a))$; states are not renamed. Agreement preserves runs, labels up to $\widehat{\rho}$, and holes of words over $\mathcal{E}$. Indeed, induction gives the same state at every cut; injectivity preserves both equality and inequality of letters, hence every comparison; and (55) preserves the target and all admissions. Agreements compose: if $\mathcal{U} \equiv_{\mathcal{E},\rho} \mathcal{V}$ and $\mathcal{V} \equiv_{\rho(\mathcal{E}),\rho'} \mathcal{W}$, then $\mathcal{U} \equiv_{\mathcal{E},\rho'\circ\rho} \mathcal{W}$.

Lemma C.3 (Agreement under derivation). *Suppose $\mathcal{U} \equiv_{\mathcal{E},\rho} \mathcal{V}$ and $\ell \in \mathcal{E}$. For any finite family F of blocks over $\mathcal{E}$ avoiding ℓ, the letterwise map ρ^* gives*

$$\mathcal{U}_\ell \equiv_{F,\rho^*} \mathcal{V}_{\rho(\ell)}.$$

The image blocks avoid $\rho(\ell)$.

Proof. Injectivity on $\mathcal{E}$ implies injectivity on words over $\mathcal{E}$ and preserves marker avoidance. The shifted runs start in the same state: for an ordinary entry state, use agreement on the marker before the block; for $\star$, use equality of start states. Induction on the block gives equal exits and labels related by $\widehat{\rho}$ at every position. A common-suffix decomposition maps to one after renaming. Conversely, any common suffix of two renamed blocks pulls back to equal suffixes, by injectivity. Thus both directions of the derived Λ agree. The terminal-comparison test ranges over the same positions and the same exit states, so its truth is preserved by the last line of (55). This proves preservation of both the derived target and terminal relation. Relations involving the derivative start label are false in both monitors. □

Fix a base monitor $\mathcal{M}_0$ with N states and markers $\boldsymbol{\ell} = (\ell_0, \ldots, \ell_{j-1})$. They determine a tower $\mathcal{M}^{\boldsymbol{\ell}}_0 = \mathcal{M}_0$ and $\mathcal{M}^{\boldsymbol{\ell}}_{i+1} = (\mathcal{M}^{\boldsymbol{\ell}}_i)_{\ell_i}$. A finite family of level-j letters is *valid for the markers* if every block avoids its level's marker and its ordinary expansion, recursively, uses valid lower-level letters. Write $\mathrm{Exp}_{\boldsymbol{\ell},j}(a)$ for the level-zero word obtained by expanding a single level-j letter, with each marker placed after its block. At level zero this is the one-letter word a.

Lemma C.4 (Supported simultaneous compression). *Fix a nonempty word w, its original hierarchy and markers $\boldsymbol{\ell}$, and $j \le d$. Suppose $|\mathcal{A}_i| \le g$ for $1 \le i \le j$. Put $K = col(N + d + 1, g)$. For a*

finite set $\mathcal{E}_j \subseteq \mathcal{A}_j$, there are new markers $\widetilde{\boldsymbol{\ell}}$ and an injective renaming ρ_j on $\mathcal{E}_j$ such that

$$\mathcal{M}_j^{\boldsymbol{\ell}} \equiv_{\mathcal{E}_j,\rho_j} \mathcal{M}_j^{\widetilde{\boldsymbol{\ell}}}, \qquad |\operatorname{Exp}_{\widetilde{\ell},j}(\rho_j(a))| \le K^j \quad (a \in \mathcal{E}_j).$$

The image family is valid for the new markers. Every recursive support is a set contained in its original dictionary; at level zero there is no cardinality assumption. Agreement is at every state, not only states visited by the fixed word.

Proof. Induct on j, keeping w, its dictionaries, and d, g fixed. At zero, take the identity map and old markers. A letter has expansion size $1 = K^0$, regardless of the size of $\mathcal{E}_0$.

For the step from j to $j+1$, the set $\mathcal{E}_{j+1}$ has at most g elements. Let $K_j = col(N+j, g)$ and $\mathcal{U} = \mathcal{M}_j^{\boldsymbol{\ell}}$. The column construction, with empty padding tracks if necessary, replaces every $X \in \mathcal{E}_{j+1}$ by a subsequence $\tau(X)$ of length less than K_j. Equality bits make τ injective on $\mathcal{E}_{j+1}$; the other fields preserve transitions, target, and both admissions at every entry state. Thus

$$\mathcal{U}_{\ell_j} \equiv_{\mathcal{E}_{j+1},\tau} \mathcal{U}_{\ell_j}.$$

Form the set

$$\mathcal{E}_j = \{\ell_j\} \cup \bigcup_{X \in \mathcal{E}_{j+1}} \operatorname{alph}(\tau(X)).$$

The marker belongs to $\mathcal{A}_j$. Each $X \in \mathcal{A}_{j+1}$ is an original block body of u_j, so all its letters belong to $\mathcal{A}_j$. Since $\tau(X)$ is a subsequence of X, this proves $\mathcal{E}_j \subseteq \mathcal{A}_j$. Counting distinct letters ensures $|\mathcal{E}_j| \le g$ whenever $j \ge 1$. At $j = 0$ the identity base case applies without such a bound.

Apply the induction hypothesis to $\mathcal{E}_j$, obtaining lower markers, a monitor $\mathcal{V} = \mathcal{M}_j^{\widetilde{\boldsymbol{\ell}}}$, and ρ_j with agreement on $\mathcal{E}_j$ and expansion bound K^j. Set

$$\widetilde{\ell_j} = \rho_j(\ell_j), \qquad \rho_{j+1}(X) = \rho_j^*(\tau(X)).$$

Changing marker j does not affect the monitors through level j. By Lemma C.3, lower agreement lifts to agreement on the shortened blocks between $\mathcal{U}_{\ell_j}$ and $\mathcal{V}_{\widetilde{\ell_j}}$. Composition with column agreement proves the displayed invariant at level $j+1$.

Both the marker and all body letters lie in $\mathcal{E}_j$. Injectivity therefore preserves marker avoidance; validity at smaller levels follows from the induction hypothesis. Each body followed by its marker has at most K_j letters. Formula (47) is increasing in its positive state argument, so $K_j \le K$. Expanding these letters gives at most $K_j K^j \le K^{j+1}$ base letters. This proves agreement, validity, containment, and the expansion bound together. No dictionary or normality property of an intermediate compressed word was used. □

Proof of Theorem 8.1. Convert the reader to its lettered monitor, or start with any lettered monitor. Put the given nonempty hole in dictionary normal form. By Lemma 8.9, its positive dictionaries through its first unary level $\Delta \le d$ are bounded by $g = \mathsf{Dict}(N, d)$. Keep this word fixed as the original hierarchy for Lemma C.4, and put $K = col(N + d + 1, g)$. The unary argument gives a hole z^L of its level-Δ monitor with $1 \le L \le N + \Delta$. Compress $\mathcal{E}_\Delta = \{z\}$ using the lemma. Agreement makes $\rho_\Delta(z)^L$ a hole of the new level-Δ monitor. Validity allows exact expansion by Lemma 8.3, yielding a nonempty base hole. Cutting that expansion recovers the renamed unary word at level Δ, so its depth is at most Δ. Its length is bounded by

$$LK^\Delta \le (N + \Delta)K^\Delta \le (N + d + 1)K^d = \mathsf{Len}(N, d).$$

Here $K \ge 1$ since $N \ge 1$. This also proves the depth-zero case, where the expansion is the identity. □

C.6 Closed-Form Estimate

Lemma C.5 (Elementary estimate). *Put*

$$\Xi(N,d) = 3N^2 + 4N(N+d)(d+1) + 3N + 3d + 4.$$

Then $\mathsf{Dict}(N,d) \le 2^{\Xi(N,d)}$ *and* $\mathsf{Len}(N,d) \le 2^{2^{5\Xi(N,d)+3}}$. *For* $d_N = 2N(4N-1)-1$ *and* $N \ge 1$,

$$\mathsf{Len}(N,d_N) \le 2^{2^{1280(N+1)^5}}. \tag{56}$$

Proof. Write $\Xi = \Xi(N,d)$ and $\Xi_0 = 3N^2 + 4N(N+d)(d+1) + 3N$. The inequality $N \le 2^N$ gives $\mathsf{Ty}(N,d) \le 2^{\Xi_0}$. For nonnegative p, q, the sum of quantities bounded by $2^p, 2^q$ is at most 2^{p+q+1}. Since $d+1 \le 2^{d+1}$, we obtain $\mathsf{Al}(N,d) \le 2^{\Xi_0+d+2}$. Using $3^d \le 2^{2d}$ gives $\mathsf{Dict}(N,d) = 2+\mathsf{Al}(N,d)3^d \le 2^{\Xi_0+3d+3} \le 2^{\Xi}$.

Put $M = N + d + 1$, $g = \mathsf{Dict}(N,d)$, and $P = 2^{\Xi}$. We have $M + 1 \le \Xi$, so $M, M+1, d, g \le P$. Replacing M by 2^M in the first factor of (47) yields

$$col(M,g) \le 2^{2M(M+1)g+g^2+(M+1)^2g^2} \le 2^{4P^4} = 2^{2^{4\Xi+2}}.$$

Therefore

$$\mathsf{Len}(N,d) \le 2^{\Xi+d2^{4\Xi+2}} \le 2^{2^{5\Xi+3}},$$

because $d \le 2^{\Xi}$ and $\Xi \le 2^{\Xi} \le 2^{5\Xi+2}$.

For the specialization, $d_N = 8N^2 - 2N - 1$, so $d_N \le 8N^2$, $N + d_N \le 8N^2$, and $d_N + 1 \le 8N^2$. Consequently

$$4N(N+d_N)(d_N+1) \le 256N^5, \qquad \Xi(N,d_N) \le 256N^5 + 27N^2 + 3N + 4.$$

Hence $5\Xi(N,d_N)+3 \le 1280N^5+135N^2+15N+23 \le 1280(N+1)^5$, because $1280(N+1)^5 - 1280N^5 \ge 12800N^2 + 6400N + 1280$. Monotonicity of exponentiation proves (56). □

D Proofs of the Domain Transfers

D.1 Regular Solutions and Countermodels

For a covariant system χ on a variable set E of size k_χ, use graph states $(U,L) \in \mathcal{P}(E)^2$, with root state $(D_\chi{\uparrow}z, D_\chi{\downarrow}z)$ for z. Put

$$L_{z,\eta} = \{v : \mathsf{Lo}(\eta, v, z)\}, \qquad U_{z,\eta} = \{v : \mathsf{Up}(\eta, z, v)\}. \tag{57}$$

On letter i, replace L by $\{v : \exists u \in L\ \mathsf{Lo}(i,v,u)\}$ and U by $\{v : \exists u \in U\ \mathsf{Up}(i,u,v)\}$. Let F_- and F_+ be the roots of lower and upper constructor literals, and B_+ the bottom-literal roots. Output

$$g(U,L) = \begin{cases} f, & L \cap F_- \neq \varnothing \wedge U \cap F_+ \neq \varnothing, \\ \bot, & L \cap F_- = \varnothing \wedge U \cap (B_+ \cup F_+) \neq \varnothing, \\ \top, & \text{otherwise.} \end{cases}$$

Only constructor-labeled vertices have outgoing edges in the unfolding.

Theorem D.1 (Least-shape graph). *This finite graph unfolds at each root z to $\sigma_{\mathrm{sh}}(z)$. It is computable and has at most 2^{2k_χ} states, with no consistency hypothesis needed for the equality of trees.*

Proof. The path rules (Lemma 4.1) at $\eta \mid i$ show that the transition takes $(U_{z,\eta}, L_{z,\eta})$ to $(U_{z,\eta i}, L_{z,\eta i})$. For the lower coordinate a lower η-path from u to z composes with a lower i-path from v to u; for the upper coordinate paths compose in forward endpoint order. The converse follows by factoring those judgments. The initial sets retain full closure, so induction also covers the empty path. Intersecting these sets with the three literal-root sets gives exactly $\ell_f, u_f, u_\perp$ in (21). Thus the output agrees with h. A path exists in the unfolding iff all its proper-prefix outputs

are f, exactly the pruning predicate of the least shape. Hence labels and absent paths both agree. Closure, transitions, and output tests are finite computations on the 2^{2k_χ} set pairs. □

Theorem D.2 (Regular countermodel transfer). *Every unsafe word has a regular countermodel at the same word and side. Consequently* (51) *holds, and so does* (53).

Proof. Use the clash-free four-spine extension from Theorem 4.9. Its least shape has a finite graph G by Theorem D.1. Selection also has a finite graph: use states (r, s, b), initially $(r_z, r_{\iota z}, \mathrm{sg}(z))$, outputting the label at r when $b = 0$ and the dual of that at s when $b = 1$. On letter i, advance both graph coordinates and update b to $b \oplus c(i)$. Common shape from Lemma 4.5 ensures that retained coordinates are both constructors or both extreme; stop at the extreme outputs. Induction on retained paths gives (23), including its domain. At most $2|G|^2$ states suffice. Decoding H_0 uses another polarity bit, initially zero, to flip output labels. Pointwise tree duality flips labels and restriction just selects roots, so these operations also preserve graph finiteness. The fixed solution and four prefix conditions proved in Theorem 4.9 therefore give a regular countermodel after decoding, on either side. Conversely, a regular countermodel is an arbitrary one. Any arbitrary countermodel supplies an unsafe word by (11), proving (51). The same construction without spines gives a regular solution whenever a solution exists, which proves (53). □

D.2 Finite Satisfiability and Least-Shape Depth

We first work covariantly over k_χ variables, with the cycle clashes (50) of Section 9: a nonempty word η and variables a, b with $\mathsf{Lo}(\eta, a, a)$, $D_\chi(a, b)$, and $\mathsf{Up}(\eta, b, b)$.

Theorem D.3 (Covariant finite satisfiability). *Finite satisfiability is characterized by*

$$\mathrm{Sat}_{\mathrm{fin}}(\chi) \iff \chi \text{ has neither a label clash nor a cycle clash.} \tag{58}$$

Proof. *Necessity.* A label clash excludes every solution. For a cycle clash, (16) gives $\mathrm{emb}_\bot(\eta, \sigma(a)) \le \sigma(a) \le \sigma(b) \le \mathrm{emb}_\top(\eta, \sigma(b))$. Iterating the two self-bounds gives the same sandwich along η^h for every h. Along this path both bounding trees have constructor labels at every proper prefix; the middle trees must therefore also have constructors there. In particular, $\sigma(a)$ has a path of length $h|\eta|$ for every h, contradicting finiteness.
Sufficiency. By Theorem 4.3, absence of a label clash makes σ_{sh} a solution. We prove that absence of a cycle clash bounds its depth, independently of label consistency. If a domain path of $\sigma_{\mathrm{sh}}(z)$ had length greater than $k_\chi^2 + 1$, its prefix τ of length $k_\chi^2 + 1$ would have constructor label. By (21) choose a lower and an upper path along τ from z, with endpoints supporting their constructor labels. Factor both paths at every cut, giving variables a_i, b_i, $0 \le i \le |\tau|$, with

$$\mathsf{Lo}(\tau[:i], a_i, z), \qquad \mathsf{Up}(\tau[:i], z, b_i), \qquad D_\chi(a_i, b_i). \tag{59}$$

The last relation is cancellation (18). To obtain simultaneous factorizations with exact endpoints, induct on the positive path length: factor its first letter, continue from the resulting variable, and absorb the closure at each cut by transitivity. For length one retain the given endpoints. This gives $a_0 = b_0 = z$ and the chosen terminal supports, with $\mathsf{Lo}(\tau[i:j], a_j, a_i)$ and $\mathsf{Up}(\tau[i:j], b_i, b_j)$ for each $i < j$, by composition. At $i = j$ use reflexivity. Thus no nontrivial empty-path closure is being replaced by equality. Among the first $k_\chi^2 + 1$ pairs, two agree, at cuts $i < j$. The nonempty factor $\tau[i:j]$ then satisfies (50) at (a_i, b_i), a contradiction. Therefore all domains of σ_{sh} have depth at most $k_\chi^2 + 1$, and finite branching makes them finite. This proves (58).

For an effective cycle test, take vertices (a, b) with $D_\chi(a, b)$ and an i-edge to (a', b') when $\mathsf{Lo}(i, a', a)$ and $\mathsf{Up}(i, b, b')$. Cancellation puts the target in the vertex set. A nonempty closed walk

composes to the two self-paths in (50). Conversely, factor those self-paths at every letter with their exact endpoints; at each cut, transport their prefixes across $D_\chi(a, b)$ and cancel to obtain a vertex pair. These pairs give a nonempty closed walk. Thus cycle clash is exactly directed cyclicity in this finite graph. A simple cycle has at most k_χ^2 edges; equivalently it suffices to test words of lengths 1 through k_χ^2. At arity zero, there are no edges. □

Lemma D.4 (Finite signed correspondence). *For variance, one still has the equivalence*

$$\mathrm{Sat}_{\mathrm{fin},c}(\phi) \quad \Longleftrightarrow \quad \mathrm{Sat}_{\mathrm{fin}}(\phi^{\pm}). \tag{60}$$

Proof. Normalization preserves domains and proves the forward implication. Conversely, a finite solution of ψ excludes both clashes by Theorem D.3. Its least shape has depth at most $(2k)^2 + 1$. Selection by Lemma 4.5 preserves that domain and gives a finite fixed solution. Decoding by Lemma 3.1 again preserves domains, proving the reverse implication. Together with the two finite clash tests this decides finite satisfiability for every variance. □

D.3 Finite Countermodel Transfer

Theorem D.5 (Finite countermodel transfer). *Suppose ϕ has a finite variance solution. Every arbitrary-tree countermodel to $x \le y$ can then be replaced by a finite countermodel at any chosen unsafe word and side of the original countermodel. This proves (52).*

Proof. *Spine extensions add no cycle clash.* This assertion holds for the four-spine extension (26), without a label-consistency hypothesis. Assign old variables rank zero, spine variables with index $j > 0$ rank j, and fillers rank $|w| + 1$. Every new lower-constructor literal has source children of strictly larger rank than its root; every new upper-constructor literal has sink children of strictly larger rank. The separation proof (28) still applies.

Induction on (15) shows that a lower path rooted at a source ends at a source of at least its rank, strictly larger for a nonempty path. At length zero separation forces identity. At the first letter, the closure into the source forces the lower-literal root to be that source. Its literal is new and its chosen child is a source of larger rank, so induction applies to the remaining path. An upper path rooted at a sink satisfies the dual assertion, using closure out of a sink. Neither kind can have a nonempty self-cycle there. In a cycle clash (50), a cannot be a source and b cannot be a sink. If a were a sink, $D(a, b)$ would force $a = b$, impossible for the upper self-cycle; if b were a source, the same argument contradicts the lower self-cycle. Thus both are old. A path between old endpoints cannot use a new constructor literal: its child would enter a source or sink and could not return to an old endpoint. Closure and both path judgments in (50) therefore reflect to the old system. Conversely, every old cycle remains. This proves cycle-clash equivalence for the extension.

The selected witness is finite. Choose an unsafe word w from the arbitrary countermodel. By Theorem 4.9, its existing four-spine extension has no label clash. Normalizing the assumed finite solution, and dualizing it on the right side, gives a finite solution of the side system, so that system has no cycle clash. The rank argument above excludes cycle clashes in the extension. Its least shape therefore has depth at most $(2k + 4|w| + 4)^2 + 1$ by the depth proof of Theorem D.3. The selector keeps precisely this finite domain. It satisfies the whole extension and is fixed, so restriction supplies all four conditions (27); decoding gives a finite countermodel at the same word. For the right side dualize the side solution back before decoding. Duality preserves both domains and fixedness.

The assumed finite solution is used only to exclude cycles in the base constraints. The spine extension preserves this exclusion, so the countermodel is reconstructed independently as a finite

least-shape solution. If no finite solution exists, finite entailment is vacuous. Otherwise, the construction just proved takes every failure of arbitrary entailment to a finite countermodel; conversely arbitrary entailment holds in every finite assignment. These observations prove (52). □

D.4 Countermodel Bounds

For a chosen unsafe word w, put $k_\chi = 2k + 4|w| + 4$. The regular construction uses at most 2^{2k_χ} least-shape graph states, $2^{4k_\chi+1}$ selected graph states, and $2^{4k_\chi+2}$ states after decoding. Under finite satisfiability, the finite construction has tree depth at most $k_\chi^2 + 1$, not a bound of that size on the total number of nodes. The finite search supplies $|w| \le H$ with H from (49), giving effective bounds in the input parameters.

E Proofs for the Upper Bounds

We prove Lemma 10.1, Proposition 10.2, and Theorem 10.3.

E.1 Polynomial-Time Tests

Proof of Lemma 10.1. The table of Section 3.1 gives ψ with $2k$ variables and two literals for each literal of ϕ; the order dual reverses its inequalities and exchanges its constants. The closure is computed by adding reflexive, transitive, and decomposition consequences (14) until none is new. At most $(2k)^2$ pairs are added, and each round inspects polynomially many pairs of literals and closure pairs. By (15) with an empty remainder, $\mathrm{Up}(i,u,v)$ holds iff $D_\psi(u,z)$, $z \le f(\vec{b}) \in \psi$, and $D_\psi(b_i,v)$ for some $z,\vec{b}$, and dually for Lo. Hence the updates E_i, G_i of (35), the tests of (32), the initial state and transitions (36), rejection (38), and admission (39) are operations on subsets of the $2k$ signed variables, computed from the closure in polynomial time. A label clash is a lower and an upper literal with incompatible root labels whose roots are related by the closure (Section 4.2). A cycle clash (50) is a directed cycle in the graph on the at most $(2k)^2$ closure pairs used in the proof of Theorem D.3; its edges need only single-letter path judgments, so cyclicity is decided in polynomial time. Finally, $\mathrm{Sat}_c(\phi)$ and $\mathrm{Sat}_{\mathrm{reg},c}(\phi)$ hold iff ψ has no label clash ((24), Theorem 4.3, and (53)), and $\mathrm{Sat}_{\mathrm{fin},c}(\phi)$ holds iff ψ has neither a label clash nor a cycle clash (Lemma D.4 and Theorem D.3). □

E.2 Summaries Determine Derived Monitors

The proof of Proposition 10.2 uses the lettered monitors and derivatives of Section 8, and the summaries, column words, and agreement of Appendix C. Throughout, $\mathcal{M}$ is a lettered monitor with state set Q and start state ζ_0, ℓ is one of its letters, and $[h] = \{1,\dots,h\}$.

The summary of blocks $X_1,\dots,X_h$ avoiding ℓ, with respect to $\mathcal{M}$ and ℓ, is the tuple $\Theta = (exit, term, eq, adm)$ of Appendix C, which the column automaton described there computes along the column word of the tuple. A column whose entries are all absent changes no component of a summary. Every valid column word whose tracks spell $X_1,\dots,X_h$ is the column word of the tuple preceded by such columns, so the automaton computes Θ along every such word.

Lemma E.1 (Summary monitor). *Let $\Theta = (exit, term, eq, adm)$ be the summary of pairwise distinct blocks $X_1,\dots,X_h$ avoiding ℓ. Let $\mathcal{M}[\ell,\Theta]$ be the lettered monitor over $[h]$ with state set $Q \sqcup \{\star\}$, start state $\star$, and, for $a,b \in [h]$ and $\zeta,\zeta' \in Q \sqcup \{\star\}$,*

$$\begin{aligned} \delta(\zeta,a) &= exit_a(\zeta), \\ T^{\mathrm{fin}}(\zeta,a) &\iff T^{\mathrm{fin}}_{\mathcal{M}}(exit_a(\zeta),\ell) \wedge \neg\, term_a(\zeta, exit_a(\zeta)), \\ \Lambda((\zeta,a),(\zeta',b)) &\iff adm(a,b,\zeta,\zeta'), \\ \Lambda^{\mathrm{fin}}((\zeta,a),(\zeta',b)) &\iff adm(a,b,\zeta,\zeta') \vee term_a(\zeta, exit_b(\zeta')), \end{aligned} \tag{61}$$

where $T^{\mathrm{fin}}_{\mathcal{M}}$ is the target of $\mathcal{M}$; every relation involving the label $\triangleright$ is false. Then

$$\mathcal{M}[\ell, \Theta] \equiv_{[h],X} \mathcal{M}_\ell,$$

where X denotes the map $a \mapsto X_a$. In particular, Θ and the targets $T^{\mathrm{fin}}_{\mathcal{M}}(\cdot, \ell)$ determine the restriction of the derivative $\mathcal{M}_\ell$ to the letters $X_1, \dots, X_h$, up to the names of these letters.

Proof. The right-hand sides are defined: an exit state is a state of Q, also from the entry state $\star$, and $term_a(\zeta, \zeta'')$ is recorded for every $\zeta'' \in Q$. Both monitors have the state set $Q \sqcup \{\star\}$ and the start state $\star$, and X is injective because the blocks are distinct. We compare (61) with Definition 8.2 at the letters X_a, X_b and the states ζ, ζ'. The derived transition $\delta_\ell(\zeta, X_a)$ is the exit state of the shifted block of X_a read from ζ, that is, $exit_a(\zeta)$. The bit $term_a(\zeta, \zeta'')$ states that $\Lambda^{\mathrm{fin}}(lab_p(\zeta, X_a), (\zeta'', \ell))$ holds for some $p \le |X_a|$. For $\zeta'' = \delta_\ell(\zeta, X_a)$ this is the condition excluded by $T^{\mathrm{fin}}_\ell(\zeta, X_a)$, whose other condition is $T^{\mathrm{fin}}_{\mathcal{M}}(\delta_\ell(\zeta, X_a), \ell)$; this proves the second line. For $\zeta'' = \delta_\ell(\zeta', X_b)$ it is the second disjunct of $\Lambda^{\mathrm{fin}}_\ell((\zeta, X_a), (\zeta', X_b))$. By definition, $adm(a, b, \zeta, \zeta')$ states $\Lambda_\ell((\zeta, X_a), (\zeta', X_b))$, which is also the first disjunct of $\Lambda^{\mathrm{fin}}_\ell$. This proves the third and fourth lines. Finally, every relation involving the start label $\triangleright$ of $\mathcal{M}_\ell$ is false by Definition 8.2. These are all the conditions of (55) for the letters of $[h]$ and the labels over them. □

Lemma E.2 (Summaries under agreement). *Let $\mathcal{U} \equiv_{E,\rho} \mathcal{V}$ with $\ell \in E$, and let $X_1, \dots, X_h$ be blocks over E avoiding ℓ. Then the summary of $X_1, \dots, X_h$ with respect to $\mathcal{U}$ and ℓ equals the summary of $\rho^*(X_1), \dots, \rho^*(X_h)$ with respect to $\mathcal{V}$ and $\rho(\ell)$.*

Proof. The image blocks avoid $\rho(\ell)$, since ρ is injective on E, and the column word of the image tuple is the letterwise image of the column word of $X_1, \dots, X_h$, with absent entries kept absent. We show by induction on the columns that the two column automata, for $\mathcal{U}, \ell$ and for $\mathcal{V}, \rho(\ell)$, reach equal summaries. Both monitors have the same states and the same start state, and summaries take values in states and bits only. In the initial summary, $exit_a(\zeta) = \delta(\zeta, \ell)$ agrees by the first line of (55), $exit_a(\star)$ is the common start state, and the *term* and *adm* bits evaluate Λ^{fin} and Λ on the labels $lab_0(\zeta)$ and (ζ'', ℓ). The map $\widehat{\rho}$ sends these labels to the corresponding labels for the marker $\rho(\ell)$, so the two lines of (55) for the relations give equal bits. An update evaluates δ at a letter of E, Λ and Λ^{fin} on labels whose letters lie in E, and the equality of two entries of the column. Agreement preserves the first two, and injectivity of ρ on E preserves equality and inequality of letters. Hence the updated summaries are equal. □

E.3 The Search

We restate the search of Figure 13 with the details that the space bound needs. Fix $\mathcal{M}_0$, with state set Q of size $N \ge 1$, start state ζ_0, and finite alphabet Γ, and fix $d \ge 0$. Put

$$g = \mathrm{Dict}(N, d), \qquad \bar{N} = N + d + 1, \qquad \bar{g} = g + |\Gamma|;$$

thus $\bar{N}$ bounds the number of states, and $\bar{g}$ the number of letters, of every monitor that the search stores. The search keeps a level $j \le d$ and a *current monitor* $\mathcal{W}$: a lettered monitor with state set $Q \sqcup \{\star_1, \dots, \star_j\}$, of size $N + j$, over a *current alphabet* $\Gamma_{\mathcal{W}}$, which is Γ for $j = 0$ and $[h]$ for some $h \le g$ for $j \ge 1$. Initially $j = 0$ and $\mathcal{W} = \mathcal{M}_0$. The search repeats one of two moves.

- *Accept.* Choose $z \in \Gamma_{\mathcal{W}}$ and $1 \le m' \le N + j$, and accept if $z^{m'}$ is a hole of $\mathcal{W}$. Every pair of cuts of a unary word is a comparison, so the test checks the target at the last label, Λ^{fin} between the label of every cut $s < m'$ and the last label, and Λ between the labels of every two cuts $s < e < m'$ (Appendix C, paragraph on unary words). The labels are computed along the run of $\mathcal{W}$ on $z^{m'}$.
- *Derive*, if $j < d$. Choose a marker $\ell \in \Gamma_{\mathcal{W}}$ and a number $1 \le h \le g$. Guess a column word with h tracks over $\Gamma_{\mathcal{W}} \setminus \{\ell\}$, one column at a time. Keep one flag per track recording whether the track

has started, and refuse a column with an absent entry on a started track, so that the word is valid. Count the columns with a binary counter and stop before $col(N + j, h)$ columns. Maintain the summary of the tracks read so far by the column automaton of Appendix C for $\mathcal{W}$ and ℓ, starting from the summary of h empty blocks. At the end, with final summary Θ, require $eq(a, b)$ to be false for all $a \neq b$. Then replace $\mathcal{W}$ by $\mathcal{W}[\ell, \Theta]$ of Lemma E.1, naming its new state $\star_{j+1}$, and replace j by $j + 1$.

The tracks of the guessed column word spell blocks $Y_1, \ldots, Y_h$ over $\Gamma_{\mathcal{W}}$ that avoid ℓ. Their summary is the final summary Θ, and the final test makes them pairwise distinct.

Space. The current monitor has $N + j < \bar{N}$ states and at most $\bar{g}$ letters. Write each state with $\lceil \log_2 \bar{N} \rceil \leq \bar{N}$ bits and each letter, or absent entry, with $\lceil \log_2(\bar{g} + 1) \rceil \leq \bar{g}$ bits. The search stores the following items.

- The current monitor: its transitions, targets, and the relations Λ and Λ^{fin} on its at most $1 + \bar{N}\bar{g}$ labels, in at most $\bar{N}^2\bar{g} + \bar{N}\bar{g} + 2(1 + \bar{N}\bar{g})^2 \leq 10\bar{N}^2\bar{g}^2$ bits.
- During a derive move, the next monitor $\mathcal{W}[\ell, \Theta]$, with $N + j + 1 \leq \bar{N}$ states and $h \leq \bar{g}$ letters, in at most $10\bar{N}^2\bar{g}^2$ bits.
- One summary, for $h \leq \bar{g}$ blocks, $N + j + 1 \leq \bar{N}$ entry states, and $N + j < \bar{N}$ states: *exit* uses at most $h\bar{N} \cdot \bar{N}$ bits, *term* at most $h\bar{N}^2$, *eq* at most h^2, and *adm* at most $h^2\bar{N}^2$, in total at most $4\bar{N}^2\bar{g}^2$ bits.
- The column counter. By (47), which is nondecreasing in both arguments, it counts below $col(\bar{N}, g)$, and

$$\log_2 col(\bar{N}, h) = (\bar{N} + 1)h \log_2 \bar{N} + \bar{N}(\bar{N} + 1)h + h^2 + (\bar{N} + 1)^2h^2 \leq 4(\bar{N} + 1)^2h^2,$$

  using $\log_2 \bar{N} \leq \bar{N}$. The counter therefore has at most $1 + 16\bar{N}^2g^2 \leq 17\bar{N}^2\bar{g}^2$ bits.
- The current column and the track flags, in at most $h(\bar{g} + 1) \leq 2\bar{g}^2$ bits.
- The level j, the choices ℓ, h, z, m', and, for the accept test, two cuts, the states at three cuts, and a running state with its counter, recomputed from the start state: at most $9\bar{N} + 3\bar{g} \leq 12\bar{N}^2\bar{g}^2$ bits.

These items take at most $55\bar{N}^2\bar{g}^2$ bits. The summary is updated in place, column by column. Of the old summary, an update of a field *term*, *eq*, or *adm* reads only its own old bit and the old exit states, and an update of a field *exit* reads only its own old value. So the search first updates the fields *term*, *eq*, and *adm*, retaining the old exit states, and then the fields *exit*. Alternatively, a second summary buffer of the same size holds the new values. The remaining work space consists of the position of the input head, the counters and indices that address entries of the stored tables, the binary arithmetic that computes $col(N + j, h)$ and compares the column counter with it, and the evaluation of single entries of the update formulas and of (61). Every number involved is below $col(\bar{N}, g)$ or below the size of the input or of a stored table, so this work space takes $O(\bar{N}^2\bar{g}^2)$ bits. Hence the search uses at most $\mathrm{Sp} = c_{\mathrm{Sp}}\bar{N}^2\bar{g}^2$ bits, for an absolute constant c_{Sp}; this is polynomial in $\bar{N}$, g, and $|\Gamma|$. All bounds above use only the column bound (47), the dictionary bound g, and the unary bound $m' \leq N + j$.

Soundness. We show that whenever the search is at level j with current monitor $\mathcal{W}$, there are markers $\widetilde{\ell} = (\widetilde{\ell}_0, \ldots, \widetilde{\ell}_{j-1})$ and a map ρ_j from $\Gamma_{\mathcal{W}}$ to letters of level j, injective with an image family valid for $\widetilde{\ell}$, such that

$$\mathcal{W} \equiv_{\Gamma_{\mathcal{W}}, \rho_j} \mathcal{M}_j^{\widetilde{\ell}},$$

in the notation of Appendix C for towers. Initially ρ_0 is the identity on Γ and the marker sequence is empty.

For a derive move, let $Y_1, \ldots, Y_h$ be the spelled blocks and Θ their summary. By Lemma E.1, $\mathcal{W}[\ell, \Theta] \equiv_{[h], Y} \mathcal{W}_\ell$, where Y is the map $a \mapsto Y_a$. By Lemma C.3, applied to the family $\{Y_1, \ldots, Y_h\}$

of blocks over $\Gamma_{\mathcal{W}}$ avoiding $\ell \in \Gamma_{\mathcal{W}}$,

$$\mathcal{W}_\ell \equiv_{\{Y_a\},\rho_j^*} \left(\mathcal{M}_j^{\widetilde{\ell}}\right)_{\rho_j(\ell)},$$

and the image blocks avoid $\rho_j(\ell)$. Put $\widetilde{\ell}_j = \rho_j(\ell)$ and $\rho_{j+1}(a) = \rho_j^*(Y_a)$. Composing the two agreements gives the invariant at level $j+1$; appending a marker does not change the lower monitors of the tower. The map ρ_{j+1} is injective because the blocks are distinct and ρ_j^* is injective on words over $\Gamma_{\mathcal{W}}$. Each image block avoids $\widetilde{\ell}_j$ and consists of valid level-j letters, so the image family is valid.

If the search accepts with $z^{m'}$, agreement preserves holes of words over $\Gamma_{\mathcal{W}}$, so $\rho_j(z)^{m'}$ is a hole of $\mathcal{M}_j^{\widetilde{\ell}}$. Expand it along $\widetilde{\ell}_{j-1}, \ldots, \widetilde{\ell}_0$. At each level the word is a nonempty word of blocks avoiding that level's marker, by validity, so Lemma 8.3 makes its expansion a hole of the monitor one level below. This yields a nonempty hole w of $\mathcal{M}_0$. Each expansion ends with its marker, and its blocks avoid that marker. As in the proof of Theorem 8.1, cutting w at its successive markers recovers the expanded words, so the canonical hierarchy of w reaches the unary word $\rho_j(z)^{m'}$ at level j, and $\mathrm{depth}(w) \le j \le d$.

Completeness. Suppose that $\mathcal{M}_0$ has a nonempty hole of depth at most d. By Lemma 8.9, it has a nonempty hole w of some depth $\Delta \le d$ whose dictionaries satisfy $g_i \le g$ for $1 \le i \le \Delta$. Let u_i, ℓ_i, $\mathcal{A}_i$, and $\mathcal{M}_i$ be its hierarchy, markers, dictionaries, and derived monitors, so that $\mathcal{M}_{i+1} = (\mathcal{M}_i)_{\ell_i}$ and u_i is a hole of $\mathcal{M}_i$ (Lemma 8.3). We follow a run that reaches, at each level $j \le \Delta$, a current monitor $\mathcal{W}_j$ and a map π_j with

$$\mathcal{W}_j \equiv_{\Gamma_{\mathcal{W}_j},\pi_j} \mathcal{M}_j, \qquad \mathcal{A}_j \subseteq \pi_j(\Gamma_{\mathcal{W}_j}).$$

At $j = 0$ take $\mathcal{W}_0 = \mathcal{M}_0$ and π_0 the identity on Γ.

Let $j < \Delta$, so a derive move is allowed. The $h = g_{j+1} \le g$ letters $X_1, \ldots, X_h$ of $\mathcal{A}_{j+1}$ are distinct blocks of u_j; their letters lie in $\mathcal{A}_j$, and they avoid $\ell_j \in \mathcal{A}_j$. Choose $\ell = \pi_j^{-1}(\ell_j)$ and $Y'_a = (\pi_j^{-1})^*(X_a)$, which avoid ℓ. By Lemma E.2, the summary Θ of $Y'_1, \ldots, Y'_h$ with respect to $\mathcal{W}_j$ and ℓ equals that of $X_1, \ldots, X_h$ with respect to $\mathcal{M}_j$ and ℓ_j; in particular, its equality fields declare the blocks pairwise distinct. The current monitor has $N + j$ states, so by column shortening (Appendix C, paragraph on shortening), some valid column word with fewer than $col(N + j, h)$ columns has the same final summary Θ. The search chooses ℓ and h, guesses this column word, and passes the final test. Its new monitor $\mathcal{W}_{j+1} = \mathcal{W}_j[\ell, \Theta]$ depends only on Θ. Since Θ is also the summary of the distinct blocks Y'_a, Lemma E.1 gives $\mathcal{W}_{j+1} \equiv_{[h],Y'} (\mathcal{W}_j)_\ell$, where Y' is the map $a \mapsto Y'_a$. Lemma C.3 gives $(\mathcal{W}_j)_\ell \equiv_{\{Y'_a\},\pi_j^*} (\mathcal{M}_j)_{\ell_j} = \mathcal{M}_{j+1}$. Composing, $\mathcal{W}_{j+1} \equiv_{[h],\pi_{j+1}} \mathcal{M}_{j+1}$ for $\pi_{j+1}(a) = \pi_j^*(Y'_a) = X_a$, a bijection onto $\mathcal{A}_{j+1}$.

At level Δ, the word $u_\Delta = z^L$ is a unary hole of $\mathcal{M}_\Delta$, which has $N + \Delta$ states. By unary shortening (Appendix C, paragraph on unary words), $z^{m'}$ is a hole of $\mathcal{M}_\Delta$ for some $1 \le m' \le N + \Delta$. Write $z = \pi_\Delta(z')$. By agreement, $z'^{m'}$ is a hole of $\mathcal{W}_\Delta$, and the search accepts. For $\Delta = 0$ this is the accept move at level zero.

Deterministic Space. The configurations of the search consist of the items counted above, a control state, and the input head position. The bound $\mathrm{Sp} = c_{\mathrm{Sp}} \bar{N}^2 \bar{g}^2$ is at least the logarithm of the input size, and it can be computed from N, d, and $|\Gamma|$ in space $O(\mathrm{Sp})$. Savitch's theorem [Savitch 1970] simulates a nondeterministic machine with a space bound of this kind, at least logarithmic and constructible, by a deterministic machine in the square of that space. Hence the search runs in deterministic space $O(\mathrm{Sp}^2) = O(\bar{N}^4 \bar{g}^4)$. Since $g \ge 1$, we have $\bar{g} \le g(1 + |\Gamma|)$, and $g \le 2^{\Xi(N,d)}$ by Lemma C.5. Moreover,

$$\Xi(N, d) = 3N^2 + 4N(N + d)(d + 1) + 3N + 3d + 4 \le 3\bar{N}^2 + 4\bar{N}^3 + 3\bar{N} + 1 \le 11\bar{N}^3,$$

and $\bar{N}^4 \le 2^{4\bar{N}}$. Hence

$$\bar{N}^4 \bar{g}^4 \le 2^{4\bar{N}+44\bar{N}^3}(1+|\Gamma|)^4 \le 2^{48\bar{N}^3}(1+|\Gamma|)^4,$$

which proves the deterministic bound of Proposition 10.2. The nondeterministic bound is the count Sp above, and soundness and completeness show that the search accepts iff $\mathcal{M}_0$ has a nonempty hole of depth at most d. □

E.4 Accounting for the Upper Bounds

Proof of Theorem 10.3. We account for each part of the procedure.

Satisfiability and transfers. By Lemma 10.1, decide in polynomial time whether ϕ is satisfiable in $\mathcal{D}$. If it is not, the entailment holds. Otherwise ϕ has a solution in arbitrary trees, and by (51) and (52) entailment in $\mathcal{D}$ coincides with entailment in arbitrary trees. By (11) and Theorem 5.4, the latter holds iff neither label reader has a hole.

Reader and monitor tables. Each label reader has $N = 2^{4k+1}$ states over $\mathbb{D}$, with $|\mathbb{D}| = n$. Its transitions, rejected states, and admitted pairs fill $Nn(4k+1) + N + N^2$ bits, and each entry is computed in polynomial time by Lemma 10.1. The lettered monitor of the reader (Section 8) has the same transitions, the targets $T^{\mathrm{fin}}(\zeta, a) \iff \delta(\zeta, a) \in \mathrm{Rej}$, and $\Lambda = \Lambda^{\mathrm{fin}}$ given by Adm on the denoted states; it has $1 + Nn$ labels. All these tables are written in space $2^{O(k)}\,\mathrm{poly}(\|\phi\|)$.

The empty word. The empty word has no comparison, so it is a hole iff the initial state is rejected; this is tested directly.

The depth bound. As in the proof of Theorem 9.1, Theorems 6.1 and 7.7 show that a reader with the rejected-path property has a hole iff the empty word is a hole or the reader has a nonempty hole of depth at most $d = 8N^2 - 2N - 1$. Both label readers have this property (Lemma 5.3), and their nonempty holes are those of their lettered monitors.

The summary search. For each side, Proposition 10.2, with these N and d and with $|\Gamma| = n$, decides the existence of such a nonempty hole in deterministic space $2^{O((N+d+1)^3)}(1+n)^4$. Here $N + d + 1 = 8N^2 - N \le 2^{8k+5}$, so $(N+d+1)^3 \le 2^{24k+15}$, and the space is $2^{2^{O(k)}}(1+n)^4$.

Total. The stages run one after another, and the space of the whole procedure is at most the sum of their bounds, which is $2^{2^{O(k)}}\,\mathrm{poly}(\|\phi\|)$. This lies in 2-EXPSPACE because $k \le \|\phi\|$. A deterministic machine that uses space $s \ge \log\|\phi\|$ has at most $2^{O(s)}$ configurations, which gives deterministic time $2^{2^{2^{O(\|\phi\|)}}}$. For the general syntax at positive arity, Proposition 2.2 produces, in polynomial time, a flat system and a variable query with the same entailment in each domain: it adds a constant number of variables per subexpression and per encoded inequality, and $O(n)$ symbols per encoding. The flat system has $O(\|\phi\|)$ variables, so the space is $2^{2^{O(\|\phi\|)}}$. At arity zero, Proposition F.3 gives polynomial time. □

F Constructors of Arity at Most One

Section 10 concentrates on constructors of arity $n \ge 2$. Its upper bound, Theorem 10.3, also covers $n \le 1$, but stronger bounds hold there: entailment is in PSPACE at arity one (Theorem F.2) and in polynomial time at arity zero (Proposition F.3), in each of the three domains. We use the input size $\|\phi\|$ and the polynomial-time tests of Lemma 10.1.

F.1 Arity One

For $n = 1$ the alphabet is $\mathbb{D} = \{0\}$. Every word is a power 0^L, every pair of cuts $s < e \le L$ is a comparison, because $0^{L-e} \preceq 0^{L-s}$, and every nonempty word is unary and so has depth zero. Theorem 8.1 with $d = 0$ gives the length bound $\mathrm{Len}(N, 0) = N + 1$ of (48). The following lemma improves it and needs no rejected-path property.

Lemma F.1 (One-letter readers). *Let $(\mathcal{M}, \mathrm{Adm}, \mathrm{Rej})$ be a reader with N states over a one-letter alphabet, and let w be a hole. Then $w[:J(w)]$ is a hole and $J(w) \le N-1$. In particular, a reader of this kind that has a hole has a hole of length at most $N-1$.*

Proof. Let $J = J(w)$; it exists because w is rejected. The prefix $w[:J]$ ends in the rejected state q_J. Its comparisons are the pairs $s < e \le J$, which are comparisons of w with the same cut states; none of them is admitted, so $w[:J]$ is a hole. Suppose $q_i = q_j$ for some $i < j \le J$. With one letter, $q_{i+t} = q_{j+t}$ for every $t \ge 0$. For $t = J - j$ this gives $q_{J-(j-i)} = q_J \in \mathrm{Rej}$ with $J - (j-i) < J$, contradicting the minimality of J. Hence $q_0, \dots, q_J$ are pairwise distinct, and $J + 1 \le N$. □

The bound is attained among general readers: take states $0, \dots, N-1$, let the letter map i to $\min(i+1, N-1)$, reject only $N-1$, and admit no pair. Its only holes are the words of length at least $N-1$.

The lemma also gives an algorithm that tests a single word per reader. All words 0^L follow one run $q_0, q_1, \dots$, so every hole has the same first rejected cut J, the first cut of this run with a rejected state. Follow the run to J; if none of $q_0, \dots, q_{N-1}$ is rejected, there is no hole. Otherwise a hole exists iff 0^J is a hole.

Theorem F.2 (Arity one). *Let $n = 1$, with either variance, and let $\mathcal{D}$ be any of the three domains. If ϕ is satisfiable in $\mathcal{D}$ and $\phi \not\models_{\mathcal{D},c} x \le y$, then some unsafe word has length at most $2^{4k+1} - 1$. Entailment at arity one is decidable in polynomial space and in deterministic time $2^{O(k)}\,\mathrm{poly}(\|\phi\|)$. In particular, it is in PSPACE in each domain, also for the general input syntax of Proposition 2.2.*

Proof. By Lemma 10.1, satisfiability in $\mathcal{D}$ is decided in polynomial time; if it fails, the entailment holds. Otherwise, by (51), (52), (11), and Theorem 5.4, the entailment fails iff some label reader has a hole, and every hole is an unsafe word. By Lemma F.1, a label reader with a hole has one of length at most $N - 1 = 2^{4k+1} - 1$, which proves the first claim.

For the algorithm, consider each label reader and its run $q_0, q_1, \dots$ on the powers of 0. A state of (36), two sets of signed variables and a latch bit, takes $4k+1$ bits, and a counter of $4k+1$ bits counts up to $N-1$. Follow the run for at most $N-1$ steps, keeping only the current state and the counter. If none of $q_0, \dots, q_{N-1}$ is rejected, the reader has no hole, since a hole w would have $q_{J(w)} \in \mathrm{Rej}$ with $J(w) \le N-1$ by Lemma F.1. Otherwise let J be the first rejected cut of the run. Every hole w is a power of 0 with $J(w) = J$, so by Lemma F.1 the reader has a hole iff 0^J is a hole, that is, iff $\neg\mathrm{Adm}(q_s, q_e)$ for all $s < e \le J$. The pairs $s < e$ are enumerated by two counters, and the states q_s and q_e are recomputed from q_0. Each transition, each rejection test, and each admission test takes polynomial time by Lemma 10.1. Only the closure, a constant number of states and counters, and the work space of these tests are stored, so the space is polynomial in $\|\phi\|$. The procedure performs $O(N^3)$ transitions and tests, which gives the time bound. For the general syntax at positive arity, Proposition 2.2 is a polynomial-time reduction to a flat system with $O(\|\phi\|)$ variables, so entailment remains in PSPACE. □

F.2 Arity Zero

For $n = 0$ the reader search of Figure 12 examines only the empty word, and general systems cannot be reduced to our restricted flat syntax by the positive-arity encoding of Proposition 2.2. Both cases are nevertheless easy.

Proposition F.3 (Arity zero). *Let $n = 0$. The three domains coincide, and entailment is decidable in polynomial time, both for flat systems and for general systems of inequalities between variables and the constants $\bot, f(), \top$, with a query between two such expressions.*

Proof. At arity zero every tree is $\bot$, $f()$, or $\top$ (Section 2.1), so the three domains coincide, and by (3) the order is the chain $\bot < f() < \top$. For a flat system, the procedure of Figure 12 computes the closure, tests for a label clash, and then tests the empty word, that is, whether the initial state of either reader is rejected. All of this takes polynomial time by Lemma 10.1. For a general system ϕ and query $t \le t'$, the query fails under an assignment σ iff $\sigma(t) > \sigma(t')$. For each of the three pairs of constants $a > b$, test whether ϕ together with $t \le a$, $a \le t$, $t' \le b$, and $b \le t'$ is satisfiable; entailment holds iff all three tests fail.

Satisfiability of a finite set Φ of inequalities between variables and constants over the chain is decided by propagation. Start with $\lambda(v) = \bot$ for every variable, and let λ map each constant to itself. While some inequality $t_1 \le v$ with a variable v has $\lambda(t_1) > \lambda(v)$, raise $\lambda(v)$ to $\lambda(t_1)$. Each variable is raised at most twice. Induction on the raises shows that $\sigma \ge \lambda$ pointwise for every solution σ. At termination, every inequality into a variable holds under λ. If λ satisfies Φ, then Φ is satisfiable. Otherwise a violated inequality $t_1 \le b$ has a constant right side and $\lambda(t_1) > b$; every solution σ has $\sigma(t_1) \ge \lambda(t_1) > b$, so Φ is unsatisfiable. All steps take polynomial time. □

G Proofs of the Lower Bounds

All trees in this appendix are covariant, and $n \ge 2$. We use the notation of Section 10.2. For a word v, $\mathrm{pr}(v)$ denotes the set of its prefixes, including v; $\mathrm{pr}(v)$ is finite and prefix closed.

G.1 Top-Free Trees

Proof of Lemma 10.4. We first note that $\widehat{t}(w) = f$ iff $w \in \mathrm{Pos}(t)$, for every tree t at positive arity. If $w \in \mathrm{dom}(t)$, then every proper prefix of w is labeled f by (2), so the walk along w reaches its end and $\widehat{t}(w) = t(w)$. Otherwise, the longest prefix of w in $\mathrm{dom}(t)$ is a leaf by (2), and a leaf is labeled $\bot$ or $\top$.

(1) For a top-free tree, leaves are labeled $\bot$, so by the remark $\widehat{t}(w) = \bot$ for $w \notin \mathrm{Pos}(t)$. For top-free t, u both leaf extensions take values in $\{\bot, f\}$, so by (9) $t \le u$ holds iff $\widehat{t}(w) = f$ implies $\widehat{u}(w) = f$ for every w, that is, iff $\mathrm{Pos}(t) \subseteq \mathrm{Pos}(u)$.

(2) The set $\{\varepsilon\} \cup P \cup P\mathbb{D}$ contains the root and is prefix closed, because P is. For a word w in it and $i \in \mathbb{D}$, the word wi belongs to it iff $w \in P$: if $wi \in P$, then $w \in P$ by prefix closure. So labeling P by f and the other words by $\bot$ satisfies (2), and $\mathrm{Pos}(T_P) = P$. A top-free tree t is determined by $\mathrm{Pos}(t)$, since by (2) its domain is $\{\varepsilon\} \cup \mathrm{Pos}(t) \cup \mathrm{Pos}(t)\mathbb{D}$; hence T_P is unique. The domain is finite iff P is, since $\mathbb{D}$ is finite.

(3) Write $u = \sigma(g)$ and $s = f(u, u, \bot, \dots, \bot)$, using $\sigma(b) = \bot$. Then $\widehat{s}(\varepsilon) = f$, $\widehat{s}(iw) = \widehat{u}(w)$ for $i \in \{0, 1\}$, and $\widehat{s}(iw) = \bot$ for $2 \le i < n$. Since $u \le s$, (9) gives $\widehat{u}(w) \le \widehat{s}(w)$ for every w. By induction on $|w|$ we show that $\widehat{u}(w) \ne \top$, and that $w \in \mathrm{Pos}(u)$ implies $w \in \{0, 1\}^*$. For $w = \varepsilon$, $\widehat{u}(\varepsilon) \le f$. For a word iw with $i \ge 2$, $\widehat{u}(iw) \le \bot$, so $\widehat{u}(iw) = \bot$ and $iw \notin \mathrm{Pos}(u)$. For $i \in \{0, 1\}$, $\widehat{u}(iw) \le \widehat{u}(w) \ne \top$; if moreover $iw \in \mathrm{Pos}(u)$, then $f = \widehat{u}(iw) \le \widehat{u}(w)$ forces $\widehat{u}(w) = f$, so $w \in \mathrm{Pos}(u)$ and, by induction, $iw \in \{0, 1\}^*$. A node labeled $\top$ at w would give $\widehat{u}(w) = \top$, so u is top free. For $t \le s$, the same comparison gives $\widehat{t}(w) \le \widehat{s}(w) \ne \top$ for every w, so t is top free; and if $iw \in \mathrm{Pos}(t)$, then $f \le \widehat{s}(iw)$ forces $i \in \{0, 1\}$ and $\widehat{u}(w) = f$, so $iw \in \{0, 1\}\,\mathrm{Pos}(u) \subseteq \{0, 1\}^*$.

(4) Following a word w in T_P^m gives

$$\widehat{T_P^m}(w) = \begin{cases} \top, & |w| \ge m \text{ and } w[:m] \in P, \\ f, & |w| < m \text{ and } w \in P, \\ \bot, & \text{otherwise.} \end{cases}$$

Let $s = f(t_0, \dots, t_{n-1})$. By (9) it suffices to show $\widehat{s}(w) \le \widehat{T_P^m}(w)$ for every w. At ε both sides are f, since $\varepsilon \in P$ and $m \ge 1$. At iv with $i \ne a$ the left side is $\bot$. At av the left side is $\widehat{T_{P'}^m}(v)$. If it is $\top$, then

$|v| \geq m$ and $v[:m] \in P'$, so $av[:m] \in P$, hence $(av)[:m] \in P$ by prefix closure, and the right side is $\top$. If it is f, then $|v| < m$ and $av \in P$; the right side is f if $|av| < m$, and $\top$ if $|av| = m$. Otherwise the left side is $\bot$. □

G.2 The Counter Automaton

Proof of Lemma 10.5. *A token automaton.* Let $\mathcal{N}_\beta$ be the nondeterministic automaton over the token alphabet $\mathbb{T}$ with the following states and transitions; a transition that is not listed does not exist. Bits are the letters $0, 1$.

- E loops on every letter.
- $\mathsf{F}_0, \ldots, \mathsf{F}_\beta$ check the format: F_i goes on a bit to F_{i+1} if $i < \beta$ and to E if $i = \beta$; on # to F_0 if $i = \beta$ and to E otherwise; and on \$ to E if $i < \beta$.
- I checks that the first block is zero: it goes on 0 to I and on 1 to E.
- C_γ, for a carry $\gamma \in \{0, 1\}$, tracks the carry of an increment and spawns checkers: on a bit a it goes both to $\mathsf{C}_{\gamma \wedge a}$ and to $\mathsf{V}_{a \oplus \gamma, \beta}$, and on # to C_1.
- $\mathsf{V}_{a,j}$, for $a \in \{0, 1\}$ and $0 \leq j \leq \beta$, waits for the corresponding bit of the next block: for $j > 0$ it goes on every letter other than \$ to $\mathsf{V}_{a,j-1}$, and $\mathsf{V}_{a,0}$ goes on the bit $1 - a$ to E.
- O_o, for $o \in \{0, 1\}$, records whether the current block is all ones so far: on a bit a it goes to $\mathsf{O}_{o \wedge a}$; on # to E if $o = 1$ and to O_1 if $o = 0$; and O_0 goes on \$ to E.

The initial states are F_0, I, C_1, and O_1, and there are $3\beta + 9$ states, all final. The following table summarizes the roles of the state families.

States	Error detected	Behavior on a prefix of H_β
E	none; records an error found earlier	never reached
$\mathsf{F}_0, \ldots, \mathsf{F}_\beta$	a block with other than β bits	counts the bits of each block; dies at \$
I	a bit 1 in the first block	reads the first block; dies at the first #
$\mathsf{C}_0, \mathsf{C}_1$	none; spawns the checkers	tracks the carry of the increment; dies at \$
$\mathsf{V}_{a,j}$	a wrong bit $\beta + 1$ tokens after the bit that spawned it	dies on reading the predicted bit, or at \$
$\mathsf{O}_0, \mathsf{O}_1$	# after the all-ones block, or \$ after another block	dies at \$

We show that the words with a run of $\mathcal{N}_\beta$ are exactly the words of $\mathbb{T}^* \setminus \mathsf{H}_\beta \mathbb{T}^*$. Write $\mathsf{H} = \mathsf{H}_\beta$; the bit of $\langle j \rangle$ at offset $p < \beta$ is at position $j(\beta + 1) + p$ of H.

No branch reaches E while reading a prefix of H. The format is correct. The first block is 0^β. Along H the carry branch is deterministic: after the bits of $\langle j \rangle$ at offsets below p, it is in C_γ with γ the carry into offset p when 1 is added to j. On the bit a at offset p it therefore spawns $\mathsf{V}_{a \oplus \gamma, \beta}$. For $j < 2^\beta - 1$, the bit $a \oplus \gamma$ is the bit of $\langle j + 1 \rangle$ at offset p. That bit lies $\beta + 1$ positions later, so the checker reads it in state $\mathsf{V}_{a \oplus \gamma, 0}$ and has no transition on it. A # follows only blocks that are not all ones, and \$ follows the all-ones block $\langle 2^\beta - 1 \rangle$. After H, no state remains: F_β, the states C_γ, and O_1 have no transition on \$, I has none on the first #, the checkers of earlier blocks have died as just shown, and a checker spawned at offset p of the last block reads \$ in state $\mathsf{V}_{a,p+1}$, which has no transition on \$. So no word of $\mathsf{H}\mathbb{T}^*$ has a run.

Every proper prefix of H has a run through the format states. Let $u \notin \mathsf{H}\mathbb{T}^*$ be no prefix of H, and let $i < |\mathsf{H}|$ be the first position with $u[i] \neq \mathsf{H}[i]$. Some branch that is active after $u[:i] = \mathsf{H}[:i]$ reaches E on $u[i]$:

- if $\mathsf{H}[i]$ is the bit at offset p of $\langle j \rangle$ and $u[i]$ is # or \$, the format branch is in F_p with $p < \beta$;

- if $u[i]$ is the other bit, then for $j = 0$ the branch I reads 1, and for $j \geq 1$ the checker spawned at offset p of $\langle j-1 \rangle$ is in $\mathsf{V}_{\mathsf{H}[i],0}$;
- if $\mathsf{H}[i] = \#$, a bit leads F_β to E, and \$ leads O_0 to E, since the block before is not all ones;
- if $\mathsf{H}[i] = \$$, a bit leads F_β to E, and # leads O_1 to E.

Since E loops, u has a run, which proves the claim.

Binary words. Let $\mathcal{B}_\beta$ have the states q, $(q, 0)$, and $(q, 1)$ for every state q of $\mathcal{N}_\beta$, all final, the initial states of $\mathcal{N}_\beta$, a transition on the bit a from q to (q, a), and a transition on the bit a' from (q, a) to every successor of q in $\mathcal{N}_\beta$ on the token encoded by aa'. It has $9\beta + 27$ states. By induction on the number of encoded letters, the states of the form q reachable on the token encoding of a word $z \in \mathbb{T}^*$ are the states of $\mathcal{N}_\beta$ reachable on z, and after one further bit a the reachable states are the (q, a) for these q; this set is nonempty iff the former is. So a binary word u has a run iff the word $z \in \mathbb{T}^*$ encoded by the longest prefix u' of u of even length has a run of $\mathcal{N}_\beta$, that is, iff $z \notin \mathsf{H}\mathbb{T}^*$. As W_β has even length and the encoding uses blocks of two bits, $z \in \mathsf{H}\mathbb{T}^*$ iff $u' \in \mathsf{W}_\beta\{0,1\}^*$ iff $u \in \mathsf{W}_\beta\{0,1\}^*$. The construction is explicit and takes time polynomial in β. □

G.3 The Embedding of Binary Trees

The proof of Proposition 10.6 for $n \geq 3$ compares the systems $\Phi_\beta^{(n)}$ with Φ_β by an embedding of binary trees. Let $n \geq 3$, let f be covariant of arity n, and let f_2 be binary and covariant. For a binary tree t, let $\mathrm{lift}(t)$ be the tree over $\mathbb{D}$ with domain $\mathrm{dom}(t) \cup \{wi : w \in \mathrm{Pos}(t),\ 2 \leq i < n\}$, the labels of t on $\mathrm{dom}(t)$ with f_2 renamed to f, and $\bot$ elsewhere. For a tree t' over $\mathbb{D}$, let $\mathrm{proj}(t')$ be its restriction to $\mathrm{dom}(t') \cap \{0,1\}^*$ with f renamed to f_2. For a binary tree, Pos collects the positions labeled f_2, and leaf extensions of trees over the two signatures are compared with f_2 identified with f. For a flat system χ over f_2 and a variable o, either with the literal $o = \bot$ in χ or fresh, let $\chi^{(n)}$ replace every $f_2(a_0, a_1)$ by $f(a_0, a_1, \bar{o})$ and contain the literal $o = \bot$.

Proposition G.1 (Embedding binary trees). *Let $n \geq 3$, and let f, f_2,* lift, proj, *and $\chi^{(n)}$ be as above.*

(1) $\mathrm{proj} \circ \mathrm{lift}$ *is the identity,* lift *and* proj *are monotone and preserve finite and regular trees, and* $\widehat{\mathrm{lift}(t)}(w) = \widehat{t}(w)$ *and* $\widehat{\mathrm{proj}(t')}(w) = \widehat{t'}(w)$ *for every* $w \in \{0,1\}^*$.

(2) In each domain, $\mathrm{lift} \circ \sigma$, *extended by* $o \mapsto \bot$, *solves $\chi^{(n)}$ whenever σ solves χ, and* $\mathrm{proj} \circ \sigma'$ *solves χ whenever σ' solves $\chi^{(n)}$. Hence χ and $\chi^{(n)}$ are satisfiable in the same domains, and for every query $x \leq y$ a word in $\{0,1\}^*$ is left (right) unsafe for χ iff it is left (right) unsafe for $\chi^{(n)}$.*

(3) If χ contains $g \leq f_2(g, g)$ and $x \leq f_2(g, g)$, then no word is left unsafe for $(\chi^{(n)}, x, y)$, and every right-unsafe word lies in $\{0,1\}^*$.

Proof of Proposition G.1. As above, f_2 is identified with f whenever labels or leaf extensions of trees over the two signatures are compared.

(1) For a binary tree t, the domain of $\mathrm{lift}(t)$ is prefix closed and contains ε. It satisfies (2): for $w \in \mathrm{dom}(t)$, a child wi with $i \in \{0,1\}$ belongs to it iff $t(w) = f_2$; a child wi with $2 \leq i < n$ belongs to it iff $w \in \mathrm{Pos}(t)$; and the added nodes are labeled $\bot$ and have no children. Similarly, $\mathrm{dom}(t') \cap \{0,1\}^*$ satisfies (2) for f_2, so $\mathrm{proj}(t')$ is a binary tree. As $\mathrm{dom}(\mathrm{lift}(t)) \cap \{0,1\}^* = \mathrm{dom}(t)$, $\mathrm{proj} \circ \mathrm{lift}$ is the identity. For $w \in \{0,1\}^*$, the walk along w visits the same nodes, with the same labels up to renaming, in t and $\mathrm{lift}(t)$, and in t' and $\mathrm{proj}(t')$, so the leaf extensions agree at w. For monotonicity use (3): the common positions of $\mathrm{lift}(t)$ and $\mathrm{lift}(u)$ in $\{0,1\}^*$ are common positions of t and u with the same labels, and the others end in a letter $i \geq 2$ and are labeled $\bot$ in both; the common positions of $\mathrm{proj}(t')$ and $\mathrm{proj}(u')$ are common positions of t' and u' with the same labels. Both maps preserve finite domains. Every subtree of $\mathrm{lift}(t)$ is the image of a subtree of t or is $\bot$, and every subtree of $\mathrm{proj}(t')$ is the image of a subtree of t', so both maps preserve regularity.

(2) Both maps fix $\bot$ and $\top$, and

$$\mathrm{lift}(f_2(t_0,t_1)) = f(\mathrm{lift}\,t_0, \mathrm{lift}\,t_1, \bot, \dots, \bot), \qquad \mathrm{proj}(f(t_0,\dots,t_{n-1})) = f_2(\mathrm{proj}\,t_0, \mathrm{proj}\,t_1).$$

Let σ solve χ. Applying the monotone map lift to a literal $f_2(\sigma a_0, \sigma a_1) \le \sigma z$ gives

$$f(\mathrm{lift}\,\sigma a_0, \mathrm{lift}\,\sigma a_1, \bot, \dots, \bot) \le \mathrm{lift}\,\sigma z,$$

which is the translated literal under $\mathrm{lift}\circ\sigma$ with $o \mapsto \bot$; if o is a variable of χ, then $\sigma(o) = \bot$ and $\mathrm{lift}(\bot) = \bot$. Upper and constant literals are treated in the same way, and $o = \bot$ holds. Conversely, applying proj to a translated literal under a solution σ' of $\chi^{(n)}$ deletes the arguments $2,\dots,n-1$ and gives the original literal under $\mathrm{proj}\circ\sigma'$. By (1), both maps preserve each domain. For a binary word w, the leaf extensions at w are preserved by (1), so a solution that witnesses w as left or right unsafe for one system is mapped to a solution that witnesses the same for the other.

(3) Let σ' solve $\chi^{(n)}$. The literals $o = \bot$, $g \le f(g,g,\bar{o})$, and $x \le f(g,g,\bar{o})$ and Lemma 10.4(3), with o in the role of b, show that $\sigma'(x)$ is top free with $\mathrm{Pos}(\sigma'(x)) \subseteq \{0,1\}^*$. Hence $\widehat{\sigma'(x)}(w) \ne \top$ for every w, and if $\widehat{\sigma'(x)}(w) \ne \bot$ then $w \in \mathrm{Pos}(\sigma'(x)) \subseteq \{0,1\}^*$ by Lemma 10.4(1). □

G.4 Exponentially Long Holes

Proof of Proposition 10.6. Let $n = 2$ first, and let L_q be the set of words accepted from q, for $q \in Q \cup \{q_{\mathrm{in}}\}$; it is prefix closed because all states are final. By Lemma 10.5, $L_{q_{\mathrm{in}}} = \{0,1\}^* \setminus \mathrm{W}_\beta\{0,1\}^*$.

Claim A. Every solution σ of Φ_β has $\sigma(x)$ top free with $\mathrm{Pos}(\sigma(x)) \subseteq \{0,1\}^*$, and $\widehat{\sigma(Y_q)}(v) \ne \bot$ for every $q \in Q \cup \{q_{\mathrm{in}}\}$ and $v \in L_q$. The first part is Lemma 10.4(3). For the second, use induction on v. For $v = \varepsilon$, the literal $f(b,b) \le Y_q$ and (9) give $\widehat{\sigma(Y_q)}(\varepsilon) \ge f$. For $v = av'$, some transition $q \xrightarrow{a} q'$ has $v' \in L_{q'}$. The corresponding literal, say $f(Y_{q'}, b) \le Y_q$ for $a = 0$, and (9) give $\widehat{\sigma(Y_q)}(0v') \ge \widehat{\sigma(Y_{q'})}(v')$, which is not $\bot$ by induction; the case $a = 1$ is symmetric.

(2) By Claim A, $\widehat{\sigma(x)}(w) \ne \top$ for every solution σ and every w, so no word is left unsafe. If w is right unsafe, witnessed by σ, then $\widehat{\sigma(x)}(w) \ne \bot$, so $w \in \mathrm{Pos}(\sigma(x)) \subseteq \{0,1\}^*$ by Lemma 10.4(1), and $\widehat{\sigma(y)}(w) = \bot$, so $w \notin L_{q_{\mathrm{in}}}$ by Claim A. Hence $w \in \mathrm{W}_\beta\{0,1\}^*$. Conversely, let $w \in \mathrm{W}_\beta\{0,1\}^*$ and $m = |w| + 1$, and put

$$\sigma_w(b) = \bot, \qquad \sigma_w(g) = T_{\{0,1\}^{<m}}, \qquad \sigma_w(x) = T_{\mathrm{pr}(w)}, \qquad \sigma_w(Y_q) = T^m_{L_q} \ \ (q \in Q \cup \{q_{\mathrm{in}}\}).$$

These trees are finite. The literal $b = \bot$ holds. By Lemma 10.4(1), $g \le f(g,g)$ holds because $\{0,1\}^{<m} \subseteq \{\varepsilon\} \cup \{0,1\}\{0,1\}^{<m}$, and $x \le f(g,g)$ holds because $\mathrm{pr}(w) \subseteq \{0,1\}^{\le m}$. Each lower literal holds by Lemma 10.4(4): $\varepsilon \in L_q$, the literal $f(b,b) \le Y_q$ is the case $P' = \varnothing$, and $q \xrightarrow{a} q'$ gives $aL_{q'} \subseteq L_q$. Thus σ_w solves Φ_β. Finally $\widehat{\sigma_w(x)}(w) = f$, while $|w| < m$ and $w \notin L_{q_{\mathrm{in}}}$ give $\widehat{\sigma_w(y)}(w) = \bot$ by the formula in the proof of Lemma 10.4(4). So w is right unsafe.

(1) The solution $\sigma_{\mathrm{W}_\beta}$ is finite, hence regular, and lies in each domain. By (9), $\widehat{\sigma(x)}(\mathrm{W}_\beta) = f > \bot = \widehat{\sigma(y)}(\mathrm{W}_\beta)$ gives $\sigma(x) \not\le \sigma(y)$ for $\sigma = \sigma_{\mathrm{W}_\beta}$.

(3) The system is satisfiable, so by Theorem 5.4 the holes of each label reader are exactly the unsafe words of its side, and (2) identifies them. The search of Figure 12 returns its answer from such a hole in each domain: by (1), the finite-satisfiability test succeeds.

For $n \ge 3$, read the binary constructor of Φ_β as f_2. Then the system $\Phi^{(n)}_\beta$ is $(\Phi_\beta)^{(n)}$ in the notation of Proposition G.1, with $o = b$. By Proposition G.1(2), the finite assignment $\mathrm{lift}\circ\sigma_w$ solves $\Phi^{(n)}_\beta$, and the unsafe words in $\{0,1\}^*$ of the two systems coincide. By Proposition G.1(3), every unsafe word of $\Phi^{(n)}_\beta$ is right unsafe and lies in $\{0,1\}^*$. So (1)–(3) carry over.

For every $n \geq 2$ there are $k = 9\beta + 31$ variables, so $\beta + 1 = (k-22)/9$ and $\lambda_\beta = (\beta+1)2^{\beta+1} \geq 2^{(k-22)/9}$. □

G.5 PSPACE-Hardness

Proof of Theorem 10.7. Let $\mathcal{A} = (Q, q_0, F, \delta)$ and $\mathcal{A}' = (Q', q_0, \{q_\mathrm{f}\}, \delta')$ be as in Section 10.2. From Q, the automaton $\mathcal{A}'$ reads 0 into a middle state or, from a final state, 1 into q_f; a middle state of (q, a, q') reads only a, into q'; and q_f has no transition. Since q_f is the only final state, the accepting runs from q_0 are exactly those that read $0a_1 0a_2 \cdots 0a_t 1$ through states $q_0, \mu_1, q_1, \ldots, \mu_t, q_t, q_\mathrm{f}$, where μ_i is the middle state of the transition (q_{i-1}, a_i, q_i) and $q_0, \ldots, q_t$ is an accepting run of $\mathcal{A}$ on $a_1 \cdots a_t$. Hence $\mathcal{A}'$ accepts exactly $\mathrm{code}(L(\mathcal{A}))$. Let $P = \{e(u), e(u)0 : u \in \{0,1\}^*\}$ be the set of proper prefixes of codes and $C = \mathrm{code}(\{0,1\}^*)$ the set of codes. The set P is prefix closed and contains ε. If $\pi \in P$ and $\pi a \notin P$ for a letter $a \in \mathbb{D}$, then $a \geq 2$ or $\pi a \in C$, since $e(u)0 \in P$ and $e(u)0a' = e(ua') \in P$ for $a' \in \{0,1\}$.

Universal automata. Let $\mathcal{A}$ be universal, and let σ be a solution of $\chi(\mathcal{A}')$, in any domain.

First, $\widehat{\sigma(r)}(\pi) \in \{f, \top\}$ for every $\pi \in P$. Let $\mathrm{s}(\pi) = r$ for $\pi = e(u)$ and $\mathrm{s}(\pi) = r'$ for $\pi = e(u)0$. We show by induction on $\pi \in P$ that $\widehat{\sigma(r)}(\pi) = \top$, or $\pi \in \mathrm{dom}(\sigma(r))$ and $\sigma(\mathrm{s}(\pi)) \leq \sigma(r)|_\pi$, where $t|_\pi$ is the subtree at π. This holds at ε. Let $\pi = \pi' a \in P$ with $\pi' \in P$. If $\widehat{\sigma(r)}(\pi') = \top$, then also $\widehat{\sigma(r)}(\pi) = \top$. Otherwise the lower literal of $\mathrm{s}(\pi')$ puts a constructor below $\sigma(r)|_{\pi'}$, so the label of $\sigma(r)$ at π' is f or $\top$. If it is $\top$, then $\widehat{\sigma(r)}(\pi) = \top$. If it is f, decomposing at the child a gives the claim: for $\mathrm{s}(\pi') = r$, the only child in P is $a = 0$, $\mathrm{s}(\pi) = r'$, and $f(\sigma r', \bot, \ldots, \bot) \leq \sigma(r)|_{\pi'}$ gives $\sigma(r') \leq \sigma(r)|_\pi$; for $\mathrm{s}(\pi') = r'$, $a \in \{0,1\}$, $\mathrm{s}(\pi) = r$, and $f(\sigma r, \sigma r, \bot, \ldots, \bot) \leq \sigma(r)|_{\pi'}$ gives $\sigma(r) \leq \sigma(r)|_\pi$. In the second alternative, the lower literal of $\mathrm{s}(\pi)$ makes the label of $\sigma(r)$ at π at least f.

Second, let $w \in (P \cup C) \cap \mathrm{dom}(\sigma(x))$. Then $\sigma(x)(w) \neq \top$; if $w \in P$ and $\sigma(x)(w) = f$, then $\sigma(x)(wi) = \bot$ for $2 \leq i < n$; and if $w \in C$, then $\sigma(x)(w) = \bot$. Indeed, w is a prefix of a code v, which is accepted because $\mathcal{A}$ is universal; take an accepting run $p_0 = q_0, p_1, \ldots, p_{|v|} = q_\mathrm{f}$ of $\mathcal{A}'$ on v. By induction on $j \leq |w|$, if $w[:j] \in \mathrm{dom}(\sigma(x))$ then $\sigma(x)|_{w[:j]} \leq \sigma(X_{p_j})$. This holds for $j = 0$, as $x = X_{q_0}$. If $w[:j+1] \in \mathrm{dom}(\sigma(x))$, then $\sigma(x)$ has the label f at $w[:j]$, and the transition from p_j to p_{j+1} on the letter $w[j]$ gives a literal $X_{p_j} \leq f(\ldots)$ with $X_{p_{j+1}}$ in the argument $w[j]$. So $\sigma(X_{p_j})$ lies between two trees with the root label f and has this root label itself, and decomposing both inequalities at the child $w[j]$ gives $\sigma(x)|_{w[:j+1]} \leq \sigma(X_{p_{j+1}})$. At $j = |w|$: if $w \in P$, then $p_{|w|}$ has a transition on $v[|w|]$, whose literal bounds $\sigma(x)|_w$ by a constructor tree with the children $\sigma(b) = \bot$ in the arguments $2, \ldots, n-1$; if $w \in C$, then $p_{|w|} = q_\mathrm{f}$ and $\sigma(x)|_w \leq \sigma(X_{q_\mathrm{f}}) = \bot$.

Now let w be a common position of $\sigma(x)$ and $\sigma(r)$. If $w \in P$, the two facts give $\sigma(x)(w) \leq f \leq \sigma(r)(w)$. Otherwise, let πa be the shortest prefix of w outside P, with $\pi \in P$. Then πa is a position of $\sigma(x)$ with $a \geq 2$ or $\pi a \in C$, so $\sigma(x)(\pi a) = \bot$ by the second fact. A leaf has no children, so $w = \pi a$ and $\sigma(x)(w) = \bot \leq \sigma(r)(w)$. By (3), $\sigma(x) \leq \sigma(r)$. Thus $\chi(\mathcal{A}') \models_\mathcal{D} x \leq r$ in each domain.

Rejecting automata. Let $\mathcal{A}$ reject w_0, and put $c_0 = \mathrm{code}(w_0)$. A prefix of c_0 accepted by $\mathcal{A}'$ would be a code, hence equal to c_0 by prefix freeness; but c_0 is not accepted, since $w_0 \notin L(\mathcal{A})$ and the code is injective. For $q \in Q'$ let J_q be the set of cuts $j \leq |c_0|$ such that q is reachable from q_0 on $c_0[:j]$, and put $P_q = \bigcup_{j \in J_q} \mathrm{pr}(c_0[j:])$. Define

$$\sigma(X_q) = T_{P_q}, \qquad \sigma(z) = \top, \qquad \sigma(b) = \bot, \qquad \sigma(r) = r_m, \qquad \sigma(r') = r'_m, \qquad m = |c_0|,$$

where $r_0 = r'_0 = \top$, $r_{j+1} = f(r'_j, \bot, \ldots, \bot)$, and $r'_{j+1} = f(r_j, r_j, \bot, \ldots, \bot)$. All these trees are finite. We check the literals. No prefix of c_0 is accepted, so $J_{q_\mathrm{f}} = \varnothing$ and $\sigma(X_{q_\mathrm{f}}) = T_\varnothing = \bot$. For a transition $q \xrightarrow{0} q'$ with $P_q \neq \varnothing$, the tree T_{P_q} has the label f at its root, its children in the arguments $i \geq 2$ are $\bot$ because $P_q \subseteq \{0,1\}^*$, its child 1 lies below $\sigma(z) = \top$, and its child 0 is $T_{P'}$ with

$P' = \bigcup\{\mathrm{pr}(c_0[j+1{:}]) : j \in J_q,\ j < |c_0|,\ c_0[j] = 0\}$. Each such j has $j+1 \in J_{q'}$, so $P' \subseteq P_{q'}$, and $T_{P'} \leq T_{P_{q'}}$ by Lemma 10.4(1). Hence $\sigma(X_q) \leq f(\sigma X_{q'}, \sigma z, \bot, \ldots, \bot)$; for $P_q = \emptyset$ this is immediate. The letter 1 is symmetric. The literal $b = \bot$ holds. By induction, $r_{j+1} \leq r_j$ and $r'_{j+1} \leq r'_j$: both hold at $j = 0$ against $\top$, and the constructor is monotone. So $f(r'_m, \bot, \ldots, \bot) = r_{m+1} \leq r_m$ and $f(r_m, r_m, \bot, \ldots, \bot) = r'_{m+1} \leq r'_m$.

It remains to refute the query. Since $0 \in J_{q_0}$, we have $c_0 \in \mathrm{Pos}(\sigma(x))$. For $u \in \{0,1\}^*$ and $j \geq 2|u| + 1$, the tree r_j has the label f along $e(u)$, and its subtree at $e(u)$ is $r_{j-2|u|}$, because r_i has the child r'_{i-1} at 0 and r'_i has the child r_{i-1} at 0 and at 1, for $i \geq 1$. With $m = 2|w_0| + 1$, the subtree of r_m at $e(w_0)$ is $r_1 = f(\top, \bot, \ldots, \bot)$, so $c_0 = e(w_0)1$ is a common position at which $\sigma(x)$ has the label f and $\sigma(r)$ has $\bot$. By (3), $\sigma(x) \not\leq \sigma(r)$. This finite countermodel lies in every domain.

The reduction. The assignment with $X_q = z = b = \bot$ and $r = r' = \top$ solves $\chi(\mathcal{A}')$, so the system is satisfiable. It has $|Q'| + 4$ variables with $|Q'| = |Q| + |\delta| + 1$, and $2|\delta| + |F| + 4$ literals, each containing at most $n + 1$ variable occurrences, and it is computed in polynomial time for fixed n. Universality of nondeterministic automata over $\{0,1\}$ with one initial state is PSPACE-hard [Krötzsch et al. 2016, Theorem 3], as explained before Theorem 10.7. Hence entailment is PSPACE-hard in each domain. The covariant instances are instances of the problem with the variance as part of the input, so that problem is PSPACE-hard as well. □

H The Mechanized Proof

This appendix gives the formal definitions, the decision chain, and the correspondence with the paper. Names are in namespace `DeciNSSE` unless qualified otherwise. Listings suppress proof bodies and implicit variable declarations where these do not affect the statement. Assignments in the source may be named ρ, σ, or A; the paper uses variants of σ for assignments. Paths are lists over `Fin n`; binary examples use $\{0,1\}$.

H.1 Trusted Definitions and Their Characterization

The primitive symbols are `Sym.bot`, `Sym.f`, and `Sym.top`, with their three-element order, induced by `Sym.rank` with values 0, 1, 2. The trusted statement also includes `polarity`, the parity of contravariant positions on a path, and the tree operations `Tree.node`, `Tree.bot`, and `Tree.top`. Trees and their variance order are defined by

```
structure Tree (n : ℕ) where
  fn : List (Fin n) → Option Sym
  wf : fn [] ≠ none ∧
       ∀ π i, (fn (π ++ [i])).isSome ↔ fn π = some Sym.f

def Tree.Le (c : Fin n → Bool) (t u : Tree n) : Prop :=
  ∀ π a b, t.fn π = some a → u.fn π = some b →
    if polarity c π then b ≤ a else a ≤ b
```

Here `true` is contravariant. The notation $\leq$ on `Tree n` means the internal covariant order `Tree.CovLe`; the public statements use `Tree.Le` explicitly.

The module `Semantics/Characterization` relates these definitions to conventional semantics. `Characterization.treeLe_eq_gfp` identifies the order with the greatest fixed point of the textbook rule; `Characterization.lfp_ne_treeLe` distinguishes the least fixed point at positive arity. `Characterization.unfold_range_iff` characterizes the range of graph unfolding by finiteness of the set of rooted subtrees, and `Characterization.toTree_range_iff` characterizes the image of finite terms by finite tree domain. The interpretation maps `RGraph.unfold` and `FTree.toTree` are themselves part of the trusted statement; the two range theorems justify quantification over

these representations of the intended tree classes. The module also proves the precise flat expressiveness distinction: `Characterization.le_definable_iff` and `Characterization.nullary_not_definable` cover positive and zero arity. General flattening of nested input syntax is not part of the exported API.

Variables are `V`, an abbreviation for `Fin k`, and `Constraint` is a list of the following literals:

```
inductive Lit (n k : ℕ) where
  | leF (x : V k) (children : Fin n → V k)
  | fLe (children : Fin n → V k) (x : V k)
  | eqBot (x : V k)
  | eqTop (x : V k)

def Sat (c : Fin n → Bool) (ρ : V k → Tree n)
    (ϕ : Constraint n k) : Prop :=
  ∀ l ∈ ϕ, Lit.holds c ρ l

abbrev Satisfiable (c : Fin n → Bool) (ϕ : Constraint n k) : Prop :=
  ∃ ρ, Sat c ρ ϕ

def Entails (c : Fin n → Bool) (ϕ : Constraint n k) (x y : V k) : Prop :=
  ∀ ρ, Sat c ρ ϕ → Tree.Le c (ρ x) (ρ y)
```

The interpretation `Lit.holds` uses constructor trees, the explicit variance order, and equality for the constant literals. Satisfaction by an assignment is distinct from existence of a solution.

```
def EntailsReg (c : Fin n → Bool) (ϕ : Constraint n k) (x y : V k) : Prop :=
  ∀ σ : V k → RGraph n, Sat c (RGraph.unfold ∘ σ) ϕ →
    Tree.Le c (σ x).unfold (σ y).unfold

def EntailsFin (c : Fin n → Bool) (ϕ : Constraint n k) (x y : V k) : Prop :=
  ∀ σ : V k → FTree n, Sat c (FTree.toTree ∘ σ) ϕ →
    Tree.Le c (σ x).toTree (σ y).toTree
```

H.2 Public API and Module Layout

Importing `DeciNSSE` exposes the three procedures:

```
def decideEntails {n k : ℕ} (c : Fin n → Bool)
    (ϕ : Constraint n k) (x y : V k) : Decidable (Entails c ϕ x y)
def decideEntailsReg {n k : ℕ} (c : Fin n → Bool)
    (ϕ : Constraint n k) (x y : V k) : Decidable (EntailsReg c ϕ x y)
def decideEntailsFin {n k : ℕ} (c : Fin n → Bool)
    (ϕ : Constraint n k) (x y : V k) : Decidable (EntailsFin c ϕ x y)

theorem decideEntails_correct {n k : ℕ} (c : Fin n → Bool)
    (ϕ : Constraint n k) (x y : V k) :
  @decide _ (decideEntails c ϕ x y) = true ↔ Entails c ϕ x y
```

The other correctness declarations are `decideEntailsReg_correct` and `decideEntailsFin_correct`. None assumes satisfiability, positive arity, an inhabited path alphabet, or a supplied reader family. `productVariance` and `arrowVariance`, in `Instances`, provide the two named binary special cases. Importing `DeciNSSE.Main` suffices for the three procedures. The summary and

unary procedures of Appendix H.6 have the same types, the unary ones at arity one. All of them instantiate `Monitor.decideEntailsOfMonitors` and its regular and finite versions, which take the two side monitors and a decision procedure for side holes.

The artifact also contains a certifying solver, `solve`. It searches for an entailment certificate, which records unsatisfiability or a closed cover of each query side, and for a refutation, which records a side and a hole; it checks every candidate with proved checkers and falls back to `decideEntails`. `solve_correct` states that its answer is `true` iff `Entails` holds, and a refutation carries a regular countermodel. Under `import DeciNSSE`, compiled code of `decideEntails` runs the packed decider `Solver.decideEntailsPacked`, by the `csimp` lemma `Solver.decideEntails_eq_decideEntailsPacked`; no other procedure is redirected, and the kernel-checked statements are unaffected.

Module group	Contents
`Words`	Finite-alphabet paths and periodicity.
`Semantics`	Trees, finite terms, regular graphs, variance, normalization, polarity selection, and characterization.
`Constraints`	Flat syntax, satisfaction, signed translation, closure, and path bounds.
`Satisfiability`	Least shapes, graph realization, clash tests, finite and regular solutions.
`Transfer`	Query sides and unsafe words, ranked extensions, cycle reflection, and countermodel transfers.
`Monitor`	Four-spine extension, closure and clashes, events, label reader, side monitors, semantic interface, and the generic procedures.
`Holes`	Readers and holes; the canonical hierarchy (`Holes/Hierarchy/*`), depth, cores, and supports; desubstitution, co-alignment, exchange types, level maps, normal forms, and boundary pairs; agreement, the column automaton, bounded-depth compression, and the elementary bound.
`RejectedTail`	Rejected-path property, branching and forced tails, return gap, depth, and finite search.
`Instances`, `Main`	Named variance maps and the three public decision procedures.
`Countermodels`	The Boolean refutation check and computable regular and finite countermodels.
`Certificates`	Certificate formats, covering products, the specification checkers and their soundness, and the JSON front end of the certificate checker.
`Solver`	The certifying solver: packed readers and checks, the packed decider, certificate search, reachable countermodels, and the JSON front end of the solver.
`Complexity`	The summary search, its configuration bound, and its procedures; one-letter readers and the unary procedures; binary automata and shared tree constructions; the hardness reduction; the counter automaton and the systems with exponentially long holes.

The executable front ends `Certificates/Command` and `Solver/Command` and their JSON parser `Certificates/Json` are built by the executable targets and are not reachable from `import DeciNSSE`; every other module is.

The files `lean-toolchain` and `lakefile.toml` pin Lean and Mathlib, respectively, to `v4.35.0`. To check the artifact, run

```
lake exe cache get
lake build DeciNSSE
```

The first command downloads compiled dependencies. The kernel checks the proofs, including the reduction to finite search; compiled execution also relies on the compiler. The three procedures are computable definitions, not classical choices of decidability instances. Proofs may use classical

reasoning. The package's axiom audit reports only `propext`, `Classical.choice`, and `Quot.sound` for the procedures, their correctness theorems, the solver's correctness theorem, and the headline theorems of `Complexity`. These dependencies can be inspected with

```
import DeciNSSE
#print axioms DeciNSSE.decideEntails
#print axioms DeciNSSE.decideEntailsReg
#print axioms DeciNSSE.decideEntailsFin
#print axioms DeciNSSE.decideEntails_correct
#print axioms DeciNSSE.decideEntailsReg_correct
#print axioms DeciNSSE.decideEntailsFin_correct
#print axioms DeciNSSE.solve_correct
#print axioms DeciNSSE.decideEntailsSummary_correct
#print axioms DeciNSSE.Complexity.Summary.card_config_label_le
#print axioms DeciNSSE.decideEntailsUnary_correct
#print axioms DeciNSSE.Complexity.Unary.exists_hole_length_le
#print axioms DeciNSSE.Complexity.entailsFin_hardnessSystem_iff
#print axioms DeciNSSE.Complexity.expSystem_shortest_hole
```

H.3 The Decision Chain

The function `signed` constructs the signed constraint system. The function `Signed.dual` implements the assignment involution and `SignSelection.select` implements polarity selection on assignments with a common shape. The shared theorem `SignSelection.select_leastShape_sat_fixed` states satisfaction and fixedness of the selected least shape of a flip-closed, sign-coherent system without label clash. It is used by the witness, regular, and finite constructions. `Monitor.satB` decides satisfiability by the signed label-clash test. `Spine.extension` enforces four prefix conditions in a signed layout directly; `Monitor/Closure` proves its closure invariants, and `Monitor/Clash` and `Monitor/Semantics` identify clashes with inherited clashes or events and then with unsafety. `Events.sideUnsafe_iff_not_labelClash` has no satisfiability premise.

The label reader `Monitor.reader` uses

```
structure Monitor.State (m : ℕ) where
  U : Finset (V m)
  L : Finset (V m)
  b : Bool
```

The reader, target, and admission definitions have no side parameter. `Events.sideSystem` chooses the signed system or its order dual, and `Events.sideQuery` chooses (x^+, y^+) or (y^+, x^+); `Monitor.sideReader` and `Monitor.sideTarget` are the reader and target of a side. The acceptance latch records readiness at a strictly earlier cut. `Monitor.target` is the complement of acceptance, and `Monitor.admission` is the disjunction of cross and self admission. `Monitor.hole_iff_not_occurs` requires no satisfiability hypothesis; `Monitor.hole_iff_unsafe` requires a solution, precisely to exclude inherited clashes. `Monitor.rejectedPath` supplies the rejected-path property without a satisfiability assumption. `Monitor.family` packages the two side readers for the given variance.

`RejectedTail.RejectedPath` expresses betweenness and the admission cone. The general procedure `RejectedTail.decideHole` enumerates finite-alphabet words up to `RejectedTail.holeLengthBound`. Its bound is defined by

```
def holeDepthBound (N : ℕ) : ℕ :=
```

```
    2 * N * (4*N - 1) - 1

def holeLengthBound (N : ℕ) : ℕ :=
  ElementaryBound.lengthBound N (holeDepthBound N)
```

Here N is the original reader-state count; Section 11 explains how derivative states enter the bound. The theorem `RejectedTail.return_gap` holds for every reader and gives $e - s \le |w| - J$; it assumes neither that w is a hole nor the rejected-path property. Intermediate depth and compression theorems assume an inhabited alphabet, which nonempty words supply; the final bounded-length theorem `RejectedTail.hole_iff_bounded_length` removes this assumption and explicitly covers the empty alphabet.

The bounded-depth theorem uses the predicate `LetteredHierarchy.InL`: $\mathsf{InL}(d, w)$ means that some canonical level $i \le d$ is unary. The bound in (48) is `ElementaryBound.lengthBound`:

```
def dictionaryBound (N d : ℕ) : ℕ :=
  2 + NormalForm.normalAllowance N d * 3 ^ d

def lengthBound (N d : ℕ) : ℕ :=
  (N + d + 1) *
    BoundedDepth.descentFactor N d (dictionaryBound N d) ^ d
```

The column count is `BoundedDepth.colBound`, and `BoundedDepth.descentFactorNdg` is that count at $N + d + 1$ states and g tracks. `NormalForm.exists_normal_form` preserves canonical depth and all start counts, never increases length or a dictionary, and recomputes the protected exchange types after each merge. `ElementaryBound.exists_bounded_dictionary_hole` bounds the positive dictionaries of the resulting word by `ElementaryBound.dictionaryBound`.

`BoundedDepth.shorten_level_supported` keeps its recursive support inside the original word's dictionary and deduplicates it. Its identity base case needs no bound on the level-zero alphabet. `BoundedDepth.exists_short_hole_of_dictionary` combines this descent with unary shortening. Finally, `ElementaryBound.exists_short_hole` composes normalization and descent; it retains membership in $\mathsf{InL}(d, \cdot)$, not necessarily equality of the first unary level. The back end invokes this theorem directly at `RejectedTail.holeDepthBound`, with no intermediate stopping reader.

The arithmetic estimates `ElementaryBound.dictionaryBound_le` and `ElementaryBound.lengthBound_le` yield the final checked inequality

```
theorem holeLengthBound_le (N : ℕ) :
  holeLengthBound N ≤ 2 ^ 2 ^ (1280 * (N + 1) ^ 5)
```

in namespace `RejectedTail`. The statements hold for every natural number N; actual monitors have a start state and hence $N \ge 1$. The cost of enumerating all words up to this length (Section 10.1) is not mechanized; Appendix H.6 describes the mechanized content of Section 10.

The entailment procedure `Monitor.decideEntailsOf` combines satisfiability testing with the two reader searches, here the enumeration `Monitor.SideMonitor.decideHoles` up to the length bound. `entailsReg_iff_entails` and `entailsFin_iff` provide the transfers. The finite procedure first tests `satFinB`; its equivalence with finite satisfiability uses the selector construction on the signed least-shape solution. `Main` instantiates the family, so there are no undischarged interface premises in the public API.

H.4 Correspondence

Table 4. Paper results and declarations in namespace `DeciNSSE`. A dagger marks differences in intermediate statement strength, retained witness information, or construction, explained below and in Appendices H.5 and H.6. The last nine rows concern Section 10 and Appendix F.

Paper result	Formal counterpart
(3)–(4), domains	`Characterization.treeLe_eq_gfp`, `Characterization.unfold_range_iff`, `Characterization.toTree_range_iff`
Normalization (Section 2.4)	`treeLe_iff_normalize`
Unsafe words, (11)	`entails_iff_not_sideUnsafe`
Signed solutions (Lemma 3.1)	`fixed_solution_correspondence`, `signedDual_involutive`, `SignSelection.sat_signedDual_of_flipClosed`, `normalize_injective`
Selection (Lemma 4.5)	`SignSelection.leastShape_sameShape_dual`, `SignSelection.select_sat_of_coherent`, `SignSelection.select_fixed`, `SignSelection.select_leastShape_sat_fixed`
Signed satisfiability, (24)	`sat_iff_signed_sat`
Fixed witnesses (Lemma 3.2 and Theorem 4.9)	`Events.sideUnsafe_iff_fixedWitness`, `Spine.Closure.fixedWitness_iff_not_labelClash`, `Events.sideUnsafe_iff_not_labelClash`
Closure and path rules (Section 4.1 and Lemma 4.1)	`Derives.sound`, `UpperAt.comp`, `LowerAt.comp`, `UpperAt.le_embed`, `LowerAt.embed_le`, `UpperAt.factor`, `LowerAt.factor`, `LowerAt.decompose_upper`, `LowerAt.decompose_lower`
Sign purity (Lemma 4.2)	`Signs.derives_sign`, `Signs.upperAt_sign`, `Signs.lowerAt_sign`
Satisfiability (Theorem 4.3)	`leastShape_sat`, `satisfiable_iff_not_labelClash`
Spine extension† (Lemma 4.8)	`Spine.Closure.witness_of_sat`, `Spine.Closure.extend_sat` (both for fixed assignments), `Spine.Closure.extension_flipClosed`, `Spine.Closure.extension_signCoherent`
Separation and bridges (Lemmas 4.10 and 4.11)	`Ranked.Extension.derives_into_source`, `Ranked.Extension.derives_out_of_sink`, `Ranked.Extension.derives_original` via `Spine.Closure.extension_ranked`, `Spine.Closure.derives_inv`, `Spine.Closure.derives_lv_of_uAt`, `Spine.Closure.derives_up_of_lAt`, `Spine.Closure.derives_bridge`
Clashes and events (Theorem 4.13)	`Spine.Closure.labelClash_extension_iff`, `Events.sideUnsafe_iff_not_events`
Label states and run (Section 5.2)	`Monitor.card_state`, `Monitor.mem_eval_U`, `Monitor.mem_eval_L`, `Monitor.eval_b_iff`
Rejected-path property (Section 5.3)	`Monitor.rejectedPath`

Paper result	Formal counterpart
Unsafe words are holes (Theorem 5.4)	`Monitor.hole_iff_not_occurs`, `Monitor.hole_iff_unsafe`
Tail (Theorem 6.1)	`RejectedTail.boundedRejectedTail_of_rejectedPath`
Return gap (Lemma 7.5)	`RejectedTail.return_gap`
Alignment (Lemma 7.3)	`LetteredHierarchy.alignment`
Cores (Lemma 7.4)	`CoreOccurrences.corePos_succ_iff_occurrence`, `LetteredHierarchy.startCore_invariants`, `RejectedTail.cores_succ_subset`, `StartCores.sCount_le_level`, `CoreOccurrences.rho_at_greatest`
Equal-support width (Lemma 7.6)	`RejectedTail.equal_support_width`
Depth (Theorem 7.7)	`RejectedTail.rejected_depth`, `RejectedTail.depth_le_of_support_width`
Derivatives (Lemma 8.3)	`LetteredHierarchy.isHole_der_iff`, `LetteredHierarchy.isHole_hierOf_iff`, `BoundedDepth.card_cores`
Comparison maps (Lemma C.1)	`CoAlignment.isComp_map_iff`, `LevelMaps.NoCollapse.reflect_last`
Exchange types (Lemma 8.5)	`Exchange.exType`, `Exchange.card_exType_le`, `NormalForm.normalTypeBound`, `Exchange.Pset`, `Exchange.Uset`
Isolated exchange (Lemma 8.6)	`Exchange.Isolated`, `Exchange.isolated_merge`, `NormalForm.isolated_merge_spec`
Level maps and start counts (Lemma C.2)	`LevelMaps.levelMap_spec`, `LevelMaps.start_below`, `LevelMaps.start_cap_up`, `LevelMaps.LevelMapResult.depth_eq`
Dictionary normal form (Lemma 8.7)	`NormalForm.Normal`, `NormalForm.dictionaryPotential_lt`, `NormalForm.exists_normal_form`
Dictionary counting (Lemma 8.8)	`NormalForm.normalAt_alphCard_le`, `NormalForm.normalAllowance`
Propagation ((44))	`BoundaryPairs.boundaryPairs_expand_subset`, `BoundaryPairs.card_endpointPairs_le`, `BoundaryPairs.boundaryCount_step_add_one`
Dictionary bounds (Lemma 8.9)	`ElementaryBound.normal_boundary_count_le`, `ElementaryBound.normal_alphCard_le`, `ElementaryBound.exists_bounded_dictionary_hole`
Agreement ((55), Lemma C.3)	`BoundedDepth.Agree`, `BoundedDepth.Agree.comp`, `BoundedDepth.Agree.isHole_iff`, `BoundedDepth.Agree.derive`
Column summary, (47)	`BoundedDepth.colEval`, `BoundedDepth.der_type_of_summ`, `BoundedDepth.exists_short_letters`, `BoundedDepth.colBound`, `BoundedDepth.card_CS`

Paper result	Formal counterpart
Supported compression (Lemma C.4)	`BoundedDepth.PositiveDictionaryBound`, `BoundedDepth.mem_lower_dictionary`, `BoundedDepth.shorten_level_supported`, `BoundedDepth.descentFactor`
Compression (Theorem 8.1)	`BoundedDepth.exists_short_hole_of_dictionary`, `ElementaryBound.exists_short_hole`, `ElementaryBound.lengthBound`
Closed form (Lemma C.5)	`ElementaryBound.dictionaryBound_le`, `ElementaryBound.lengthBound_le`, `RejectedTail.holeLengthBound_le`
Finite search (Theorem 9.1)	`RejectedTail.hole_iff_bounded_length`, `RejectedTail.decideHole`
Regular transfer† (Appendix D.1)	`unfold_shapeGraph`, `regular_fixed_solution`, `regular_countermodel`, `RegularTransfer.countermodel_spec`, `entailsReg_iff_entails`
Finite satisfiability (Theorem D.3 and Lemma D.4)	`satFin_iff`, `satFin_iff_signed`, `satFinB_iff`
Finite transfer† (Theorem D.5)	`Ranked.Extension.cycleClash_iff`, `FiniteTransfer.finite_fixedWitness`, `FiniteTransfer.finite_countermodel_of_unsafe`, `finite_countermodel_transfer`, `entailsFin_iff`
Decision (Theorems 1.1 and 9.2)	`decideEntails`, `decideEntailsReg`, `decideEntailsFin`, and their correctness theorems
Summary monitor (Lemma E.1)	`Complexity.Summary.derivedTable`, `Complexity.Summary.derivedTable_agree`
Summary search† (Proposition 10.2)	`Complexity.Summary.Config`, `Complexity.Summary.summarySearch`, `Complexity.Summary.summarySearch_iff_reachable`, `Complexity.Summary.summarySearch_sound`, `Complexity.Summary.summarySearch_complete`, `Complexity.Summary.card_config_le`, `Complexity.Summary.configBits_le`
Upper bounds† (Theorem 10.3)	`Complexity.Summary.hole_iff_summarySearch`, `Complexity.Summary.card_config_label_le`, `decideEntailsSummary`, `decideEntailsRegSummary`, `decideEntailsFinSummary`, and their correctness theorems
One-letter readers† (Lemma F.1)	`Complexity.Unary.isReaderHole_take`, `Complexity.Unary.firstRejected_lt_card`, `Complexity.Unary.exists_hole_length_le`, `Complexity.Unary.hole_iff_length_le_card_sub_one`

Paper result	Formal counterpart
Arity one (Theorem F.2)	`Complexity.Unary.unsafe_iff_bounded`, `Complexity.Unary.firstRejectedCut`, `Complexity.Unary.hole_iff_firstRejectedCut`, `decideEntailsUnary`, `decideEntailsRegUnary`, `decideEntailsFinUnary`, and their correctness theorems
Top-free trees, item (3), and item (4) for $a \in \{0, 1\}$ (Lemma 10.4)	`Complexity.Rejection.trace_confined`, `Complexity.FiniteTrees.trunc`, `Complexity.FiniteTrees.node_letterChildren_le_trunc`
Counter automaton (Lemma 10.5)	`Complexity.Counter.exists_runs_iff`, `Complexity.Counter.counterRecord`, `Complexity.Counter.length_counterRecord`, `Complexity.Counter.counterNFA`, `Complexity.Counter.counterNFA_final`, `Complexity.Counter.counterNFA_accepts_iff`, `Complexity.Counter.length_counterWord`
Exponentially long holes† (Proposition 10.6)	`Complexity.rejectionSystem`, `Complexity.Rejection.not_unsafe_left`, `Complexity.Rejection.unsafe_iff`, `Complexity.expSystem`, `Complexity.satisfiableFin_expSystem`, `Complexity.not_entails_expSystem`, `Complexity.not_entailsReg_expSystem`, `Complexity.not_entailsFin_expSystem`, `Complexity.expSystem_unsafe_iff`, `Complexity.expSystem_hole_iff`, `Complexity.expSystem_shortest_hole`, `Complexity.expSystem_shortest_refutesB`, `Complexity.two_pow_le_length_counterWord`
PSPACE-hardness† (Theorem 10.7)	`Complexity.Hardness.doublingCode`, `Complexity.Hardness.eq_of_doublingCode_prefix`, `Complexity.hardnessSystem`, `Complexity.entails_hardnessSystem_iff`, `Complexity.entailsReg_hardnessSystem_iff`, `Complexity.entailsFin_hardnessSystem_iff`, `Complexity.Hardness.numVars_eq`, `Complexity.hardnessSystem_length_le`, `Complexity.satisfiable_hardnessSystem`

The minimal-constructor-position result (Lemma 4.4), the equivalence and alphabet reduction in Section 14.2, and the countermodel size bounds in Appendix D.4 are not mechanized. The explicit elementary estimate in (56) is mechanized. The parts of Section 10 that are not mechanized are listed in Appendix H.6.

`RegularTransfer.countermodel_spec` records failure at the same word as a disjunction of the two sides. The regular satisfiability equivalence at the end of Appendix D.1 follows from the paper's construction; there is no separately exported declaration of that equivalence. For finite

trees, `FiniteTransfer.finite_fixedWitness` retains the word in the signed witness, whereas the decoded export `FiniteTransfer.finite_countermodel_of_unsafe` records only order failure.

H.5 Paper and Formalization

Both presentations work directly over the child alphabet and use the same label-state count and final rejected-tail, depth, and search bounds. All three domains use the same four-spine construction and selected least shape. The paper and formalization differ in representation, in the strength of some intermediate statements, and in intermediate numerical bounds.

- *Representation.* The paper uses $V \times \{0, 1\}$ for signed variables and freely names fresh spine variables. The code builds Ψ_w^4 directly in the signed finite ordinal type of size $2(k + 2|w| + 2)$ and generates its upper half by flipping the lower half. Its old-variable map is an injective embedding. The general `Ranked.Extension` accepts this embedding, rather than requiring the old signed variables to occupy an initial segment; `Spine.Closure.extension_ranked` supplies the instance. Its rank is the base-variable index, whereas the paper uses 0, j, and $|w| + 1$ for old variables, spine cuts, and fillers.
- *Proof organization.* The paper packages the bridge formula as an equivalence. The code uses an invariant for forward closure derivations and separate backward construction lemmas; the correspondence table lists both.
- *Intermediate statement strength.* The paper proves both directions of Lemma 4.8 for any old assignment satisfying the base system. The code's extension and restriction interface is stated for fixed assignments, which suffices for every witness transfer. The paper assigns suffix embeddings to fresh spine variables; `Spine.Closure.extend` and `Spine.Closure.extendBase` instead extend the fixed witness using traces of its root trees.
- *Hypotheses.* The clash equivalence `Spine.Closure.labelClash_extension_iff` holds for any flip-closed, sign-coherent system and any roots X, Y. The fixed-witness equivalence `Spine.Closure.fixedWitness_iff_not_labelClash` has the same scope. The formal reader and rejected-path property require neither flip closure nor sign coherence. The inhabited-alphabet assumptions of intermediate depth and compression results are removed by the final bounded-length theorem, as explained in Appendix H.3.
- *Graph representation.* For the least-shape graph, the paper stores upper and lower sets. The code computes the upper set as a lower set in the order-dual system; these are the same supporting-variable sets by path duality. Decoding uses `decodedGraph`, whose normalization `RGraph.normalize` of a graph g is `RGraph.mix` applied to g twice, with $2|g|^2$ states and only diagonal graph pairs reachable, whereas the paper uses a single graph state and a polarity bit; the size and depth bounds are not exported theorems.
- *Elementary compression.* The functions C, A, G and `Len` match `NormalForm.normalTypeBound`, `NormalForm.normalAllowance`, `ElementaryBound.dictionaryBound`, and `ElementaryBound.lengthBound`. The paper uses finite sets for recursive supports; `BoundedDepth.shorten_level_supported` uses lists with a no-duplicates proof and explicitly calls `dedup`. The fixed-original-dictionary invariant is the same. The paper's ordered boundary-pair inequality subtracts one; the formal recurrence uses `BoundaryPairs.boundaryCount_step_add_one`, with one added to the left side, avoiding natural-number subtraction. The normalization maps preserve geometry and start counts; neither presentation assumes global agreement of higher derived predicates after a merge. The stronger all-state agreement is used separately for column compression.
- *Periodicity proofs in the tail bound.* The paper's branching case uses a rotation argument where the code uses Fine–Wilf periodicity [Fine and Wilf 1965]. Both prove the same intermediate bounds, $4N - 2$ for a branching cycle and $2N$ for a forced cycle.

The separate kernel-checked formalization of the NP03 counterexample agrees with the argument in Appendix J. It is not part of this artifact or a dependency of the decision procedures.

H.6 Complexity Results

The module `Complexity` formalizes the combinatorial content of Section 10 and of Appendix F; the last nine rows of Table 4 give the correspondence. Importing `DeciNSSE` also exposes six further procedures. `decideEntailsSummary`, `decideEntailsRegSummary`, and `decideEntailsFinSummary` decide the three entailment relations for every arity and variance by the summary search, at the depth bound `RejectedTail.holeDepthBound` and the dictionary bound `ElementaryBound.dictionaryBound` at that depth; at arity zero their hole tests examine only the empty word (`Complexity.Summary.hole_iff_nil_of_eq_zero`). `decideEntailsUnary`, `decideEntailsRegUnary`, and `decideEntailsFinUnary` decide them at arity one, by the test of Theorem F.2. Each has a correctness theorem of the form of `decideEntails_correct`, without premises. Beyond the trusted statement of Appendix H.1, the hardness theorems and the theorems on exponentially long holes depend on the definitions of binary automata and their languages (`Complexity.BinNFA`, `Complexity.BinNFA.Accepts`) and of the constructed systems.

The complexity classes themselves are not mechanized: space-bounded machines, the space accounting of Appendix E, Savitch's theorem, and the space hierarchy theorem. Nor are the PSPACE-hardness of universality, which Theorem 10.7 cites, the polynomial-time tests of Lemma 10.1, the polynomial-time computability of the reduction and of $\mathcal{B}_\beta$, the polynomial-time procedure of Proposition F.3 with its general systems, Proposition G.1, and Example 10.8. Items (1) and (2) of Lemma 10.4 are not stated separately. The formalization differs from the paper as follows.

- *Summary search.* Proposition 10.2 describes a nondeterministic search that counts the columns of the guessed column word and stops before $col(N + j, h)$ of them. `Complexity.Summary.summarySearch` instead decides reachability of an accepting configuration in a finite graph (`Complexity.Summary.summarySearch_iff_reachable`), which it computes by saturating the set of reachable configurations (`Complexity.reachSet`). A configuration, `Complexity.Summary.Config`, holds the level, the number of current letters, the current table, and, during a derive move, the marker, the number of blocks, the summary, and the track flags. It has no column counter, since reachability in a finite graph needs no bound on the length of a path. Hence completeness uses neither column shortening nor Lemma E.2: the search reads the whole column word of the lifted blocks, and `Complexity.Summary.derivedTable_agree` composed with `BoundedDepth.Agree.derive` gives the agreement at the next level. For a monitor with N states over n letters, at most g blocks per level, and depth bound d, `Complexity.Summary.card_config_le` and `Complexity.Summary.configBits_le` bound the number of configurations by $2^{11(N+d+1)^2(n+g+1)^2}$, matching the space bound of Proposition 10.2 with $|\Gamma| = n$. For the label readers, `Complexity.Summary.card_config_label_le` gives the bound $2^{(n+1)^2 \cdot 2^{2^{20k+20}}}$, so a configuration fits in $(n+1)^2 \cdot 2^{2^{20k+20}}$ bits, that is, in $2^{2^{O(k)}} \cdot \mathrm{poly}(n)$ bits, as in the proof of Theorem 10.3. These bounds support the space argument; the saturation itself is not proved to run within that space.
- *Hardness.* The paper treats an automaton with one initial state q_0, sets $x := X_{q_0}$, and builds $\chi(\mathcal{A}')$ from the code automaton $\mathcal{A}'$. `Complexity.hardnessSystem` is built from the automaton directly and allows several initial states. Its transition variables Y_t stand for the middle states of $\mathcal{A}'$, and b stands for X_{q_f}, which is $\bot$ in every solution. Flat literals have no inequality between two variables, so a sandwich variable z, with the literals $f(x, b, \ldots, b) \le z$ and $z \le f(X_p, b, \ldots, b)$ for every initial state p, expresses that $x \le X_p$ for all of them: by decomposition, such a z exists iff these inequalities hold. Both proofs therefore establish the same theorem, for every $n \ge 2$ and

each domain: an always satisfiable covariant system of linear size entails $x \le r$ iff the automaton is universal. With the PSPACE-hardness of universality, either gives Theorem 10.7. The universal case of the paper's proof uses the identification $x = X_{q_0}$ only through $\sigma(x) \le \sigma(X_{q_0})$, and in the rejecting case $z = f(\sigma(x), \bot, \ldots, \bot)$ completes the countermodel. With several initial states, the reduction to one initial state in Section 10.2 is not needed. For an automaton with the states Q and a list δ of transitions, the formal system has $|Q| + |\delta| + 6$ variables and at most $2|Q| + 2|\delta| + 4$ literals (`Complexity.Hardness.numVars_eq`, `Complexity.hardnessSystem_length_le`); the paper's has $|Q'| + 4 = |Q| + |\delta| + 5$ variables.

- *Exponentially long holes.* `Complexity.rejectionSystem` produces the literals of $\Phi_\beta^{(n)}$, up to repetition, from any automaton all of whose states are final. Its variables x, g, and b are named as in the paper, and its variable y receives the lower literals of the initial states directly, as $Y_{q_{\text{in}}}$ does. The semantic theorems `Complexity.Rejection.unsafe_iff` and `Complexity.Rejection.hole_iff` hold for every such automaton whose language is the set of words without a fixed nonempty prefix, and `Complexity.expSystem` applies them to `Complexity.Counter.counterNFA`. In `Complexity.expSystem`, the Lean parameter named `k` corresponds to β, and the variable count $9\beta + 31$ is the paper's k. The literal count is only bounded, by $81\beta + 247$ (`Complexity.expSystem_length_le`).
- *One-letter readers.* `Complexity.Unary.exists_hole_length_le` holds for every reader over an alphabet with at most one letter, which includes the empty alphabet, whereas Lemma F.1 assumes one letter; neither needs the rejected-path property. As in the proof of Theorem F.2, the unary procedures follow the run to the first rejected cut $J < N$ (`Complexity.Unary.firstRejectedCut`) and test the single word 0^J (`Complexity.Unary.hole_iff_firstRejectedCut`).
- *Arity $n \ge 3$.* The formalization proves the hardness theorems and the theorems on exponentially long holes directly for every $n \ge 2$, with b in the arguments $2, \ldots, n-1$, and does not use Proposition G.1. The paper proves Theorem 10.7 directly as well, but reaches $n \ge 3$ in Proposition 10.6 through the embedding.

I Distinctions Used in the Proof

The following distinctions organize the arguments compared in Section 13. Each row names a tempting simplification that fails, its consequence for the proof, and where the proof handles it.

Distinction	Consequence	Where
Preserving the final reader state does not preserve the absence of admitted comparisons.	Compression shortens all blocks of a level simultaneously and records admissions and block equalities as well as transitions.	Example 5.7; Section 8
Betweenness does not make every continuation of a rejected word rejected.	The tail proof works inside the graph of reachable rejected states.	Section 6
Shortening an arbitrary prefix of a forced tail can create an admitted pair of equal suffixes.	The proof replaces the prefix before the earliest periodic onset and keeps the letter just before that onset.	Lemmas 6.5 and 6.6
A finite solution need not refute the query.	It serves only to exclude a cycle clash in the base system; the four-spine extension supplies the finite countermodel.	Theorem D.5

J A Counterexample to NP03 Proposition 16

This appendix uses a binary covariant constructor f and the child alphabet $\{0, 1\}$. NP03's binary child indices $1, 2$ become $0, 1$ here. A separate kernel-checked formalization of this counterexample, not part of the artifact (Appendix H), agrees with the argument below.

J.1 Cap Automata and the Back-Translation

A *cap automaton* consists of a finite automaton and an additional relation of *P-edges*, written $q' \rightsquigarrow q$. Write $q \xrightarrow{w} q'$ for a run of the underlying finite automaton. For one initial state q_0 and final-state set F, its language is

$$\mathcal{L}(P) = \{w \mid q_0 \xrightarrow{w} q,\ q \in F\} \ \cup\ \{\pi u \mid q_0 \xrightarrow{\pi} q \xrightarrow{\rho} q',\ q' \rightsquigarrow q,\ u \in \mathrm{pr}(\rho^*)\}.$$

Here ρ^* is the set of finite powers of ρ, and pr takes all prefixes (Section 2.1). Thus a P-edge accepts prefixes of repetitions of a word read between its target and its source. We allow $\rho = \varepsilon$, as in NP03's Definition 1; the example below has no cap at all and also works if roots are required to be nonempty. A cap automaton is *complete* if every word has a run from an initial state, whether or not that run accepts. NP03's Definition 3 calls it *restricted* when the following four conditions hold.

(1) *Strictly epsilon free:* there is one initial state and no epsilon transition.
(2) *Gap universal:* if $q \xrightarrow{i} q'$ with $q \notin F$ and $q' \in F$, then every word has a run from q' to a final state. This condition uses ordinary finite-automaton acceptance.
(3) *Strictly cap:* if $q \xrightarrow{\rho} q' \rightsquigarrow r$ with $\rho \neq \varepsilon$, then $q \in F$.
(4) *Shuffled:* if $q_0 \xrightarrow{\pi} q$, $q_0 \xrightarrow{\pi} q'$, and $q' \rightsquigarrow q$ with $q \neq q'$, then $\pi\tau \in \mathcal{L}(P)$ for every word τ.

NP03's Definition 5 introduces distinct variables $l(q), r(q)$ for each state, and defines $J(P) = (\phi_P \models^? l(q_0) \le r(q_0))$. The binary rules needed here are as follows. For each choice of successors $q \xrightarrow{0} p_0$ and $q \xrightarrow{1} p_1$, include

$$l(q) \le f(l(p_0), l(p_1)) \quad (q \in F), \qquad f(r(p_0), r(p_1)) \le r(q).$$

A transition $q \xrightarrow{i} q'$ with $q \notin F$ and $q' \in F$ contributes $r(q') = \top$. Finally, $q \xrightarrow{i} q' \rightsquigarrow p$ with $q' \neq p$ contributes

$$l(q) \le f(r(p), t) \quad (i = 0), \qquad l(q) \le f(t, r(p)) \quad (i = 1), \qquad t = \top,$$

where t is a fresh filler variable. These last constraints are the binary form of the notation $l(q) \le i[r(p)]$ in Table 7. NP03 does not fix the other argument of $i[\cdot]$. Leaving t unconstrained gives an equivalent judgment: it occurs only inside this upper bound, so raising it to $\top$ preserves every solution without changing x, y. Conversely, a solution with $t = \top$ satisfies the unconstrained version. Thus both versions have the same solutions on the original variables in each tree domain.

In Table 7, the finality side conditions of the left and top rules are conveyed by typographic marks. We read the left rule as applying at final sources and the top rule at final targets, as NP03's proof of Proposition 16 uses in the auxiliary claim in case (3) and the gap-universality argument in case (3b). Dropping these conditions adds left bounds at nonfinal sources and top equations at nonfinal targets. We prove the counterexample for both readings and every constraint system between them, so the conclusion does not depend on these marks.

J.2 The Six-State Automaton

Let the states be q_0, A, B, C, C_2, U, with initial state q_0, final states q_0, A, U, and the single P-edge $C \rightsquigarrow B$. The following table lists all letter transitions; an entry denotes the set of possible successors.

State	on 0	on 1
q_0	$\{U\}$	$\{A, B\}$
A	$\{U\}$	$\{C\}$
B	$\{U\}$	$\{C_2\}$
C	$\{U\}$	$\{C_2\}$
C_2	$\{U\}$	$\{C_2\}$
U	$\{U\}$	$\{U\}$

Every state has at least one successor on each letter, so P is complete. It has one initial state and no epsilon transitions. Every transition from a nonfinal to a final state enters U, which accepts every continuation by ordinary acceptance. The only states with a nonempty run to the P-edge source C are A and q_0, both final. Finally, B is reachable from q_0 only on 1, and C only on 11. No word reaches both endpoints of the P-edge, so shuffledness is vacuous. This checks all four restrictions.

Every word containing 0 reaches U. On 1^j, the reachable state sets are $\{q_0\}$ for $j = 0$, $\{A, B\}$ for $j = 1$, $\{C, C_2\}$ for $j = 2$, and $\{C_2\}$ for $j \geq 3$. No run from B reaches C, so no P-edge contributes to acceptance. Consequently

$$\mathcal{L}(P) = \{0,1\}^* \setminus \{1^j \mid j \geq 2\}, \qquad 11 \notin \mathcal{L}(P).$$

J.3 The Generated Constraints and Their Solutions

Write $l_q = l(q)$ and $r_q = r(q)$, and set $x = l_{q_0}$ and $y = r_{q_0}$. With the final-state marks, Table 7 gives the following conjunction $\phi_{\min}$; the grouping records which rule supplies each literal.

$$\begin{aligned}
\text{left:}\quad & x \leq f(l_U, l_A), & & x \leq f(l_U, l_B),\\
& l_A \leq f(l_U, l_C), & & l_U \leq f(l_U, l_U);\\
\text{right:}\quad & f(r_U, r_A) \leq y, & & f(r_U, r_B) \leq y,\\
& f(r_U, r_C) \leq r_A, & & f(r_U, r_{C_2}) \leq r_B,\\
& f(r_U, r_{C_2}) \leq r_C, & & f(r_U, r_{C_2}) \leq r_{C_2},\\
& f(r_U, r_U) \leq r_U;\\
\text{top:}\quad & r_U = \top;\\
\text{P-edge:}\quad & l_A \leq f(t, r_B), & & t = \top.
\end{aligned}$$

In particular, the P-edge literal comes from $A \xrightarrow{1} C \rightsquigarrow B$. Without the final-state marks, the system $\phi_{\max}$ adds precisely

$$l_B \leq f(l_U, l_{C_2}), \quad l_C \leq f(l_U, l_{C_2}), \quad l_{C_2} \leq f(l_U, l_{C_2}), \quad r_{C_2} = \top.$$

Set every l_q to $\bot$ and every r_q and t to $\top$. Each upper bound then has $\bot$ on the left, each lower bound has $\top$ on the right, and all top equations hold. This is a finite solution of $\phi_{\max}$, hence of any system ϕ with $\phi_{\min} \subseteq \phi \subseteq \phi_{\max}$.

We next prove $\phi_{\min} \models x \leq y$ over arbitrary trees. Fix a solution σ and abbreviate its values by $X = \sigma(x)$, $Y = \sigma(y)$, $a = \sigma(l_A)$, $b = \sigma(r_B)$, and $c = \sigma(r_{C_2})$. The lower bounds give

$$f(\top, c) \leq c, \qquad f(\top, c) \leq b, \qquad f(\top, b) \leq Y.$$

They imply that every label of Y is either $\top$, or f at a path of the form 1^j. Here is a direct verification. From $f(\top, c) \leq c$, c is either $\top$ or $f(\top, c')$ with $c \leq c'$. In the latter case, substitution gives $f(\top, c') \leq c'$. Repeating this argument along any finite path proves the asserted label property for c. That property is preserved by $s \mapsto f(\top, s)$ and by passing to a larger tree: at a constructor root the larger tree either becomes $\top$ or keeps a top left child and a larger right child. Induction on

path length justifies the latter assertion even for infinite trees. Applying these two preservation facts to the remaining lower bounds proves the property for Y.

For a tree s, let $s|_w$ denote its subtree at w, when w belongs to its domain, and write $s(w)$ for its label there. We check the labels of X and Y at every common path w, which suffices for covariant subtyping. If $Y(w) = \top$, the labels compare. Otherwise, $w = 1^j$ and $Y(w) = f$. For $j = 0$, the bound $X \le f(\sigma(l_U), a)$ excludes a top root of X. For $j = 1$, that bound and $a \le f(\sigma(l_U), \sigma(l_C))$ exclude a top label of X at 1. For $j \ge 2$, the existence of 1^j in X makes its root and its child at 1 constructors. The bounds

$$X \le f(\sigma(l_U), a), \qquad a \le f(\top, b), \qquad f(\top, b) \le Y$$

then force a to be a constructor too: $X|_1 \le a$ excludes $\bot$, and the upper bound on a excludes $\top$. Since 1^j also exists in Y, its root is a constructor. Decomposing these three inequalities gives the subtree inequalities

$$X|_{11} \le a|_1 \le b \le Y|_1.$$

By transitivity, $X|_{11} \le Y|_1$. Comparing at their common path 1^{j-2} yields $X(1^j) \le Y(1^{j-1}) = f$, because 1^{j-1} is a proper prefix of a path in Y. Thus $X(w) \le Y(w)$ in every case, proving $X \le Y$. Entailment persists when literals are added, so every intermediate reading ϕ is both satisfiable and entails $x \le y$. The proof covers arbitrary trees and hence also regular and finite trees.

J.4 Forward Acceptance and the Failed Proof Step

Write $P_{\mathrm{l}}(J(P))$ and $P_{\mathrm{r}}(J(P))$ for the left and right cap automata obtained from $J(P)$ by NP03's Table 3. The rules used here can be read directly on these constraints: support between variables is trivial, as in NP03's Lemma 21. States are pairs (u, v) of variables or a blank coordinate, with initial state (x, y), and an accepting sink. An upper bound $u \le f(u_0, u_1)$ and a lower bound $f(v_0, v_1) \le v$ permit a transition on i to (u_i, v_i); either coordinate may instead descend alone, leaving the other blank. The left automaton makes a state final when its left coordinate has an upper constructor bound; the right automaton uses a lower constructor bound on the right coordinate. The bottom, top, and reflexivity rules send ready states to the accepting sink on every letter. In particular, a right coordinate constrained to $\top$ is ready. The left P-edge rule includes $(u, \square) \rightsquigarrow (v, u)$, where $\square$ denotes a coordinate that may be a variable or blank.

The left automaton of $J(P)$ has the transitions and P-edge

$$(x, y) \xrightarrow{1} (l_A, r_B) \xrightarrow{1} (r_B, \text{blank}) \rightsquigarrow (l_A, r_B).$$

The first transition combines the upper bound using l_A with the lower bound using r_B; the second uses the P-edge literal. This cap uses only descend steps and a P-edge. Its entry state is final because $l_A \le f(l_U, l_C)$ is an upper constructor bound. Hence $11 \in \mathcal{L}(P_{\mathrm{l}}(J(P)))$ under either finality convention for Table 3, whereas $11 \notin \mathcal{L}(P)$. The inclusion $\mathcal{L}(P_{\mathrm{l}}(J(P))) \subseteq \mathcal{L}(P)$, claimed in case (3) of NP03's proof of Proposition 16, therefore fails independently of that convention, even if cap entry states are required to be final.

To show universality, we additionally count ready states as final. NP03's proofs of Propositions 12 and 14 use this convention: Proposition 12 uses bottom, top, and reflexivity readiness to accept the prefix itself; Proposition 14 uses top-readiness for gap universality and reflexivity for shuffledness. NP03's own Section 4 example witnesses the need for this reading of Table 3. For the unary judgment $z = \top \wedge f(z) \le y \models^{?} x \le y$, NP03 states that every path in 1^+ is both left safe and right safe, using its child index 1. Read literally, Table 3 reaches only (blank, z) on 1; that state is not final in either automaton, and there is no cap, so both reject 1.

Under this convention, the displayed cap accepts $1\,\mathrm{pr}(1^*)$, and (x, y) is final, accepting ε. For a word containing 0, let the right coordinate follow any run of P through the lower constructor bounds.

At its first 0, it reaches $r_U = \top$, so this path and all its extensions are accepted. The right automaton is universal as well: since every state of P has a successor on each letter, a right-coordinate run exists on every word, and every r_q has a lower constructor bound, so every resulting state is final. These runs remain available under every intermediate reading of Table 7. They verify directly the conclusion also supplied by NP03's Propositions 1 and 3 (soundness, Proposition 8; completeness, Proposition 11):

$$\mathcal{L}(P_{\mathrm{l}}(J(P))) = \{0,1\}^* \neq \mathcal{L}(P), \qquad \mathcal{L}(P_{\mathrm{r}}(J(P))) = \{0,1\}^*.$$

The convention is needed only for this universality conclusion; acceptance of 11 already refutes Proposition 16.

Case (3d) of NP03's proof writes the cap's entry state as $(l(q_i), r(q_i))$, assuming that both coordinates follow the same run of P. In this example the entry state is $(l(A), r(B))$. Both A and B are reached on 1, but only the run through A can next reach C, whose P-edge returns to B. The back-translation combines coordinates of different runs; the original automaton has no run $B \xrightarrow{1} C$ to close that cap. Shuffledness does not apply: it tests endpoints reached on the same word, whereas B and C occur at different cuts here.

J.5 Consequences for the Published Reductions

Theorem 4 explicitly invokes Proposition 16 to reduce restricted cap-automaton universality to entailment. The counterexample invalidates that construction and therefore the converse reduction underlying Theorem 1. This is the implication from decidability of NSSE to decidability of restricted cap-set universality. It does not rule out a different reduction between the two decision problems. Corollary 1, asserting equivalence of arities at least two and of finite, regular, and arbitrary tree classes, also relies on that converse. Its arity part additionally uses Proposition 17's alphabet extension and its alphabet restriction, which encodes each letter by two letters of a smaller alphabet. Theorem 4, Theorem 1, and Corollary 1 themselves are not refuted; their published proofs are incomplete. We claim no defect in Proposition 17.

The forward reduction of Theorem 2 first checks satisfiability in the chosen tree domain, then constructs the two Table 3 automata. Its soundness and completeness proofs precede the back-translation and do not invoke Proposition 16. The unary result, Corollary 3, follows in NP03 from Corollary 2: cap-set membership is expressed by existential word equations with regular constraints, and universality adds a universal word quantifier. Although the text introduces this derivation through Theorem 1, it uses only the forward reduction to cap sets. For a unary alphabet, word equations and regular constraints become constraints in Presburger arithmetic on lengths, so this quantified formulation is decidable. Neither the converse reduction nor Proposition 17 is needed.